



UNIVERSITAT D'ANDORRA

TREBALL FINAL de CARRERA

Curs 2025 2026

Per optar al títol de
Bàtxelor en informàtica

Architecting a modular 3D online party game
From MVP concept to vertical slice in Unity

Autor *Eric Torrontera Ruiz*

Tutors *Jan Sau Batlle i Josep Ribó Ferriz*

Sant Julià de Lòria, 9 de juny 2026

Dedicatòria i agraïments

En primer lloc, vull expressar el meu sincer agraïment als meus tutors, el Sr. Jan Sau Batlle i el Sr. Josep Ribó Ferriz, pels seus consells, la seva paciència i per guiar-me en la definició d'aquest projecte. Les seves orientacions han estat clau per mantenir el focus i assegurar la viabilitat d'aquest videojoc.

A la Sra. Montse Pellicer Bullich, per ser una font constant d'inspiració i suport. Li agraeixo profundament haver-me transmès la passió per la Programació Orientada a Objectes i els patrons de disseny, així com la seva insistència a adoptar una visió analítica rigorosa. Gran part de l'arquitectura modular d'aquest treball és fruit de les seves lliçons i del disseny de diagrames UML.

Al Sr. Alfred Alcón Beltrán, per introduir-me en el complex i fascinant món de les xarxes i l'arquitectura servidor-client. Els seus coneixements han estat la base sobre la qual s'ha pogut plantejar el repte del multijugador en línia en aquest projecte.

Al meu germà, el Sr. Isaac Torrontera Ruiz, KAM d'Andorra Telecom, pel seu suport tècnic i per ser una font de motivació constant. Els seus coneixements sobre la infraestructura de xarxa i la connectivitat han estat de gran ajuda per comprendre millor l'entorn d'execució del projecte i resoldre dubtes estructurals sobre el funcionament de la xarxa.

Finalment, vull dedicar aquest treball a la meva família i amics pel seu suport incondicional. Així mateix, faig extensiu el meu agraïment als companys i professors de la Universitat d'Andorra amb qui he compartit aquest procés, tant en l'aprenentatge de la gestió comptable com en el desenvolupament tècnic del projecte, especialment per la seva col·laboració en la implementació de les físiques personalitzades del joc i per fer aquest camí més enriquidor.

Fitxa del treball final

Títol

Architecting a modular 3D online party game. From MVP concept to vertical slice in Unity

Autor/a

Eric Torrontera Ruiz (etorrontera@uda.ad)

Tutors

Jan Sau Batlle i Josep Ribó Ferriz

Àrea del treball

Videojocs

GitHub

https://github.com/Andormix/TFG_UDA_Modular_3D_Party_Game_MVP_in_Unity_URP

Paraules clau

Modular architecture, Party game, Unity 6

Summary

The main objective of this Final Degree Project (TFG) in Computer Engineering is the design and implementation of a functional vertical slice video game, serving as a practical synthesis of the technical skills and theoretical knowledge acquired throughout the degree. This project provides a comprehensive first look into the professional landscape of indie game development, focusing on software architecture, networking, and project management within a collaborative student environment.

The project is realized through a combination of industry-standard tools and a mix of open-source and commercially licensed resources. Unity was selected as the primary development engine due to its robust ecosystem, paired with Visual Studio 2026 (Community Edition) for advanced C# scripting and debugging. To ensure high-quality visual and auditory standards while adhering to legal requirements, various assets from the Unity Store were integrated, ensuring all components—including 3D models, textures, and SFX—are used under valid commercial licenses.

Technically, the development prioritizes a "multiplayer-first" approach using the Netcode for GameObjects (NGO) framework. This architectural decision ensures server-authoritative logic and efficient data synchronization, replacing heavy JSON-based systems with compact network structs to optimize performance. The workflow followed professional standards such as Conventional Commits and Agile methodologies, organized into sprints to manage complex features like physics-based item behavior and TPV payment systems.

The final result of this project is a Minimum Viable Product (MVP) of a basic Party Game. The game is fully playable and features a "prepare-serve-collect" loop. Designed with scalability in mind, the current build provides a solid foundation that is easily expandable, allowing for future iterations, additional mechanics, and further performance refinements.

Taula de continguts.

Capítol 1: Pla de treball.....	12
1.1 Context i justificació.....	12
1.1.1 Anàlisi de riscos i definició de l'abast.....	13
1.1.2 Definició de l'abast i objectius del projecte	13
1.2 Objectius	14
1.2.2 Objectius específics.....	14
1.3 Compromís amb els ODS	15
1.3.1 ODS amb impacte positiu	15
1.3.2 ODS amb impacte neutre.....	15
1.3.3 ODS amb impacte negatiu i mesures de mitigació	15
1.4 Enfocament i metodologia de treball	16
1.4.1 Metodologia àgil (Scrum/Kanban).....	16
1.5 Planificació	19
1.5.1 Taula de tasques i fites	19
1.6 Productes finals.....	20
1.6.1 Disseny i arquitectura de la solució	20
1.6.2 Desenvolupament del programari (Executables de Windows)	20
1.6.3 Documentació de qualitat i evidències	21
1.7 Recursos.....	21
1.7.1 Recursos materials (Hardware)	21
1.7.2 Recursos de programari (Software).....	22
1.7.3 Recursos immaterials.....	22
1.7.4 Valoració econòmica.....	22
1.8 Anàlisi de riscos.....	23
1.9 Estructura de la memòria	25
Capítol 2: Anàlisi	26

2.1 Context i situació actual	26
2.1.1 Descripció del projecte i arquitectura tècnica	26
2.1.2 Problemàtiques detectades, conseqüències i solucions.....	28
2.2 Estat de l'art	29
2.2.1 Referents del gènere.....	29
2.2.2 Estat de la tecnologia	31
2.2.3 Tendències futures.....	31
2.3 Objectius tècnics.....	32
2.3.1 Objectius del Producte Mínim Viable (MVP).....	33
2.3.2 Objectius Post-MVP (Escalabilitat)	34
2.3.3 Objectius transversals.....	34
2.4 Requisits tècnics	35
2.4.1 Requisits funcionals (RF)	35
2.4.2 Requisits no funcionals (RNF)	36
2.4.3 Requisits operacionals (ROp)	36
2.4.4 Requisits de qualitat (RQ)	36
Capítol 3: Disseny	37
3.1 Arquitectura	37
3.1.1 Visió general de la solució: Hybrid-Core.....	37
3.1.2 Funcionament general: Integració i fluxos	39
3.1.3 Components principals i relació amb requisits.....	40
3.1.4 Models i patrons de disseny utilitzats	41
3.2 Components	42
3.2.1 Component: Capa de presentació (UI / Menús / HUD)	45
3.2.2 Component: Coordinador de sessió	46
3.2.3 Component: Persistència i progressió (offline)	47
3.2.4 Component: Pool / Selector de mapes.....	48
3.2.5 Component: Core de gameplay (Core loop).....	49

3.2.6 Component: Domini de taules (FSM de servei)	50
3.2.7 Component: Seating NPC (ambient pre-activació).....	51
3.2.8 Component: Sistema de comandes.....	52
3.2.9 Component: TPV (pagament) — UI local + validació autoritària.....	53
3.2.10 Component: Sistema de puntuació i combo.....	54
3.2.11 Component: Capa d'autoritat i sincronització	55
3.2.12 Component: Planificador de taules (Scheduler).....	57
3.3 Player - Control de jugador i interacció contextual (WASD / E / Q)	58
Capítol 4: Implementació	62
4.1 Arquitectura	62
4.1.2 Flux general de funcionament	63
4.1.3 Decisions tecnològiques clau	65
4.2 Components	65
4.2.1 Coordinació de sessió i cicle de vida	65
4.2.2 Lobby, relay i bootstrap multijugador.....	67
4.2.3 Gestor de partida i estat de match.....	68
4.2.4 Personalització i sincronització de jugador.....	69
4.2.5 Core gameplay: interacció, taules, TPV i puntuació	70
4.2.6 UX, tutorial i controls ampliat	71
4.2.7 Persistència, campanya i contextos de runtime.....	72
4.3 Desenvolupament i execució	73
4.3.1 Entorn tecnològic de desenvolupament.....	73
4.3.2 Entorn necessari d'execució	74
4.3.3 Posada en marxa (desenvolupament).....	75
4.3.4 Procés de build i distribució	75
4.3.5 Annexos de suport (detall tècnic)	75
Capítol 5: Proves	76
5.1 Definició.....	76

5.2 Avaluació de requisits	77
5.2.2 Resultats de proves de networking i robustesa	78
5.2.3 Resultats de regressió i rendiment.....	79
5.3.2 Valoració global	81
Capítol 6: Distribució, manteniment i evolució	82
6.1 Distribució.....	82
6.1.1 Empaquetat i canals de distribució.....	82
6.1.2 Procés d'instal·lació i actualització en fase beta.....	82
6.2 Manteniment	83
6.2.1 Model de suport	83
6.2 Evolució	84
6.3.1 Línies de treball futur.....	84
6.3.2 Calendari de versions previst	84
Capítol 7: Estratègia de màrqueting i llançament.....	85
7.1 Cronograma de visibilitat.....	85
7.2 Canals de difusió	86
Capítol 8: Conclusions	88
Capítol 9: Glossari.....	89
Capítol 10: Bibliografia	93
Capítol 11: Llicències del projecte.....	97
11.1 Inventari de recursos de pagament (Unity Asset Store).....	97
11.2 Declaració de compliment.....	98
11.3 Paquets CC0 utilitzats al projecte	99
11.3 Programari, eines i llicències de desenvolupament	100
11.4 Política de propietat intel·lectual del projecte	100
12 Annexos	101
Annex AT-01: Flux de sessió i teardown	101
Robustesa i recuperació (timeouts, fail-closed, teardown unificat)	108

Annex AT-02: Integració Lobby/Relay	109
Annex AT-03: Estat de partida i replicació.....	113
Integritat d'ítems (invariant atòmic)	117
Annex AT-04: Persistència i contextos.....	118
Annex AT-05: Proposta d'Arquitectura Basada en Esdeveniments	121
Annex AT-06: Desenvolupament realitzat i evidències de progrés (Primera fita)	123
Arquitectura de Programari: Desenvolupament Unificat (Hybrid-Core).....	123
Full de Ruta Tècnic i Estat de les EPICS	124
Desenvolupament del Sistema Multijugador (Networking).....	127
Evidències de Proves (Testing).....	127
Incidències i Decisions Tècniques Preses	127
Observacions del desenvolupador:	127
Annex AT-10: Diagrama de GANTT	129
Taula de tasques i fites	129
Annex AT-11: Sistema de Puntuació.....	132
Annex AT-12 Patrons implementats al projecte	134

Índex de figures

Figura 1. Relació entre gèneres, llançaments i ingressos mitjans a Steam.	12
Figura 2. Planificació de Sprints de desenvolupament.	16
Figura 3. Detall del taulell Kanban analògic. Cicle de desenvolupament.	17
Figura 4. Comparativa de rendiment en la serialització de dades del NetworkList.	23
Figura 5. Diagrama de flux de l'accés als modes de joc i arquitectura de xarxa.	27
Figura 6. Interfície de joc i dinàmica de cuina cooperativa a Overcooked! 2.	29
Figura 7. Fase de planificació, automatització i gestió del restaurant a PlateUp!.	30
Figura 8. Mecànica cooperativa a Moving Out.	30
Figura 9. Diagrama de tipus C4-like Context	38
Figura 10. Activity Diagram (Flux global de modes)	39
Figura 11. Wireflow d'alt nivell del projecte.	45
Figura 12. Diagrama FSM de sessió	46
Figura 13. Data Flow Diagram.	47
Figura 14. Diagrama de decisió (selecció de mapa)	48
Figura 15. Loop de gameplay (alt nivell)	49
Figura 16. Diagrama d'estats de taula (alt nivell).	50
Figura 17. Flux de seating/cleanup.	51
Figura 18. Lifecycle de comanda (alt nivell).	52
Figura 19. Seqüència de pagament (alt nivell)	53
Figura 20. Flux de decisió de scoring (resum)	54
Figura 21. Flux d'autoritat (client-servidor).	55
Figura 22. Messaging System [Il·lustración]. Unity Documentation.	55
Figura 23. Messaging System [Il·lustración]. Unity Documentation.	56
Figura 24. Messaging System [Il·lustración]. Unity Documentation.	56
Figura 25. Activació temporal de taules.	57
Figura 26. Diagrama d'activitat (entrada → decisió contextual → acció)	60
Figura 27. Diagrama de seqüència (interacció amb validació autoritària)	61
Figura 28. Diagrama d'arquitectura de components	63
Figura 29. Diagrama de flux online de inici de sessió	64
Figura 30. Imatge del component Session Coordinator a Unity GO.	66
Figura 31. Imatge del component Lobby a Unity GO.	67
Figura 32. Component de gestor de partida i estat de match a Unity GO.	68
Figura 33. Fragment de codi. Personalització i sincronització de jugador.	69
Figura 34. Component Recipe Manager a Unity GO.	70
Figura 35. Table State Pattern a l'entorn de Unity 6.	71
Figura 36. Scene view. Network status Ownership	73
Figura 37. Monitoratge del bandwidth i dades consumides.	73
Figura 38. Ingame Skins Tab Scene View.	74
Figura 39. Visualització del videojoc en Development Build.	78
Figura 40. Visualització del videojoc. Comandes i fase de pagament.	79
Figura 41. Interfície web del portal comunitari amb la secció de DevLog.	83
Figura 42. Anàlisi comparatiu d'enginyeria inversa. FAR: Lone Sails.	95
Figura 43. Integració simultània de shaders d'herba dinàmica.	95
Figura 44. Implementació d'efectes atmosfèrics de boira a INSIDE.	96
Figura 45. Anàlisi de referents visuals: integració d'efectes atmosfèrics a INSIDE.	96

Figura 46. Inventari de llicències i recursos adquirits a la Unity Asset Store.....	98
Figura 47. Inventari de llicències i recursos adquirits a Itchio.	99
Figura 48. Arquitectura funcional del flux de sessió	102
Figura 49. Interfície de joc durant el flux de creació d'un lobby.	104
Figura 50. Interfície de joc durant el flux d'unió al lobby.	105
Figura 51 Visualització de sessió en temps real.	106
Figura 52. Diagrama de flux del protocol de recuperació d'errors i neteja asíncrona ..	108
Figura 53. Flux detallat de creació de lobby (host)	110
Figura 54. Dades de lobby rellevants – Join modes.	110
Figura 55. Interfície de joc durant el flux d'unió al lobby.	111
Figura 56. Diagrama d'estats de partida.....	114
Figura 57. Diagrama de replicació d'estat de taules.....	115
Figura 58. Interfície de joc durant replicació d'estat de taules.	115
Figura 59. Interfície de joc durant la publicació de resultats de match	116
Figura 60. Diagrama d'integritat d'ítems (invariant atòmic)	117
Figura 61 Interfície de joc – Campaign menu.	119
Figura 62. Historial de planificació de sprints de desenvolupament.....	126
Figura 63. Cronograma i planificació temporal del projecte. Gantt.	129

Capítol 1: Pla de treball

1.1 Context i justificació

El present projecte s'emmarca en el disseny i desenvolupament d'un videojoc 3D pertanyent al gènere dels "Online Party Games". Aquest gènere es caracteritza per oferir sessions de joc breus, una alta rejugabilitat i una mecànica central basada en reptes ràpids o minijocs. L'arquitectura del programari s'ha plantejat inicialment per a una experiència individual de simulació, tot i que es dissenyarà sota un paradigma multiplayer-first per facilitar-ne l'expansió cap a un mode cooperatiu en línia.

La decisió d'optar per aquest gènere no és casual, sinó que respon a una anàlisi exhaustiva de les dades de mercat recollides durant els darrers anys a la plataforma Steam que és on llançarem el producte. Mitjançant l'estudi de mètriques clau, s'ha identificat que el nínxol dels "Online Party Games" presenta un equilibri òptim en la relació "Game Genre Median Earnings vs. Number of Games Released".

Aquesta posició favorable en el mercat indica una demanda sòlida per part dels usuaris davant d'una saturació moderada de títols, convertint-lo en un segment estratègic per al desenvolupament independent. A més, la naturalesa modular d'aquests jocs permet una iteració constant del contingut, alineant-se amb les tendències actuals de consum on la immediatesa i la interacció social són pilars fonamentals de l'experiència de l'usuari.

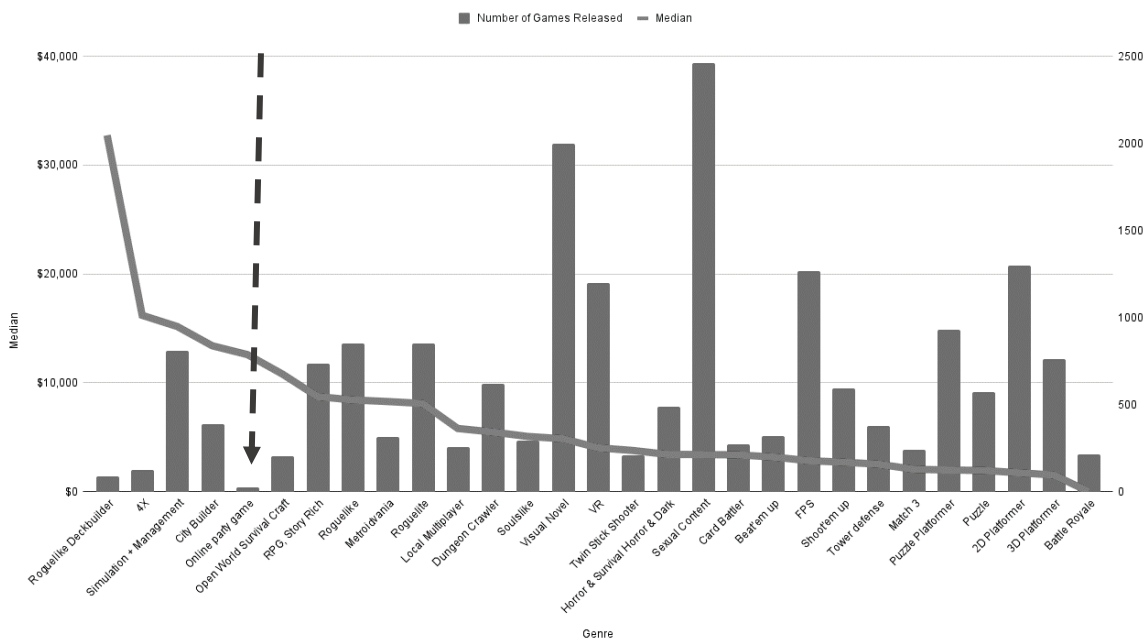


Figura 1. Relació entre gèneres de videojocs, volum de llançaments i ingressos mitjans a Steam.

Font: Carless (2024) / Game DiscoverCo.

Aquí veiem clarament com els jugadors de Steam realment si tenen una preferència. Exploterem aquest nínxol marcat amb una fletxa.

1.1.1 Anàlisi de riscos i definició de l'abast

El desenvolupament d'un videojoc 3D amb ambició multijugador implica una sèrie de complexitats que poden comprometre la viabilitat del projecte si no es gestionen correctament. S'han identificat tres àrees crítiques:

- **Complexitat tècnica en entorns 3D:** La implementació d'un control de personatge fluid, la gestió de físiques, la sincronització d'animacions i l'optimització del rendiment mitjançant *Managers* i *Scriptable Objects* representen un repte constant. Una gestió deficient d'aquests elements podria derivar en una experiència de joc poc polida (*unpolished*) o en problemes d'eficiència tècnica.
- **Gestió del multijugador en línia:** L'arquitectura de xarxa afegeix una capa de dificultat elevada pel que fa a la latència, la sincronització d'estats i la gestió de desconnexions. Una implementació inestable de l'apartat *online* podria transformar la interacció en una experiència frustrant i no funcional.
- **Càrrega de producció de continguts:** La creació d'assets 3D, el disseny d'interfícies (UI), la composició sonora i el postprocessat són tasques que consumeixen un temps considerable que podria restar-se de la programació core. Existeix el risc d'abastar un abast excessiu que impedeixi completar els objectius mínims en els terminis establerts.

1.1.2 Definició de l'abast i objectius del projecte

Aquest projecte es concreta en el desenvolupament d'un producte mínim viable (MVP, de l'anglès Minimum Viable Product) que consisteix en un prototip polit d'un Party Game en 3D, amb un mode individual complet i una selecció de dos a quatre mapes o minijocs. L'arquitectura s'ha dissenyat sota el paradigma de les bones pràctiques de programació per garantir un codi net, mantenible i escalable; d'aquesta manera, es facilita la implementació d'un mode cooperatiu en línia com a objectiu avançat en funció de la disponibilitat temporal del cronograma.

1.2 Objectius

L'objectiu principal d'aquest projecte és el disseny i la implementació d'un videojoc en 3D enquadrat en el gènere Party Game, utilitzant el motor Unity i el canal de renderitzat URP (Universal Render Pipeline). El projecte busca oferir una experiència de joc individual completa i polida, construïda sobre una arquitectura preparada per a multijugador (multiplayer-ready) que permeti, en funció del cronograma, la integració d'un mode cooperatiu en línia estable.

Aquest treball representa la culminació dels estudis de grau a la Universitat d'Andorra i permet aplicar de forma transversal coneixements de patrons de disseny de programari (com ara Singleton, Observer, State o Factory), refactorització de codi i metodologies de testeig (caixa blanca i caixa negra). Així mateix, el projecte serveix com a nexa d'unió amb el seminari de desenvolupament de videojocs (Game Development) realitzat de forma paral·lela, amb la projecció a llarg termini de publicar el títol a la plataforma Steam.

1.2.2 Objectius específics

Dominar l'Universal Render Pipeline (URP) de Unity: Aprofundir en les capacitats gràfiques i d'optimització del motor per assolir un acabat professional, posant èmfasi en:

- **Confecció d'escenes:** Gestió avançada de *Prefabs*, sistemes d'UI dinàmics i control de càmeres amb *Cinemachine*.
- **Il·luminació i atmosfera:** Configuració de *Skyboxes*, il·luminació global (*baked lightmaps*) i sistemes de partícules per a efectes ambientals (*Godrays*, boira lineal/exponencial).
- **Materials i postprocessat:** Ús de materials PBR i aplicació de tècniques de *Tonemapping*, *LUTs* i *Shaders* personalitzats.
- **Optimització tècnica:** Implementació de *Scriptable Objects* per a la gestió de dades i tècniques de *rendering* eficients per garantir un rendiment estable.

Desenvolupar el nucli de joc (Game core): Implementar el flux complet d'un *Party Game*, des dels menús principals i la lògica de sessió fins al sistema de resultats i progressió, assegurant un MVP (Producte Mínim Viable) sòlid i escalable.

Arquitectura modular de sistemes i minijocs: Dissenyar sistemes reutilitzables (temporitzadors, sistemes de puntuació, gestors d'interfície) mitjançant patrons de disseny de programari. L'objectiu és que cada nou minijoc es pugui integrar de forma modular sense comprometre l'estabilitat del codi existent.

Integració de capacitats multijugador (Objectiu avançat): Implementar un sistema de *Lobbies* i sincronització de partida en línia mitjançant la integració de *Steamworks* o *Unity Relay Transport*. Això inclou la gestió de latències, personalització de jugadors i control d'estat de la xarxa per garantir una experiència cooperativa fluida.

1.3 Compromís amb els ODS

En aquest apartat s'analitza l'impacte del Projecte Final de Grau en relació amb l'Agenda 2030 de les Nacions Unides. S'han categoritzat els objectius segons la naturalesa de la seva incidència en el projecte:

1.3.1 ODS amb impacte positiu

- **ODS 10 – Reducció de les desigualtats:** El projecte aposta per la inclusió digital mitjançant la implementació de funcionalitats d'accessibilitat bàsica. Això inclou la personalització del mapatge de controls (*key binding*), l'ús de subtítols i modes visuals adaptats per a persones amb daltonisme, garantint que el programari sigui usable per a un espectre més ampli d'usuaris.
- **ODS 11 – Ciutats i comunitats sostenibles i ODS 12 – Consum i producció responsables:** El videojoc integrarà en un futur una mecànica de *gameplay* centrada en la gestió de residus (plàstic, cartró, vidre i rebuig). Mitjançant un sistema de *feedback* immediat i recompenses per puntuació, el joc actua com una eina d'aprenentatge informal que fomenta el civisme urbà i la separació correcta de residus, promovent hàbits de consum responsable i sostenibilitat ambiental en un entorn simulat. Actualment hem incorporat parereres de reciclatge per fomentar la idea de reciclatge als usuaris.

1.3.2 ODS amb impacte neutre

- **ODS 9 – Indústria, innovació i infraestructura:** El desenvolupament d'aquest producte digital es realitza sota estrictes criteris d'enginyeria del programari. L'ús de metodologies àgils, el prototipat iteratiu i l'aplicació de patrons de disseny contribueixen a la innovació en el sector de l'oci digital, tot i que el seu impacte directe en la infraestructura física és limitat.

1.3.3 ODS amb impacte negatiu i mesures de mitigació

- **ODS 3 – Salut i benestar:** Com qualsevol producte d'entreteniment digital, existeix el risc que sessions de joc excessivament llargues fomentin el sedentarisme. Per mitigar aquest impacte, el disseny del joc es basa en "sessions curtes" i minijocs d'alta intensitat, una estructura que convida a fer pauses freqüents i evita el consum compulsiu de temps davant la pantalla.

1.4 Enfocament i metodologia de treball

L'estratègia del projecte es fonamenta en la creació d'un producte original sota un enfocament incremental i modular. La premissa de disseny és construir una arquitectura basada en un nucli (*core*) sòlid que permeti la integració independent de minijocs. Aquesta modularitat és clau per facilitar la iteració, substitució o ampliació de contingut sense comprometre la integritat de la base de codi.

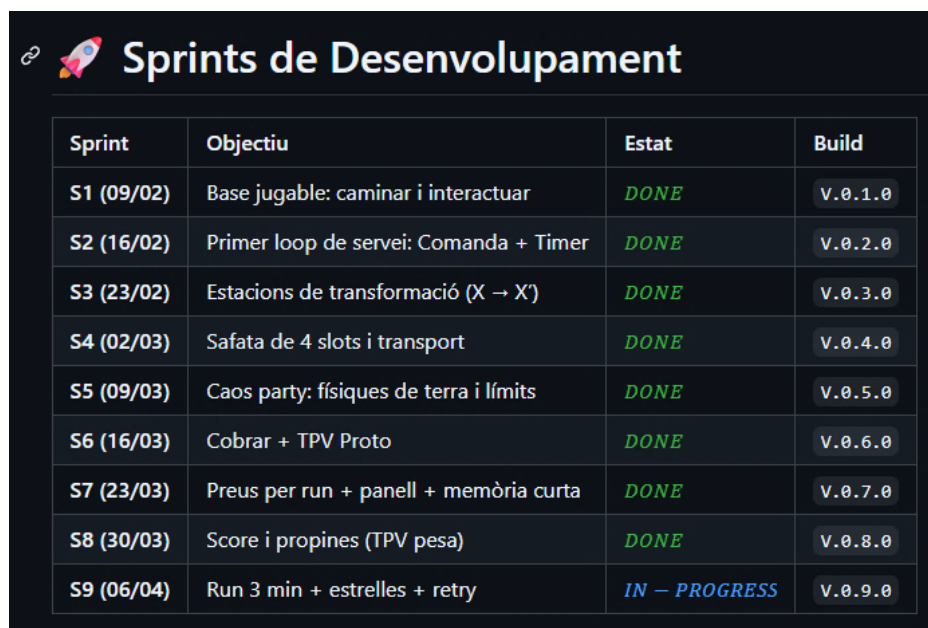
Es prioritzarà l'assoliment d'un MVP (Producte Mínim Viable) per a un sol jugador que sigui complet i estable. Un cop consolidada aquesta fase, s'avaluarà la implementació del mode cooperatiu en línia, evitant que la complexitat del *networking* posi en risc l'entrega de les funcionalitats base. Per a la gestió de tasques, s'utilitzarà un *Backlog* amb la següent jerarquia de prioritats:

Core > Minijocs/Mapes > Poliment (*Polish*) > Mode Online.

1.4.1 Metodologia àgil (Scrum/Kanban)

S'aplicarà una combinació de metodologies àgils per garantir un control estricte de l'abast i el temps:

- **Sprints setmanals:** Permetran realitzar una revisió i testatge continu del producte. Al final de cada cicle es generarà una build jugable per identificar ajustaments necessaris en la dificultat tècnica o l'abast.



Sprint	Objectiu	Estat	Build
S1 (09/02)	Base jugable: caminar i interactuar	DONE	V.0.1.0
S2 (16/02)	Primer loop de servei: Comanda + Timer	DONE	V.0.2.0
S3 (23/02)	Estacions de transformació (X → X')	DONE	V.0.3.0
S4 (02/03)	Safata de 4 slots i transport	DONE	V.0.4.0
S5 (09/03)	Caos party: físiques de terra i límits	DONE	V.0.5.0
S6 (16/03)	Cobrar + TPV Proto	DONE	V.0.6.0
S7 (23/03)	Preus per run + panell + memòria curta	DONE	V.0.7.0
S8 (30/03)	Score i propines (TPV pesa)	DONE	V.0.8.0
S9 (06/04)	Run 3 min + estrelles + retry	IN – PROGRESS	V.0.9.0

Figura 2. Planificació de Sprints de desenvolupament, objectius associats i versions de la build.

Font: Elaboració pròpia a partir del repositori de GitHub.

- **Tauler Kanban:** Es visualitzarà setmanalment el flux de treball (*Backlog / Next / In Progress / Testing / Done*) per identificar colls d'ampolla i bloquejos de forma immediata.



Figura 3. Detall del taulell Kanban analògic utilitzat durant el cicle de desenvolupament.

Font: Elaboració pròpia

1.4.2 Fases del projecte

L'estratègia es divideix en quatre fases: una Preproducció per definir l'arquitectura modular i el GDD; una Producció iterativa mitjançant *Sprints* per implementar el *core loop* i els minijocs; una fase de Validació centrada en el *bugfixing* i el testeig (caixa blanca/negra); i un Tancament per al lliurament de la memòria i la *Build* final.

FASE 1 - Preproducció (Anàlisi, disseny i prototipatge)

L'objectiu és definir el marc conceptual i tècnic per mitigar riscos primerencs.

- **Disseny de joc (GDD):** Definició del concepte, referents i *core loop* del MVP (2 minijocs/mapes).
- **Arquitectura del programari:** Disseny de la interfície comuna per als minijocs i el nucli del sistema.
- **Prototip tècnic (PoC):** Validació de mecàniques base (moviment, càmera, col·lisions) en un entorn de proves (*greyboxing*).
- **Planificació i requeriments del sistema:** Estimació inicial i temporització dels recursos necessaris per al desenvolupament. S'estableix com a plataforma objectiu (*target platform*) l'entorn de PC, definint un conjunt de requeriments i capacitats de maquinari de referència per garantir l'òptim rendiment de l'arquitectura de xarxa i el motor gràfic.
- **Entregables:** Prototip jugable bàsic i documentació d'Anàlisi i Disseny de la primera fita.

FASE 2 - Producció (Desenvolupament iteratiu)

Construcció incremental del videojoc mitjançant cicles de programació i integració.

- **Desenvolupament del Core Loop:** Implementació de menús, gestió d'estats (*State Machine*), sistemes de puntuació, temporitzadors i mapatge de controls (*Bindings*).
- **Framework modular de minijocs:** Creació de plantilles comunes mitjançant *ScriptableObjects* i *Singletons* (GameManager, MusicManager, InputManager) per garantir una instanciació neta en cada canvi d'escena.
- **Creació, cerca i integració d'assets:** Recerca, disseny i desenvolupament dels elements gràfics (UI, art) i sonors (àudio) necessaris, seguit de la seva posterior integració i ajustament del feedback de control (game feel).
- **Control de qualitat:** Al final de cada *Sprint* es realitzarà un *playtest* per actualitzar el *Backlog*.
- **Revisables:** Actualitzacions setmanals al repositori privat de GitHub.

FASE 3 - Validació i estabilització (*Hardening*)

Fase dedicada a la qualitat final i la robustesa del programari.

- **Pla de proves:** Execució de tests de caixa blanca per als mòduls crítics i de caixa negra per a la jugabilitat.
- **Testeig d'usuaris:** Recollida de *feedback* extern per ajustar la corba de dificultat i el rendiment.
- **Bugfixing:** Resolució d'errors i optimització de recursos per a la versió Beta.
- **Entregables:** Documentació de la fita d'implementació i proves + Versió Beta.

FASE 4 - Tancament i lliurament

Preparació del paquet final del TFG.

- **Release candidate (RC):** Versió final 1.0 amb crèdits, llicències i instruccions de configuració.
- **Documentació final:** Elaboració de la memòria, manual d'usuari i vídeo demostratiu de la solució.
- **Entregables:** Memòria final + Build definitiva.

1.5 Planificació

En aquest apartat es detalla l'estructura temporal i organitzativa del projecte. La planificació s'ha dimensionat tenint en compte la càrrega lectiva de 15 crèdits ECTS (equivalents a unes 450 hores de treball segons l'estàndard europeu) i les dates oficials de lliurament del TFG a la Universitat d'Andorra.

S'ha optat per una planificació flexible mitjançant relacions de Gantt de tipus *rubber*, que permeten marges de maniobra davant d'imprevistos tècnics sense comprometre les fites (*milestones*) inamovibles. La fase d'implementació se segueix mitjançant la metodologia àgil per *sprints* setmanals detallada anteriorment.

1.5.1 Taula de tasques i fites

Fase / Sprint	Objectiu / Fita Principal	Període / Data límit
Fase 0: Inicial	Estudi d'eines, anàlisi de l'abast i definició del MVP.	16/02 – 21/02
Fita 1	Lliurament del Pla de Treball	22/02
Fase 1: Disseny	Anàlisi de requisits, definició de l'arquitectura i documentació.	23/02 – 21/03
Fita 2	Lliurament del 1r Seguiment	22/03
Fase 2: Desenvolupament	Implementació del Core loop, minijocs del MVP i mode l'Online.	23/03 – 26/04
Fase 3: Validació	Testeig tècnic, lliurament d'usuaris (<i>playtesting</i>) i poliment (<i>polishing</i>).	27/04 – 13/05
Fita 3	Lliurament del 2n Seguiment	17/05
Fase 4: Tancament	Redacció de la memòria final, conclusions i valoració crítica.	17/05 – 04/06
Fita 4	Prelliurament de la Memòria	31/05
Fita 5	Lliurament Final (Memòria definitiva + Build RC)	09/06

Fase 5: Defensa	Preparació de la presentació, oratòria i pràctica de la demo.	09/06 – 14/06
Fita 6	Lliurament de la Presentació (PPT)	14/06

Nota: La taula de tasques completa es troba l'Annex AT-10: Diagrama de GANTT.

1.6 Productes finals

El resultat del projecte es materialitza en un conjunt d'entregables que abasten des de la conceptualització teòrica fins a la implementació tècnica i la validació. Els productes finals són els següents:

1.6.1 Disseny i arquitectura de la solució

Documentació tècnica que defineix i delimita el sistema sense dependre estrictament de la tecnologia, justificant les decisions estructurals:

- **Game Design Document (GDD):** Definició del concepte, *core loop* i especificacions funcionals.
- **Disseny de mecàniques:** Detall del sistema de puntuació i la mecànica de reciclatge.
- **Arquitectura del programari:** Disseny modular (Core + Interfície comuna) i justificació de patrons (*State, Observer, Factory*).
- **Modelat de fluxos:** Diagrames d'estats de la partida i jerarquia de la interfície d'usuari (UI).

1.6.2 Desenvolupament del programari (Executables de Windows)

L'evolució del codi es presentarà mitjançant diferents versions del producte:

- **Prova de concepte (PoC / Early Alpha):** Prototip inicial en Unity URP per validar la viabilitat tècnica. Inclou control de personatge, càmera, interaccions bàsiques i col·lisions.
- **MVP single player (Alpha):** Versió mínima completa. Inclou el menú principal, el gestor d'estats (*Countdown, Playing, Gameover*), sistemes reutilitzables i els dos primers minijocs integrats.

- **Versió estabilitzada (Beta):** Versió polida que incorpora millores d'UX (tutorials), ajustos de dificultat segons el *feedback* del testeig i optimitzacions de rendiment URP.
- **Release candidate (RC):** Versió final del videojoc (v1.0) preparada per a l'avaluació. Inclou el manual d'usuari, crèdits, el fitxer README i la gestió de llicències d'assets.

1.6.3 Documentació de qualitat i evidències

Productes destinats a la verificació i tancament acadèmic:

- **Paquet de proves:** Documentació del pla de proves (caixa blanca i caixa negra), casos d'ús i resultats de les proves amb usuaris.
- **Memòria final i annexos:** Document complet que recull tot el procés de realització del TFG.
- **Repositori de codi font:** Accés al repositori privat de GitHub amb l'historial de *commits* (seguint la convenció *Conventional Commits*) i les *builds* setmanals.
- **Material multimèdia:** Vídeo demostratiu (*gameplay*) que resumeix les funcionalitats del videojoc.
- **Informe d'autoavaluació:** Valoració crítica sobre l'assoliment dels objectius i el procés d'aprenentatge.

1.7 Recursos

Per a la correcta execució del projecte, s'han identificat i mobilitzat diversos recursos que es classifiquen en materials, programari i immaterials.

1.7.1 Recursos materials (Hardware)

- **Estació de treball:** Un ordinador personal d'alt rendiment amb targeta gràfica dedicada, necessari per a la renderització en temps real mitjançant Unity URP i el processament de *lightmaps*.
- **Perifèrics de control:** Mínim de dos controladors (gamepads) per a les proves de jugabilitat i la validació del mapatge de controls (*input binding*).
- **Infraestructura de xarxa:** Connexió a Internet de banda ampla per a la gestió del repositori, la descàrrega d'assets i les proves de sincronització del mode en línia.

1.7.2 Recursos de programari (Software)

- **Motor de desenvolupament:** Unity (Versió LTS) amb el *pipeline* de renderitzat URP.
- **Entorn de desenvolupament integrat (IDE):** Visual Studio Code per a la programació en C#.
- **Gestió de versions:** GitHub i GitHub Desktop per al control de versions, assegurant la traçabilitat del codi i les còpies de seguretat.
- **Eines de disseny i àudio:** Blender per a la modificació de models 3D i Audacity per a l'edició de clips de so i SFX.
- **Llibreries i assets:** Recursos de la Unity Asset Store sota llicència comercial o lliure (CC0).

1.7.3 Recursos immaterials

- **Capital humà extern:** Temps cedit per tercers per a les sessions de *playtesting* i recollida de *feedback*.
- **Fonts de coneixement:** Documentació oficial de Unity, tutorials tècnics especialitzats i l'ús d'intel·ligència artificial (GitHub Copilot) com a suport en la resolució de problemes algorítmics.

1.7.4 Valoració econòmica

S'estima un pressupost directe d'aproximadament **140 € - 150 €**, sense comptar els costos estructurals (electricitat, lloguer i amortització del maquinari existent) o el cost d'oportunitat:

Concepte	Descripció	Cost estimat
Assets de pagament	Compra de llicències comercials per a models o sistemes específics.	~50 €
Steam Direct Fee	Taxa d'alta de l'aplicació a Steam (reemborsable en superar un llindar de vendes).	100 €
Codi i eines	Ús de programari gratuït i edicions <i>Community/LTS</i> .	0 €
TOTAL		150 €

Nota sobre els serveis de xarxa: El projecte integra la infraestructura de Unity Gaming Services (Relay i Lobby). Aquests serveis operen sota un model de pay-as-you-go (pagament per ús) on els trams inicials orientats al desenvolupament i testeig són completament gratuïts; per exemple, Unity Relay permet fins a 100 usuaris concurrents (CCU) sense cost, mentre que Unity Lobby admet fins a 5.000 jugadors actius mensuals. Atès que el trànsit previst per a la validació del TFG i el llançament del MVP se situa molt per sota d'aquests llindars, el cost operatiu actual s'estima en 0 €.

No obstant això, cal tenir en compte que **a partir d'un cert nivell de volum de negoci i concurrència d'usuaris, Unity aplica tarifes de pagament escalables** segons el consum de dades. Per aquest motiu, i amb l'objectiu de garantir la viabilitat comercial futura, es posarà especial atenció a l'optimització del networking per minimitzar la transferència de paquets per segon. A continuació, es detalla un exemple de les millores aplicables (registrades en l'historial de GitHub) per reduir la càrrega del servidor:

Comparativa de Rendiment:			
Mètrica	Abans (JSON String)	Ara (Struct Compacte)	Millora
Mida al <code>NetworkList</code>	4.096 bytes (Reservats)	10 bytes	-99.7%
Càrrega de CPU	Deserialització constant	Accés directe a bytes	Alta
Escalabilitat (4 jug.)	~16.384 bytes	40 bytes	Dramàtica
Payload Connexió	JSON UTF-8 (~300b)	JSON UTF-8 (~300b)	Estable ¹

¹ El payload de connexió es manté com a JSON per compatibilitat amb el sistema de guardat local, però la gestió en temps de joc és 100% binària.

Figura 4. Comparativa de rendiment en la serialització de dades del `NetworkList`.

Font: Elaboració pròpia a partir de l'historial de branques de GitHub.

1.8 Anàlisi de riscos

En el desenvolupament de programari, i especialment en el sector dels videojocs, la identificació primerenca de riscos és clau per evitar desviacions en el cronograma. A continuació, es presenten els principals riscos identificats, el seu impacte potencial i les estratègies de mitigació (prevenció) i correcció.

Risc	Impacte	Prevenció	Mesura Correctora (Pla B)
Abast excessiu (Mecàniques o poliment)	Alta	Definició estricta del Producte Mínim Viable (MVP) i ús de <i>Backlog</i> prioritzat.	Eliminar funcionalitats secundàries i centrar-se en el <i>core loop</i> perquè el joc sigui jugable.

Complexitat del mode multijugador	Alta	Assignar una prioritat baixa i dissenyar l'arquitectura de forma <i>Multiplayer-First</i> des de l'inici.	En cas de problemes d'estabilitat, lliurar el mode <i>Single Player</i> complet i documentar l'anàlisi tècnica de l'intent d'implementació.
Mecàniques de control deficientes (<i>Game feel</i>)	Alta	Prioritat màxima al prototipatge i realització de <i>playtests</i> setmanals des de les primeres fases.	Reajustar els valors físics, la sensibilitat de la càmera o simplificar el sistema de control mitjançant un "parxe" funcional.
Problemes de rendiment gràfic/tècnic	Mitjana	Realització de <i>Profiling</i> periòdic del programari en diferents configuracions de hardware.	Reduir el postprocessat, optimitzar sistemes de LOD, substituir llums en temps real per <i>Baked Lighting</i> i marcar objectes com a estàtics.
Dependència d'assets de tercers	Mitjana	Ús de tècniques de <i>Greyboxing</i> (formes bàsiques) durant la fase de preproducció per no aturar el codi.	Substituir assets problemàtics per alternatives sota llicència CC0 o recursos propis simplificats.
Bugs crítics de sistema	Mitjana	Ús de patrons de disseny (Observer , Singleton) i una arquitectura modular que aïlli les funcions.	Execució de <i>Hotfixes</i> i priorització de la correcció d'errors sobre la creació de nou contingut.
Incompliment de terminis en la memòria	Mitjana	Redacció concurrent de la documentació tècnica a mesura que s'avança en la implementació.	Aturar el desenvolupament de noves funcionalitats i prioritzar la compleció de la memòria final del TFG.

1.9 Estructura de la memòria

Aquest document s'estructura en els capítols següents, els quals reflecteixen el cicle de vida del desenvolupament del programari:

- **Capítol 2. Anàlisi:** Descripció detallada del context, la situació actual i les problemàtiques detectades en el sistema. Inclou l'estat de l'art amb els referents del gènere Party Game, la definició dels objectius tècnics del Producte Mínim Viable (MVP) i l'especificació formal dels requisits funcionals, no funcionals, operacionals i de qualitat.
- **Capítol 3. Disseny:** Especificació conceptual de l'arquitectura del videojoc, basada en el model unificat **Hybrid-Core**. Es detallen l'estructura i els fluxos de treball dels components modulars de l'aplicació (core loop, controladors de sessió, persistència, lògiques de servei o el sistema de comandes i puntuació), el disseny del sistema de control de jugador basat en interaccions contextuais amb el ratolí, i els mecanismes de robustesa i integritat de dades.
- **Capítol 4. Implementació:** Descripció del procés tècnic de construcció i codificació de la solució sobre el motor Unity i el canal de renderitzat URP (Universal Render Pipeline). Es documenta la integració dels serveis de multijugador (Lobby, Relay i el gestor de partides), el desplegament dels components clau del joc en línia i els entorns, processos de build i distribució necessaris per a l'execució del programari.
- **Capítol 5. Proves:** Recull de la metodologia i els resultats de les fases de validació del sistema. Inclou l'avaluació formal del compliment dels requisits, l'anàlisi de rendiment i regressió, els resultats de les proves específiques de networking i robustesa del mode en línia, i una valoració global de l'estat de qualitat del producte.
- **Capítol 6. Distribució, manteniment i evolució:** Definició dels canals i processos d'empaquetat, instal·lació i actualització del programari en fase beta. Així mateix, s'estableix el model de suport per al manteniment del sistema i es detallen les línies de treball futur juntament amb el calendari de versions previst.
- **Capítol 7. Estratègia de màrqueting i llançament:** Presentació de la planificació comercial i de comunicació del projecte orientada a la seva publicació a la plataforma Steam. S'hi detallen el cronograma de visibilitat i l'anàlisi dels canals de difusió seleccionats.
- **Capítol 8. Conclusions:** Reflexió objectiva sobre el grau d'assoliment dels objectius plantejats i valoració crítica del procés de desenvolupament del projecte en relació amb les competències del títol de grau.
- **Capítols 9 al 12. Glossari, bibliografia, llicències i annexos:** Seccions de tancament del document destinades a la definició de termes tècnics, el recull de les fonts bibliogràfiques consultades i l'especificació de les llicències del projecte. Finalment, s'incorporen els annexos amb el detall de l'arquitectura tècnica, diagrames de GANTT, anàlisi de patrons implementats i les evidències del progrés de desenvolupament.

Capítol 2: Anàlisi

2.1 Context i situació actual

En aquest capítol es detallen els fonaments teòrics i tècnics que sustenten el desenvolupament del projecte. L'anàlisi s'estructura partint de la situació actual del mercat dels videojocs i la definició de l'arquitectura usada, passant per una revisió exhaustiva de l'estat de l'art i la tecnologia utilitzada. Finalment, s'estableixen els objectius i els requisits tècnics concrets que han de guiar la implementació del sistema per garantir una solució escalable, robusta i alineada amb les necessitats de l'usuari final.

2.1.1 Descripció del projecte i arquitectura tècnica

El projecte s'integra en el gènere dels "Party Games" 3D per a PC, un nínxol amb gran potencial a Steam. La proposta se centra en una experiència de gestió frenètica on el jugador adopta el rol d'un cambrer. El nucli del joc (core loop) consisteix a preparar comandes, servir-les i gestionar els cobraments mitjançant un sistema de TPV d'increment, decrement i preus, tot condensat en sessions intenses i curtes que desafien l'agilitat de l'usuari.

Tot i que el Producte Mínim Viable (MVP) està dissenyat per a ser gaudit en solitari (*single-player*), el desenvolupament s'ha executat sota una filosofia d'arquitectura "Multiplayer-First". Això significa que el joc no es construeix com un títol local tradicional, sinó que utilitza Unity Netcode for GameObjects (NGO) des de la primera línia de codi. Aquesta decisió estratègica permet que la base del projecte estigui totalment preparada per a una futura expansió a mode cooperatiu en línia sense necessitat de refactoritzacions massives.

Tècnicament, el sistema opera mitjançant un model d'autoritat del servidor (Host-Authoritative). En aquesta configuració, el client del jugador únic (Singleplayer) actua simultàniament com a servidor, garantint que tota la lògica de joc, les col·lisions i els estats de les comandes estiguin centralitzats. Aquesta infraestructura assegura una simulació robusta i coherent, facilitant una escalabilitat immediata cap a l'entorn multijugador en futures iteracions del projecte. - *El qual ja s'està completant actualment en un cicle de sprints setmanals.*

El joc es dissenyarà per suportar tres modes principals de funcionament. En primer lloc, una campanya singleplayer amb progressió local, nivells desbloquejables i sistema de guardat. En segon lloc, un mode cooperatiu online que permet jugar amb un amic mitjançant un sistema de lobby privat i codi d'unió. Finalment, es contempla un mode multiplayer SoloQ, on els jugadors entren automàticament en sessions generades pel sistema que carrega un mapa aleatori d'un conjunt predefinit.

Campanya Singleplayer (MVP) — sense Wi Fi / internet amb progressió:

- Nivells bloquejats/desbloquejats.
- Guardat de progrés local.
- Menú de selecció de nivell.

Cooperatiu online (No MVP):

- A partir d'un nivell de campanya, poder activar “Jugar amb amic”.
- El host crea lobby privat i fa “Relay allocation” per obtenir un join code.
- L'amic introdueix el join code i entra com a client.
- El nivell es carrega de forma sincronitzada.

Multiplayer “SoloQ” (No MVP):

- Botó “SoloQ” que crea sessions automàticament: Host + Clients.
- Host crea lobby privat i Relay allocation.
- Clients entren automàticament amb join code (sense input manual).
- La partida carrega un mapa aleatori d'un pool de mapes predeterminats.

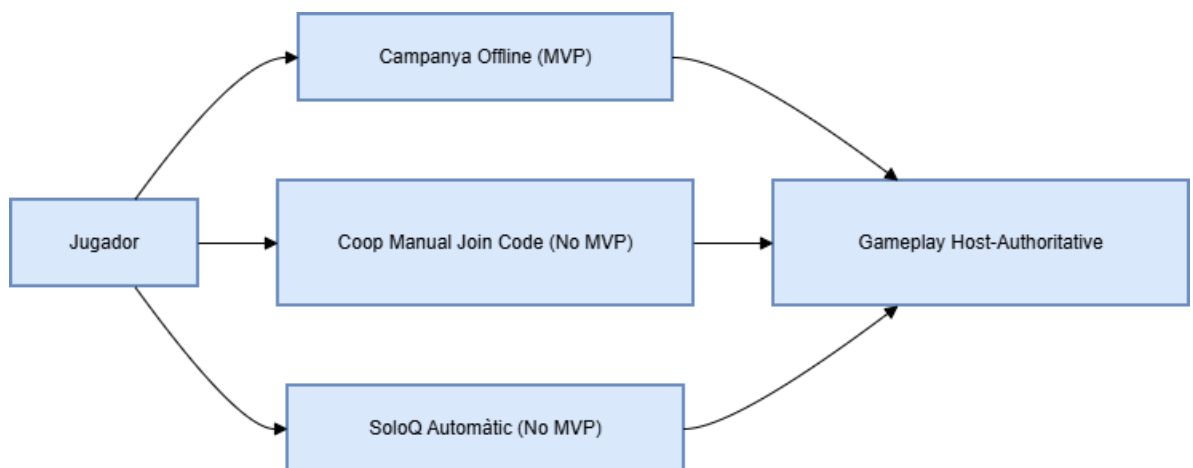


Figura 5. Diagrama de flux de l'accés als modes de joc i arquitectura de xarxa.

Font: Elaboració pròpia mitjançant l'eina Draw.io.

2.1.2 Problemàtiques detectades, conseqüències i solucions.

P1 — Convivència de modes offline i online sobre el mateix codi

- **Descripció:** La base de gameplay està pensada per funcionar sota autoritat del host. Si el singleplayer no s'executa com a "host local", apareixen divergències (lògica server-only no s'executa).
- **Solució:** S'implementarà un coordinador de sessió que gestioni aquesta capa. Aquest coordinador serà l'encarregat de gestionar el connector de network i canviar el seu estat entre Offline i Online. La base del joc s'implenetrà directament amb una estructura de xarxa. [1]

P2 — Requisit de "offline real" (sense internet)

- **Descripció:** La campanya ha de funcionar sense serveis online. Qualsevol inicialització obligatòria de Lobby/Relay/Auth pot impedir jugar sense connexió.
- **Solució:** S'implementarà d'un coordinador de sessió que gestioni aquesta capa. Aquest coordinador serà l'encarregat de gestionar el connector de network i canviar el seu estat entre Offline i Online. La base del joc s'implenetrà directament amb una estructura de xarxa. El mode Offline no activarà les capes de Lobby i Relay. El NetworkManager s'activarà en mode local al 127. [1]

P3 — Gestió de sessions i teardown

- **Descripció:** Els canvis d'escena i el retorn al menú han de tancar correctament l'estat de sessió (xarxa, lobby, objectes persistents).
- **Solució:** Al coordinador de sessió crearem els helper de tancament de sessió unificat per evitar divergències. [1]

P4 — Complexitat tècnica 3D en solitari:

- **Descripció:** Control de personatge, físiques, sincronització d'animacions, càmera semi-fixa, URP i shaders al mateix temps
- **Conseqüències:** Risc d'un producte inacabat o poc polit; bugs de físiques difícils de depurar
- **Solució:** Prototipat ràpid amb formes bàsiques (cubs), sprints setmanals amb build jugable i playtests.

P5 — Producció d'assets 3D:

- **Descripció:** Creació manual de models, UI, música i postprocessat
- **Conseqüències:** Temps de programació reduït; dependència d'assets de tercers
- **Solució:** Ús d'assets CC0 / Asset Store amb llicència comercial + prioritzar placeholders en fase de desenvolupament. [2]

P6 — Mode online (futur):

- **Descripció:** Sincronització, latència, desconnexions, seguretat amb NGO
- **Conseqüències:** Si s'implementa prematurament, compromet l'entrega del producte base
- **Solució:** Baixa prioritat al backlog; l'arquitectura Hybrid-Core permet afegir-ho sense reescriure el core. Implementar si ja anem adelantat als Sprints.

[1] **Nota sobre el SessionCoordinator:** Aquesta solució ja s'ha provat i materialitzat en un entorn de proves amb el component SessionCoordinator.

2.2 Estat de l'art

En aquest apartat es realitza una revisió del gènere dels *Party Games* 3D i les tendències actuals de gestió frenètica en la plataforma Steam, analitzant com el model de negoci s'orienta cap a experiències altament rejugables. Així mateix, s'explora la problemàtica tècnica de l'escalabilitat multijugador, justificant l'adopció d'una arquitectura "Multiplayer-First" mitjançant Unity Netcode for GameObjects (NGO). Aquesta estratègia permet que el projecte funcioni com un MVP en solitari mentre manté una base tecnològica robusta i preparada per a una futura expansió cooperativa en línia.

2.2.1 Referents del gènere

- **Overcooked! 2 (Ghost Town Games, 2018):** És el referent principal. Defineix el gènere del joc cooperatiu de cuina amb loop "preparar → servir → cobrar". Els seus punts forts són el caos controlat, els nivells amb obstacles dinàmics i la progressió de dificultat. La seva limitació és que no té mecàniques econòmiques ni sistemes de pagament. [1]



Figura 6. Interfície de joc i dinàmica de cuina cooperativa a *Overcooked! 2*.

Font: Ghost Town Games (2018).

- **PlateUp! (Yogscast Games, 2022):** Evoluciona el gènere afegint construcció de restaurants i automatització. Demostra que el gènere admet profunditat econòmica.



Figura 7. Fase de planificació, automatització i gestió del restaurant a PlateUp!.

Font: Yogscast Games (2022).

- **Moving Out (SMG Studio, 2020):** És un referent de caos físic party game, amb col·lisions i física ragdoll com a nucli del gameplay. Confirma que les caigudes d'objectes per col·lisió són divertides si estan ben ajustades. [3]

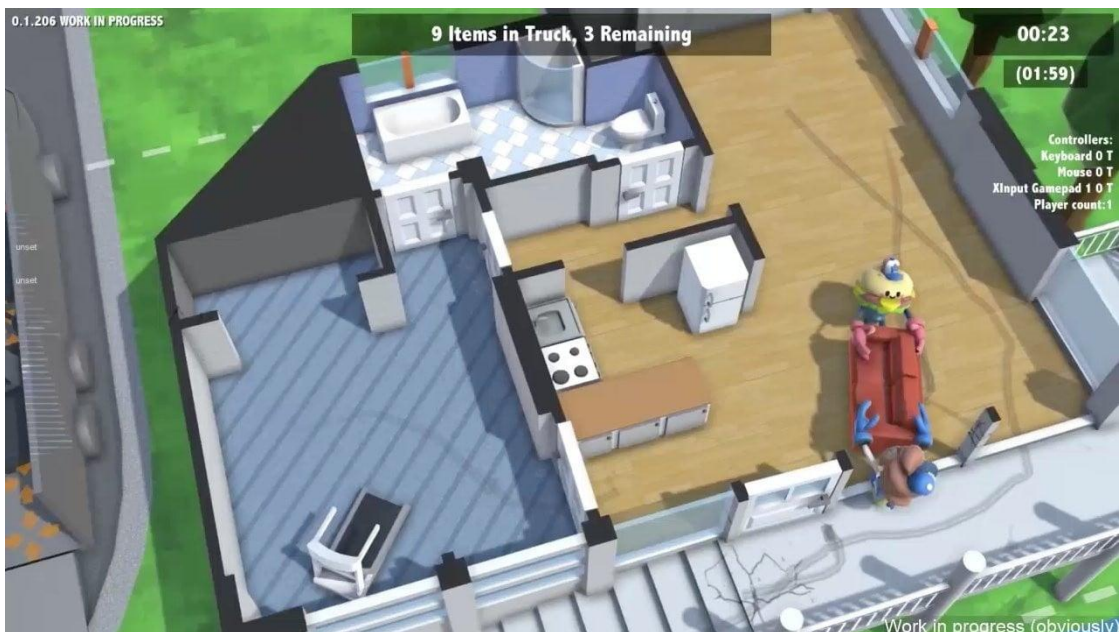


Figura 8 Mecànica cooperativa basada en el desplaçament d'objectes i físiques d'impacte a Moving Out.

Font: SMG Studio (2020).

2.2.2 Estat de la tecnologia

- **Unity URP (Universal Render Pipeline)** és el pipeline de renderitzat actual per a jocs mid-range. Ofereix il·luminació PBR, post-processat i optimitzacions de rendiment per a PC i plataformes. Substitueix el Built-in RP per a nous projectes. [4]
- **Unity Netcode for GameObjects (NGO)** és la solució oficial de Unity per a multijugador. Utilitza una topologia Host-Client Authoritative que és adequada per a jocs de fins a 4-8 jugadors. La versió 1.x+ ha millorat la seva estabilitat significativament. [5]
- **Patró State Machine per a sistemes de joc:** La literatura de patrons de videojocs (Nystrom, 2014) proposa màquines d'estat finit (FSM) per gestionar comportaments complexos com els estats d'una comanda. [9]
- **Patró Observer per a comunicació desacoblada:** El patró observer (GoF, 1994) és àmpliament adoptat a Unity per separar la lògica de joc de la UI. La variant amb ScriptableObject Event Channels (Unite 2017) permet visibilitat a l'inspector. [10][11]
- **Patró MVP per a UI de videojocs:** El Model-View-Presenter adaptat a Unity permet tenir Presenters que centralitzen les subscripcions a esdeveniments, fent el cablejat observable i depurable sense usar frameworks DI. [9]
- **Samples i patrons de flux:** Les demos i samples oficials mostren fluxos típics: lobby/ready room, sincronització d'escenes i entrada a gameplay. Aquestes referències serveixen com a “estat de l'art” per patrons de sessió i scene management. [8]
- **Lobby + Relay + join code (invitar amics):** Un enfocament molt estès per a sessions P2P assistides és utilitzar Relay i join codes compartibles per canals externs. Això facilita coop manual sense requerir IP pública ni configuracions NAT complexes. [6][7]

2.2.3 Tendències futures

- **Multijugador online amb Steamworks:** Integració directa amb Steam per a lobbies, invite friends i estadístiques. Steamworks.NET és la biblioteca C# recomanada.
- **Procedural level design:** Generació procedural de mapes per augmentar la rejugabilitat sense cost de producció manual.
- **Accessibilitat:** Opcions de daltonisme, remapatge de controls i subtítols.

2.3 Objectius tècnics

Per tal d'assegurar la viabilitat del projecte dins dels terminis acadèmics i garantir la robustesa de l'arquitectura, s'ha definit una línia divisòria estricta entre el producte final compromès per a la defensa d'aquest Treball Final de Grau (MVP Final) i les extensions previstes per al seu llançament comercial (Post-MVP):

A continuació, es detalla la llegenda de sigles utilitzada al llarg d'aquest document per a la classificació i traçabilitat dels diferents requeriments i fites del projecte:

- **RF:** Requisit Funcional
- **RNF:** Requisit No Funcional
- **ROp:** Requisit Operacional
- **RQ:** Requisit de Qualitat
- **OT:** Objectiu Tècnic

1. MVP Final (Abast tancat i compromès per al TFG)

El nucli central del TFG es concreta en un prototip funcional i altament polit en mode individual (Singleplayer). Aquest entorn s'executa de forma offline real (sense dependències de servidors externs) però s'ha programat de forma nativa sota una arquitectura *multiplayer-ready* (Host-Authoritative) utilitzant el *framework* Netcode for GameObjects (NGO).

- **Mode de Joc:** Campanya per a un sol jugador (*offline*) on l'amfitrió (*host*) local actua com a font d'autoritat lògica.
- **Mecàniques completes:** Bucle de joc core (*Core Loop*) basat en la seqüència "preparar, servir i cobrar" (comandes, estacions de transformació, inventari de safata de 3 slots i sistema de pagament TPV).
- **Contingut:** Menú principal, sistema de puntuació/combos i un pool de 2 mapes/nivells diferenciats amb configuració de dificultat progressiva.
- **Interacció:** Sistema de control basat íntegrament en la navegació amb el ratolí i WASD + E.

2. Post-MVP (Línies d'escalabilitat i llançament comercial)

S'exclouen de la versió de lliurament obligatori del TFG, quedant definides com a evolucions tecnològiques viables gràcies a l'arquitectura modular (*Hybrid-Core*) dissenyada:

- **Connexió multijugador cooperatiu en línia:** Creació i unió de sessions remotes de forma privada mitjançant codis d'unió (*Join Codes*) gestionats amb *Unity Relay* o *Steamworks*.
- **Sistema de matchmaking (SoloQ):** Assignació automatitzada de jugadors a sessions actives en línia amb selecció determinista de mapa administrada pel servidor.
- **Persistència local avançada:** Sistema de guardat local en disc per registrar l'històric del jugador, puntuacions màximes permanents i el desbloqueig progressiu de la llista de nivells.
- **Ampliació de contingut comercial:** Integració de nous mapes, esdeveniments aleatoris en partida, disseny d'una botiga cosmètica (*Skins*) amb moneda pròpia (*Diamant*) i monetització de cara al llançament final previst a la plataforma Steam.

2.3.1 Objectius del Producte Mínim Viable (MVP)

- **Sistemes de joc i mecàniques:**
 - **OT-01:** Desenvolupar un sistema de control de personatge en tercera persona amb interaccions contextuais (Input System).
 - **OT-02:** Implementar el bucle de joc principal (*core loop*) que integri les fases de preparació, servei i cobrament en sessions de 3 a 5 minuts.
 - **OT-03:** Dissenyar un sistema de TPV funcional amb lògica d'ajust d'imports i validació d'estats de pagament.
 - **OT-04:** Programar un gestor de comandes basat en màquines d'estats per controlar el cicle de vida de cada producte.
 - **OT-05:** Desenvolupar un sistema d'inventari físic per al jugador (safata de 3 *slots*) amb lògica de recollida, llançament i pèrdua per impacte (La pèrdua per impacte s'està avaluant la seva implementació per la seva complexitat al mode online futur.).

- **Arquitectura i infraestructura:**

- **OT-06:** Implementar una arquitectura modular mitjançant patrons de disseny (Observer, State, MVP i Adapter) per garantir el desacoblament del codi.
- **OT-07:** Executar el desenvolupament sota un model de xarxa Host-Authoritative, assegurant que la lògica *single-player* sigui compatible amb una futura expansió multijugador via NGO.
- **OT-08:** Garantir el funcionament de la campanya en mode *offline* real, sense dependències de servidors externs.

- **Contingut i progressió:**

- **OT-09:** Dissenyar i integrar dos mapes diferenciats amb configuracions de *spawn* i dificultat progressiva.

2.3.2 Objectius Post-MVP (Escalabilitat)

- **OT-10:** Desenvolupar un sistema de persistència de dades local per gestionar el progrés de l'usuari i el desbloqueig de nivells.
- **OT-11:** Estendre l'arquitectura per permetre la creació de sessions cooperatives mitjançant codis d'unió (*Join Codes*) amb Unity Relay.
- **OT-12:** Desenvolupar un sistema de *matchmaking* simplificat (SoloQ) per a la creació i unió automàtica a partides existents.
- **OT-13:** Implementar un selector de mapes aleatori amb autoritat de servidor per garantir la consistència en entorns multijugador.

2.3.3 Objectius transversals

- **OT-14:** Optimitzar el cicle de vida de les escenes i la càrrega de recursos per minimitzar els errors de memòria (*memory leaks*) i les transicions duplicades.
- **OT-15:** Assegurar un feedback visual i sonor coherent en totes les interaccions del joc (estacions de transformació, TPV i sistema de puntuació).

2.4 Requisits tècnics

Aquest apartat detalla les especificacions i capacitats que el sistema ha de complir per garantir el funcionament correcte del videojoc. Els requisits s'han categoritzat segons la seva funcionalitat, comportament operatiu i estàndards de qualitat.

2.4.1 Requisits funcionals (RF)

Els requisits funcionals defineixen les accions i comportaments del sistema que l'usuari final experimentarà directament.

Mecàniques del jugador:

RF-01: Moviment del personatge en espai 3D mitjançant tecles WASD amb detecció de col·lisions via *CapsuleCast*.

RF-02: Interacció amb l'entorn (agafar/deixar ítems i activar estacions) mitjançant les tecles E i Q.

RF-03: Sistema de safata amb capacitat limitada a 3 ítems i pèrdua d'objectes per col·lisions físiques. *(La pèrdua per col·lisió és un requisit identificat pendent de validació de disseny)*

Bucle de Joc (Core loop):

RF-04: Gestió del cicle de vida de les comandes a les taules: des de la petició fins al consum i l'espera de pagament.

RF-05: Transformació d'ítems en estacions de treball mitjançant interaccions mantingudes.

RF-06: Sistema de pagament TPV amb ajust d'import i validació amb penalitzacions per errors de cobrament.

RF-07: Gestió de la partida amb sessions de 3 minuts, càlcul de puntuació (propinas i bonus) i pantalla de resultats.

Gestió i persistència:

RF-08: Menú principal amb accés a la campanya, selecció de nivells i ajustos de configuració.

RF-09: Sistema de persistència de dades per al guardat local del progrés i nivells desbloquejats.

Capacitats multijugador (Preparació/No MVP):

RF-10: Arquitectura preparada per a la creació de sessions cooperatives amb codi d'unió (*Join Code*).

RF-11: Sincronització autoritària del *host* en la selecció de mapes i càrrega determinista de nivells.

2.4.2 Requisites no funcionals (RNF)

Defineixen les propietats del sistema, com el rendiment, l'arquitectura i les restriccions tecnològiques.

RNF-01: Arquitectura modular basada en principis SOLID per permetre l'extensió de contingut (mapes/modes) sense modificar el nucli.

RNF-02: Disseny "Multiplayer-First" mitjançant Unity Netcode for GameObjects (NGO) per a una escalabilitat nativa al mode cooperatiu.

RNF-03: Rendiment optimitzat per mantenir 60 FPS estables en equips de gamma mitjana usant l'Universal Render Pipeline (URP).

RNF-04: Gestió del codi font mitjançant control de versions seguint l'estàndard de Conventional Commits.

2.4.3 Requisites operacionals (ROp)

Detallen les condicions sota les quals el programari ha d'executar-se i ser utilitzat.

ROp-01: Compatibilitat exclusiva amb sistemes Windows PC (arquitectura x64).

ROp-02: Suport per a entrada de dades mitjançant teclat i ratolí utilitzant el *Unity New Input System*.

ROp-03: Execució en mode *offline* real per a la campanya en solitari, sense dependència de serveis de xarxa.

ROp-04: Perspectiva de càmera en tercera persona semi-fixa amb sistemes de seguretat contra oclusions.

2.4.4 Requisites de qualitat (RQ)

Asseguren la robustesa, mantenibilitat i una experiència d'usuari sense errors crítics.

RQ-01: Prevenció de *softlocks* mitjançant validació d'estats de joc i gestió de sortides d'emergència al menú.

RQ-02: Integritat de dades d'objectes: un ítem només pot existir en un estat atòmic (mà, safata, terra o taula) per evitar duplicacions.

RQ-03: Gestió eficient de la memòria mitjançant la subscripció i desubscripció estricta d'esdeveniments en els cicles de vida d'Unity (*OnEnable/OnDisable*).

RQ-04: Robustesa davant casos límit com desconexions o càrregues interrompudes, garantint un buidatge de recursos net (*teardown*).

Capítol 3: Disseny

3.1 Arquitectura

3.1.1 Visió general de la solució: Hybrid-Core

La solució es dissenya amb una arquitectura **Hybrid-Core** orientada a la reutilització d'un únic nucli de gameplay per a tres modes d'execució:

- A. Campanya singleplayer (MVP):** execució local amb autoritat del host local (offline real). RF-01...RF-09, ROp-03, RQ-01, RQ-03.
- B. Cooperatiu per codi (Post-MVP):** sessió privada amb lobby + codi d'unió. RF-10, RNF-02, RQ-04.
- C. SoloQ (Post-MVP):** assignació automàtica de sessió i selecció autoritària de mapa aleatori. RF-11, RNF-02.

Aquesta decisió redueix risc de refactorització, simplifica manteniment (RNF-01) i preserva escalabilitat multijugador (RNF-02), mantenint compatibilitat amb el requisit d'execució offline real (ROp-03).

Diagrama de context del sistema (Model C4)

Un diagrama C4 (o Diagrama de Context de Nivell 1) és la representació de més alt nivell d'abstracció dins del model C4 creat per Simon Brown. Aquí podem veure com com encaixa el nostre sistema en el món real,

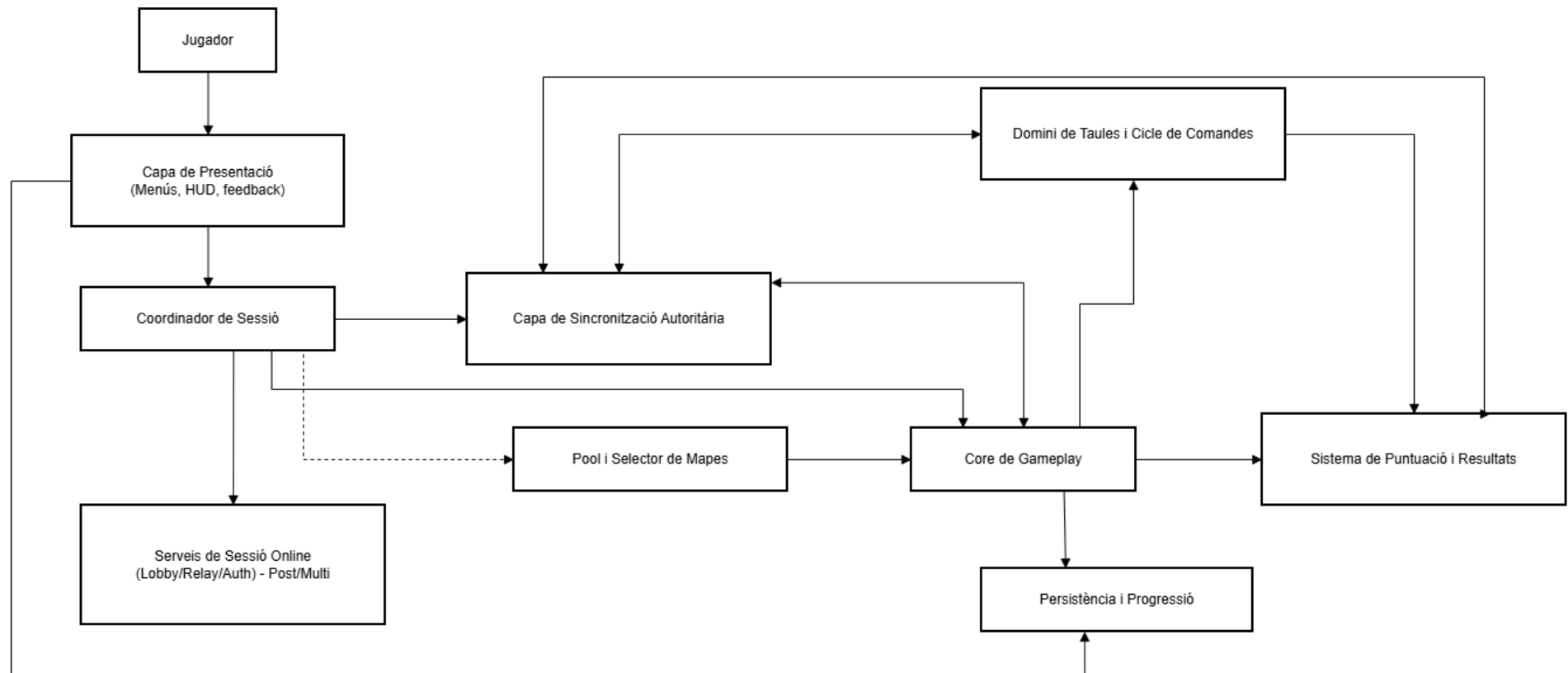


Figura 9. Diagrama de tipus C4-like Context

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.1.2 Funcionament general: Integració i fluxos

El funcionament global es basa en tres fluxos principals:

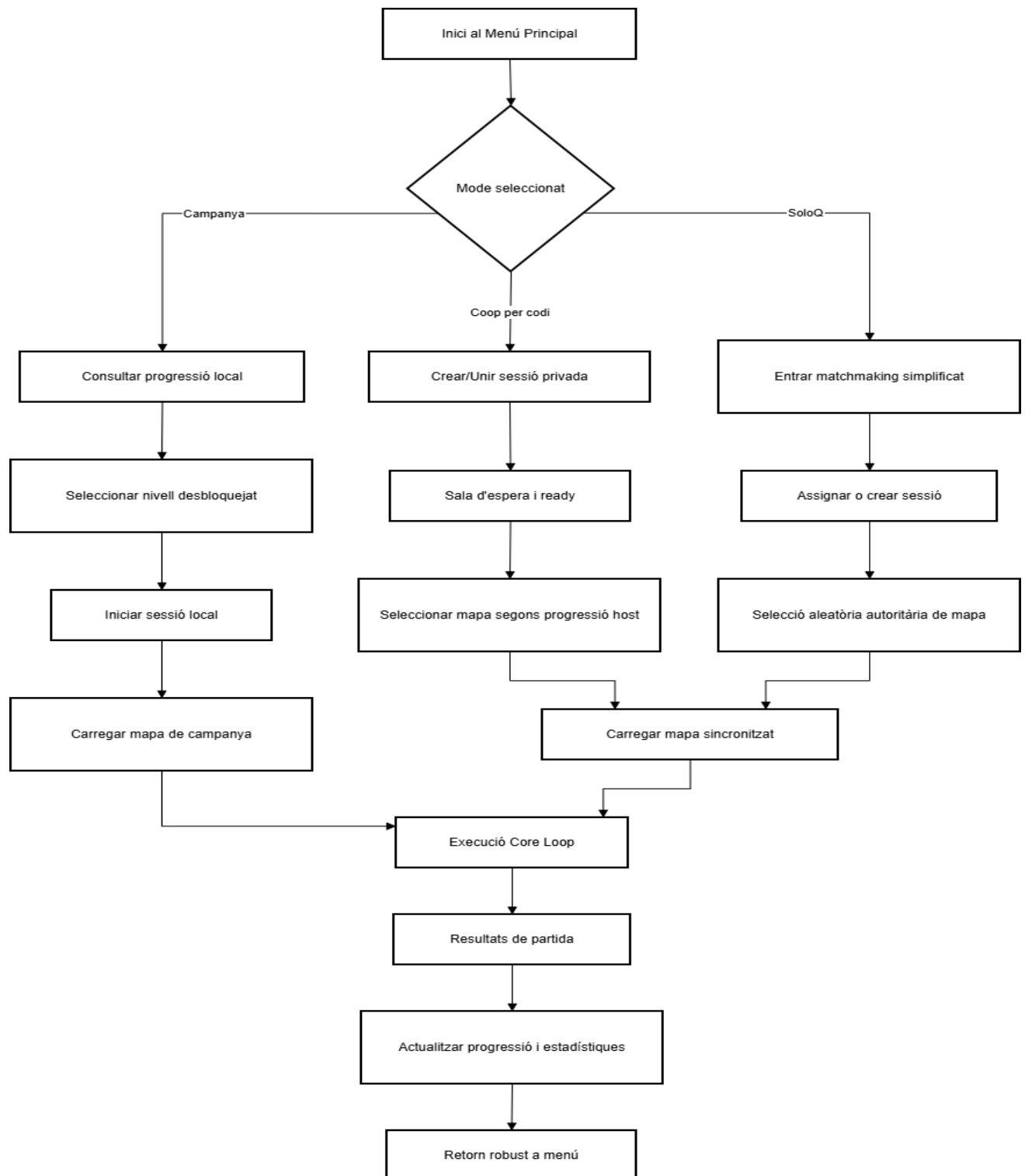


Figura 10. Activity Diagram (Flux global de modes)

Font: Elaboració pròpia mitjançant l'eina Draw.io

A. Flux singleplayer (MVP)

Selecció de nivell desbloquejat, inici de sessió local, execució del core loop, resultats i actualització de progrés. RF-01...RF-09, ROp-03, RQ-01, RQ-03.

B. Flux cooperatiu per codi (Post-MVP)

Creació de sessió privada, entrada de participants, sala d'espera (ready), càrrega sincronitzada i partida. RF-10, RNF-02, RQ-04.

C. Flux SoloQ (Post-MVP)

Entrada automàtica a sessió, formació de partida, selecció autoritària de mapa del pool i inici sincronitzat. RF-11, RNF-02.

3.1.3 Components principals i relació amb requisits

Capa de presentació (UI / Menús / HUD)

Mostra i gestiona l'entrada d'usuari: selecció de mode, selecció de nivell, pantalla de lobby/ready, HUD de partida, pantalla de resultats i opcions.

Alineació: RF-07, RF-08, ROp-02, RQ-03.

Persistència i progressió (guardat local)

Gestiona nivells desbloquejats, continuïtat de campanya i guardat offline real.

Alineació: RF-09, ROp-03, RNF-01.

Coordinador de sessió (Session Coordinator)

Orquestra l'entrada i sortida d'una sessió, assegura teardown net i centralitza fluxos per evitar divergències entre offline/online.

Alineació: RQ-01, RQ-04, RNF-02, P1-P3.

Core de gameplay (Core Loop)

Implementa mecàniques de moviment, interacció, transformació, servei i cobrament, sessions curtes i ritme de joc.

Alineació: RF-01...RF-07, ROp-04.

Domini de taules i NPCs ambientals

Modela el cicle de servei de cada taula mitjançant FSM i gestiona la fase ambiental prèvia (seating d'NPCs), amb timeouts i recuperació segura.

Alineació: RF-04, RF-07, RQ-01, RQ-04, OT-15.

Sistema de comandes

Crea, temporitza i tanca comandes; distingeix comandes assignades i no assignades; sincronitza temporitzadors amb fases de taula.

Alineació: RF-04, RF-07.

TPV (pagament)

Separació entre UI local (entrada d'import) i validació autoritària, amb definició de pagament correcte/perfecte i impacte a puntuació/compos.

Alineació: RF-06, RF-07, RQ-01, RNF-02.

Sistema de puntuació i resultats

Centralitza score, combo, mètriques de qualitat i penalitzacions; garanteix consistència en online.

Alineació: RF-07, RNF-02, RQ-04.

Capa d'autoritat i sincronització

Canalitza interaccions client→servidor i propagació d'estat servidor→clients per a coherència en sessions online.

Alineació: RNF-02, RF-10, RF-11, RQ-04.

Pool / Selector de mapes

Gestiona selecció manual en campanya i selecció autoritària (inclosa aleatòria) en modes online.

Alineació: OT-09, OT-13, RF-11.

3.1.4 Models i patrons de disseny utilitzats

- **Coordinator / Facade:** Concentrar control de sessió i teardown (reducció de dispersió).. RQ-01, RQ-04
- **State Pattern** (màquina d'estats de les taules): Cicle de taules i fases de servei. Ens permet encapsular comportaments per fase, reduint condicional complexa i millorant mantenibilitat. RF-04, RQ-01
- **Observer / Event-driven:** Desacoblament entre domini i UI. Notificació de canvis de progrés, requeriments i puntuació cap a UI. (Veure Annex: Proposta d'Arquitectura Basada en Esdeveniments) RQ-03, RNF-01
- **Adapter:** desacoblament entre contractes de comanda i implementació concreta del gestor de receptes. RNF-01
- **Server-authoritative model:** Consistència de regles crítiques en online. En general totes les decisions crítiques es resolen al servidor, minimitzant desincronitzacions i trampes. Aquí he de parlar que les decisions no crítiques com sincronització d'UI potser les fiquem client authoritative per aminorar la càrrega de networking. Com ja vam veure a l'anterior capítol aquest tema d'optimització és important per aminorar els costos operatius de l'aplicació final i maximitzar beneficis. RNF-02, RF-11
- **Fail-closed + guard clauses:** recuperació segura davant condicions límit (Race conditions). En errors de spawn/seating, el sistema torna a Idle de forma segura. RQ-01, RQ-04
- **Scheduler:** Activació temporal de càrrega de taules i ritme de gameplay. Activa de forma periòdica i controlada de noves taules, evitant saturació.

3.2 Components

En aquest apartat es detalla l'arquitectura modular del sistema, desglossant les unitats funcionals que permeten complir amb els requeriments del projecte. Cada component es descriu des d'una perspectiva tècnica i funcional, justificant-ne la seva elecció i integració

A continuació es presenta la traçabilitat entre els components definits al Capítol 3 (Disseny) i els requisits identificats al Capítol 2 (Anàlisi). L'objectiu és demostrar que el disseny cobreix tant requisits funcionals (RF) com no funcionals (RNF), operacionals (ROp), de qualitat (RQ) i objectius tècnics (OT), i que l'arquitectura manté coherència amb l'enfocament Hybrid-Core i la filosofia Multiplayer-First.

Component	Requisits coberts	Justificació (resum)
Capa de presentació (UI / Menús / HUD) (3.2.1)	RF-07, RF-08, ROp-02, RQ-03	Proporciona navegació entre modes, selecció de nivell, HUD i resultats; desacoblada del domini mitjançant esdeveniments/subscripcions controlades.
Persistència i progressió (campanya offline) (3.2.2)	RF-09, ROp-03, RNF-01, OT-10	Manté guardat local i desbloqueig de nivells sense dependències online; separació dades/servei per facilitar extensió i manteniment.
Coordinador de sessió (Session Coordinator) (3.2.3)	RNF-02, ROp-03, RQ-01, RQ-04, OT-07	Centralitza inici/fi de sessió, evita divergències entre offline/online i aplica teardown unificat per prevenir softlocks i fuites de recursos.
Pool / Selector de mapes (3.2.4)	OT-09, OT-13, RF-11	Unifica la política de selecció de mapa: manual (campanya), coherent amb host (coop) i aleatòria autoritària (SoloQ), garantint determinisme en online.
Core de gameplay (Core Loop) (3.2.5)	RF-01...RF-07, OT-01, OT-02, OT-06	Implementa moviment/interacció i el loop preparar→servir→cobrar, mantenint modularitat i extensibilitat per a l'evolució del projecte.

Component	Requisits coberts	Justificació (resum)
Domini de taules (FSM de servei) (3.2.6)	RF-04, RF-06, RF-07, OT-04, RQ-01, RQ-04	Modela el cicle de comanda per fases amb timeouts i “fail-closed”, evitant bloquejos i assegurant transicions deterministes.
Seating NPC (ambient pre-activació) (3.2.7)	OT-15, RNF-02, RQ-04	Afegeix ambientació funcional abans del servei; spawn/cleanup robust server-authoritative per evitar inconsistències i errors de lifecycle.
Navegació i animació NPC (3.2.7)	OT-15, RQ-04 (i RNF-03 indirecte)	Garanteix coherència visual (seure/aixecar-se/sortir) i prevenció d'errors (dobles callbacks, neteja segura).
Sistema de comandes (3.2.8)	RF-04, RF-07, OT-02, RNF-01	Gestiona generació, límits i finalització d'ordres; separa el domini de comandes de la representació i facilita escalabilitat.
Planificador de taules (Scheduler) (3.2.12)	RF-04, RNF-01, RQ-01	Activa taules elegibles de forma periòdica per regular càrrega i ritme, evitant saturació i situacions incoherents.
TPV (pagament) (3.2.9)	RF-06, RF-07, OT-03, RQ-01, RNF-02	UI local + validació autoritària: només es pot pagar en fase correcta i amb condicions; suporta pagament perfecte i impacte en scoring.
Sistema de puntuació i combo (3.2.10)	RF-07, RNF-02, RQ-04	Score i combo coherents per a tots els participants; penalitza fallides/timeouts i registra mètriques per resultats.
Capa d'autoritat i sincronització (3.2.11)	RNF-02, RF-10, RF-11, RQ-02, RQ-04	Canalitza interaccions i propagació d'estat, reforça coherència en online i integritat de dades davant desconnexions i casos límit.

Component	Requisits coberts	Justificació (resum)
Integritat d'ítems (invariant atòmic) (3.3.2)	RQ-02, RF-03, RF-02	Assegura que un ítem només existeix en un contenidor lògic alhora (món/jugador/taula/estació), evitant duplicacions i inconsistències.
Teardown unificat i prevenció de softlocks (3.3.1)	RQ-01, RQ-04, ROp-03	Defineix sortida segura des de qualsevol estat, neteja de recursos i retorn robust a menú, tant en offline com en online.
Estratègia MVP vs Post-MVP (control de risc) (3.3.3)	OT-01...OT-09 (MVP), OT-10...OT-13 (Post), RNF-01, RNF-02	Manté el core estable (MVP) mentre habilita evolució controlada cap a coop i SoloQ, limitant abast i risc tècnic.
Rendiment i restriccions operacionals (context de disseny) (3.3.3 i transversals)	RNF-03, ROp-01, ROp-02, ROp-04	Condiciona el disseny cap a modularitat i fluxos simples per facilitar optimització i estabilitat a PC Windows x64 amb 60 FPS.

Resum de cobertura per grup de requisits

RF-01 / RF-02 (moviment + interacció): Core de Gameplay, Integritat d'ítems, Capa d'autoritat i sincronització.

RF-03 (safata 3 ítems + pèrdua per col·lisions): Core de Gameplay + Integritat d'ítems (i, si s'implementa completament la pèrdua per col·lisió en online, reforç amb validacions d'autoritat).

RF-04 / RF-05 (lifecycle de comandes i estacions): Domini de Taules (FSM) + Sistema de Comandes + Core de Gameplay.

RF-06 (TPV): TPV + Domini de Taules + Puntuació.

RF-07 (puntuació i resultats): Score/Resultats + UI + integració amb el core loop.

RF-08 (menú i accés campanya): UI + Persistència + Coordinador de sessió.

RF-09 (persistència): Persistència i progressió.

RF-10 / RF-11 (online preparat i càrrega consistent): Coordinador de sessió + Capa d'autoritat/sincronització + Selector de mapes (+ serveis online post-MVP).

3.2.1 Component: Capa de presentació (UI / Menús / HUD)

Descripció funcional: Gestiona els punts d'entrada i la navegació: menú principal (selecció de mode), selecció de nivell (campanya), sala de lobby/ready (online), HUD i pantalles de resultat, així com opcions i feedback contextual.

Requisits que resol: RF-07, RF-08, ROp-02, RQ-03.

Casos d'ús

- UC-UI-01: Seleccionar mode (campanya / coop / SoloQ).
- UC-UI-02: Seleccionar nivell (campanya).
- UC-UI-03: Mostrar HUD de partida i feedback d'estat.
- UC-UI-04: Mostrar resultats i retornar al menú.

Models/patrons

MVP (presenters), Observer per a actualitzacions de valors de joc i estat de partida.

Tipus de diagrama: Wireflow d'alt nivell

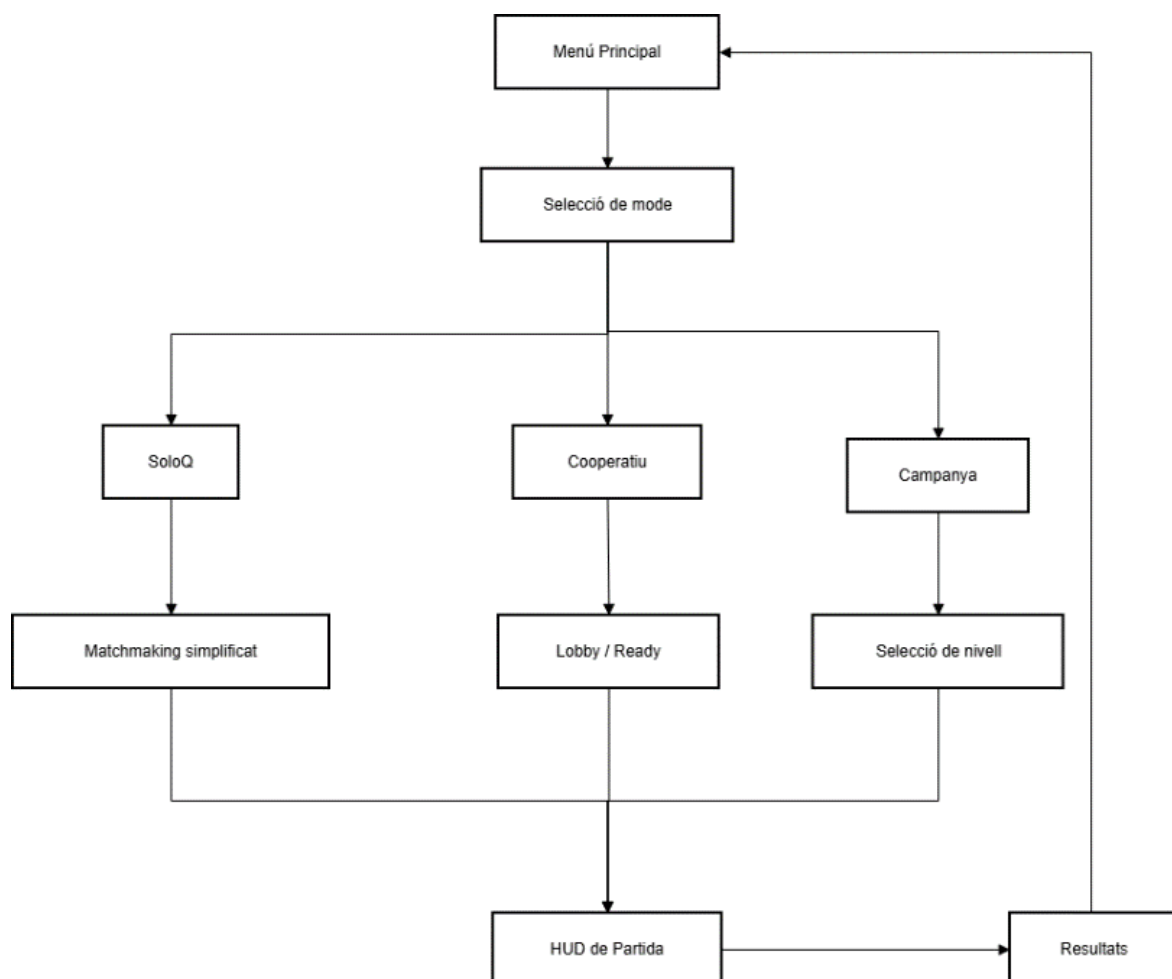


Figura 11. Wireflow d'alt nivell del projecte.

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.2 Component: Coordinador de sessió

Descripció funcional: Punt únic d'orquestració de sessió: iniciar campanya offline, iniciar online (coop/SoloQ), gestionar transicions d'escena i executar teardown net en qualsevol sortida (incloses desconnexions o errors).

Requisits: RQ-01, RQ-04, RNF-02, ROp-03.

Casos d'ús

- UC-SC-01: Iniciar sessió offline per un nivell de campanya.
- UC-SC-02: Iniciar sessió online amb sala d'espera.
- UC-SC-03: Sortida a menú des de qualsevol estat (teardown unificat).

Models/patrons

Coordinator/Facade + FSM de sessió + guard clauses.

Tipus de diagrama: FSM de sessió (alt nivell)

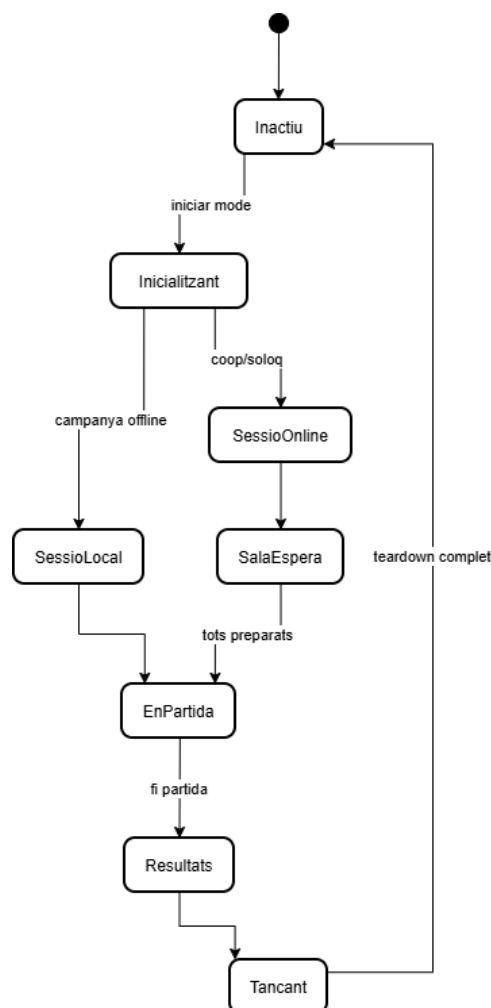


Figura 12. Diagrama FSM de sessió

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.3 Component: Persistència i progressió (offline)

Descripció funcional: Manté l'estat persistent de la campanya: nivells desbloquejats, registre de progrés i dades necessàries per habilitar selecció de nivell i continuïtat sense internet.

Requisits: RF-09, ROp-03, RNF-01.

Casos d'ús

- UC-PROG-01: Carregar progrés en entrar al menú de campanya.
- UC-PROG-02: Desbloquejar nivell en completar objectius.
- UC-PROG-03: Guardar progrés i estadístiques al final de partida.

Models/patrons

Separació dades vs servei (modularitat), fail-safe de lectura/guardat.

Tipus de diagrama: Data flow diagram

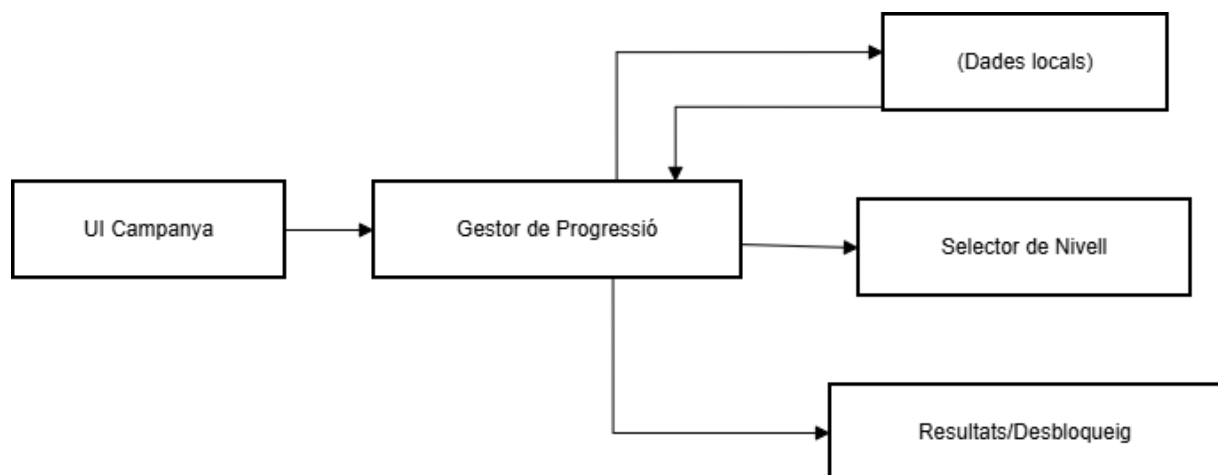


Figura 13. Data flow diagram.

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.4 Component: Pool / Selector de mapes

Descripció funcional: Defineix la política de selecció de mapa segons mode:

- Campanya: selecció per nivell desbloquejat.
- Coop: selecció coherent amb progressió del host.
- SoloQ: selecció aleatòria autoritària en servidor (determinisme).

Requisits que resol: OT-09, OT-13, RF-11.

Casos d'ús

- UC-MAP-01: Obtenir mapes disponibles segons progrés.
- UC-MAP-02: Escollir mapa segons mode.
- UC-MAP-03: Publicar mapa final per a càrrega sincronitzada.

Models/patrons

Policy/Strategy (criteri de selecció), server-authoritative en SoloQ.

Tipus de diagrama: Diagrama de decisió (selecció de mapa)

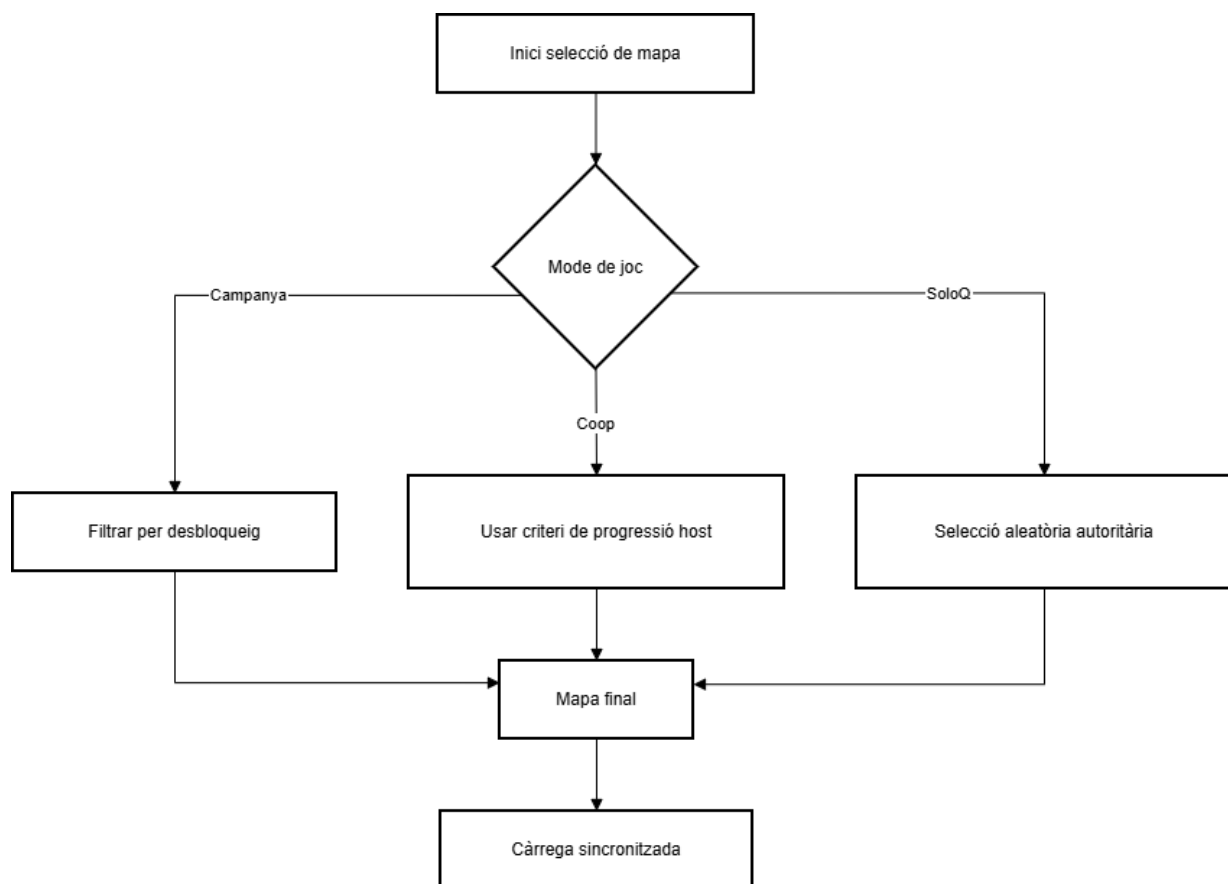


Figura 14. Diagrama de decisió (selecció de mapa)

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.5 Component: Core de gameplay (Core loop)

Descripció funcional: Implementa el bucle principal de joc: moviment i col·lisions, interaccions contextuais, transformació d'ítems, entrega de requisits, cobrament i càlcul de resultat dins sessions curtes.

Requisits que resol: RF-01...RF-07, ROp-04.

Casos d'ús

- UC-CORE-01: Interactuar amb entorn i estacions.
- UC-CORE-02: Completar servei i cobrament.
- UC-CORE-03: Finalitzar ronda i generar resultats.

Models/patrons

Separació domini/UI, comunicació per events, regles d'integritat.

Tipus de diagrama: Loop de gameplay (alt nivell)

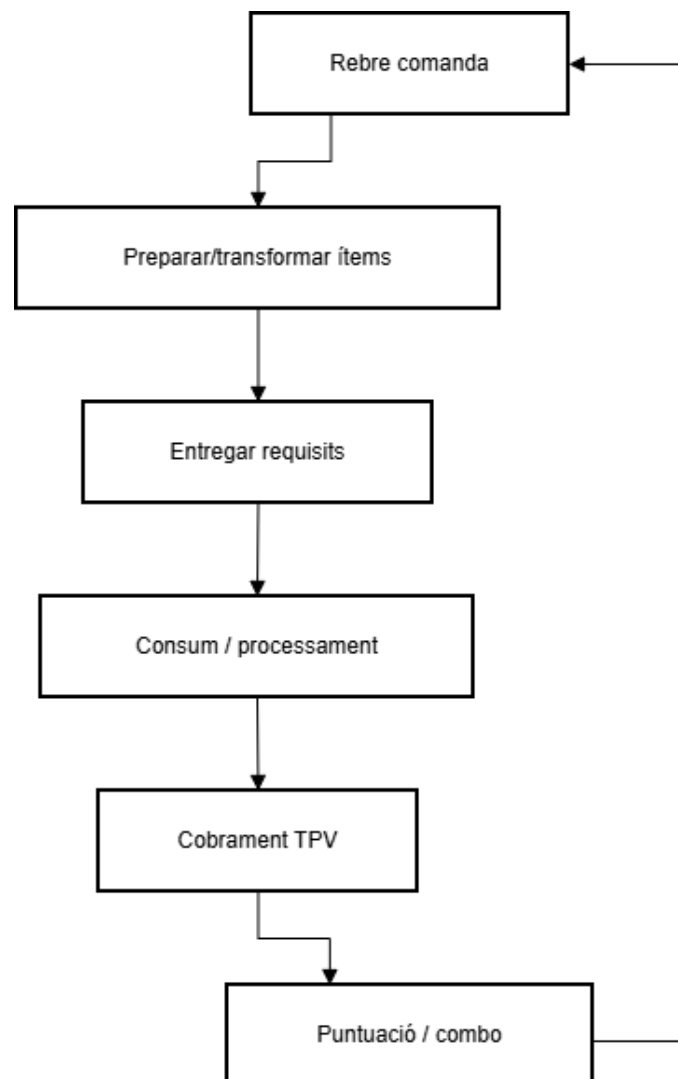


Figura 15. Loop de gameplay (alt nivell)

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.6 Component: Domini de taules (FSM de servei)

Descripció funcional: Cada taula opera com una FSM amb fases deterministes i timeouts: pre-servei (seating), finestra per acceptar comanda, entrega, consum i pagament. Qualsevol error o timeout retorna la taula a un estat segur per evitar softlocks.

Requisits que resol: RF-04, RF-06, RF-07, RQ-01, RQ-04.

Casos d'ús

- UC-TABLE-01: Activació de taula i finestra de servei.
- UC-TABLE-02: Acceptar comanda i establir requisits.
- UC-TABLE-03: Entregar ítems, consumir i passar a pagament.
- UC-TABLE-04: Completar per pagament correcte o fallar per timeout.

Models/patrons

State Pattern, fail-closed, guard clauses.

Tipus de diagrama: Diagrama d'estats de taula (alt nivell)

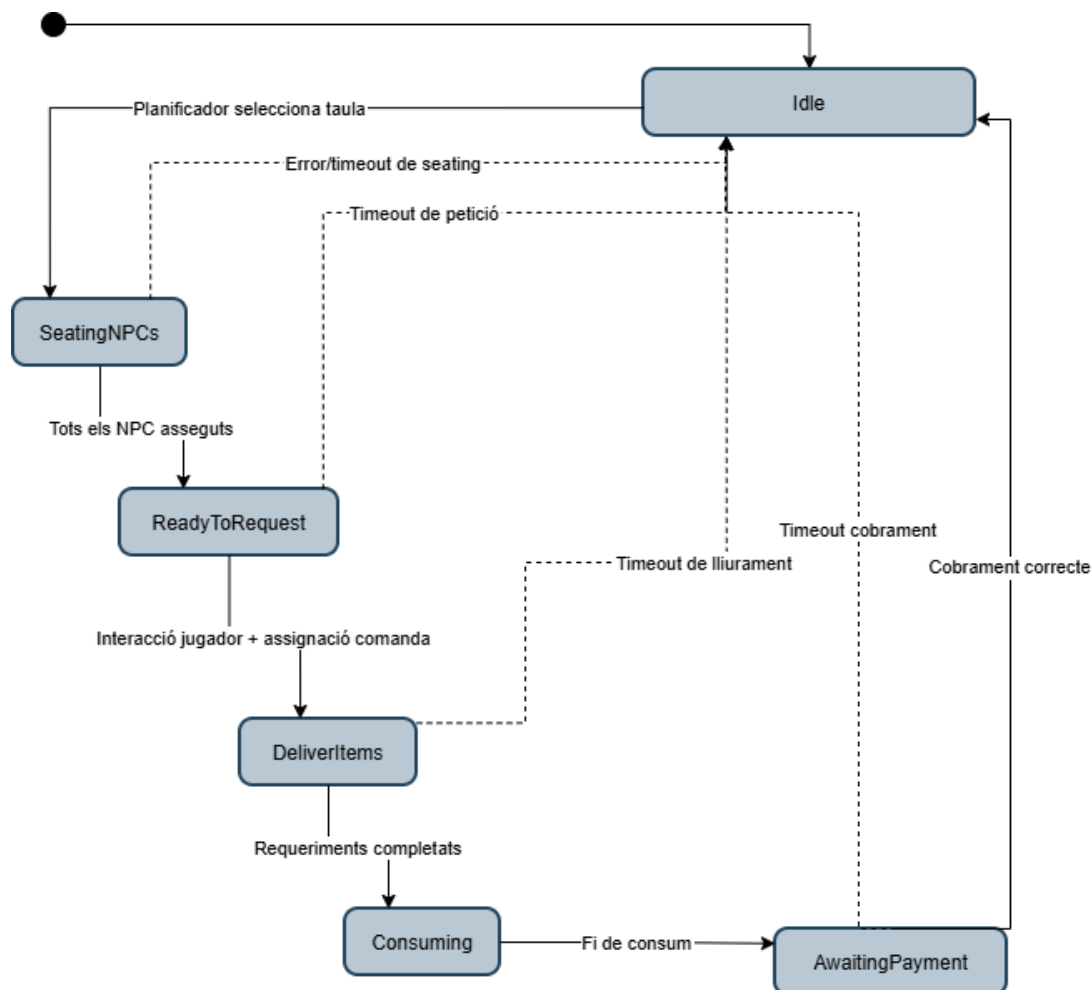


Figura 16. Diagrama d'estats de taula (alt nivell).

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.7 Component: Seating NPC (ambient pre-activació)

Descripció funcional: Abans d'entrar en fase de servei, la taula executa un cicle ambiental: spawn d'NPCs, assignació aleatòria de seients, navegació fins a l'anchor de cadira i validació "tots asseguts". En finalitzar la sessió de taula o en retornar a Idle, s'executa cleanup robust (stand-up, sortida i despawn).

Requisits que resol: OT-15, RQ-04, RNF-02.

Casos d'ús

- UC-NPC-01: Iniciar seating en una taula elegible.
- UC-NPC-02: Confirmar tots asseguts i habilitar servei.
- UC-NPC-03: Cleanup segur i alliberament de seients.

Models/patrons

Orquestració per controlador, guard clauses, timeouts, fail-closed.

Tipus de diagrama: Flux de seating/cleanup

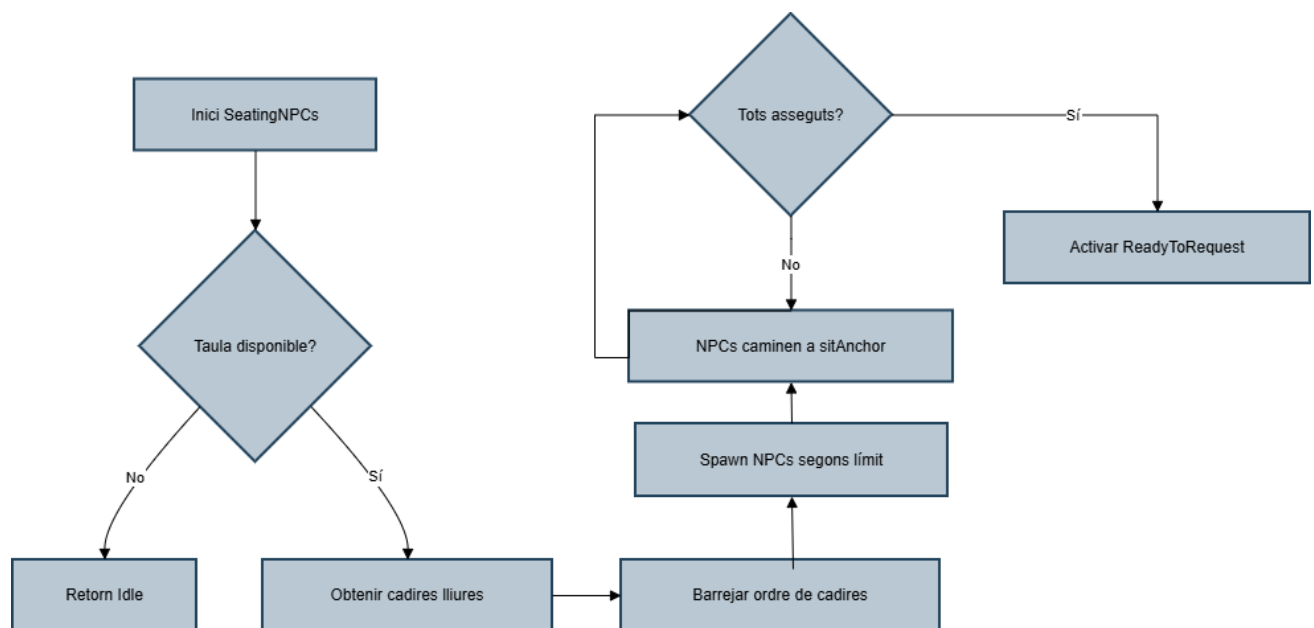


Figura 17. Flux de seating/cleanup.

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.8 Component: Sistema de comandes

Descripció funcional: Crea comandes, controla temporització i gestiona tancament per èxit o fallida. Distingeix entre comandes “no assignades” (temporitzador global) i comandes assignades a una taula (temporitzador governat per fases de servei) per mantenir coherència temporal.

Requisits que resol: RF-04, RF-07, RNF-01.

Casos d'ús

- UC-ORD-01: Generar comandes dins capacitat màxima.
- UC-ORD-02: Assignar comanda quan una taula l'accepta.
- UC-ORD-03: Completar o eliminar comanda per timeout/fracàs.

Models/patrons

Servei de domini + Adapter (contracte desacoblat).

Tipus de diagrama: Lifecycle de comanda (alt nivell)

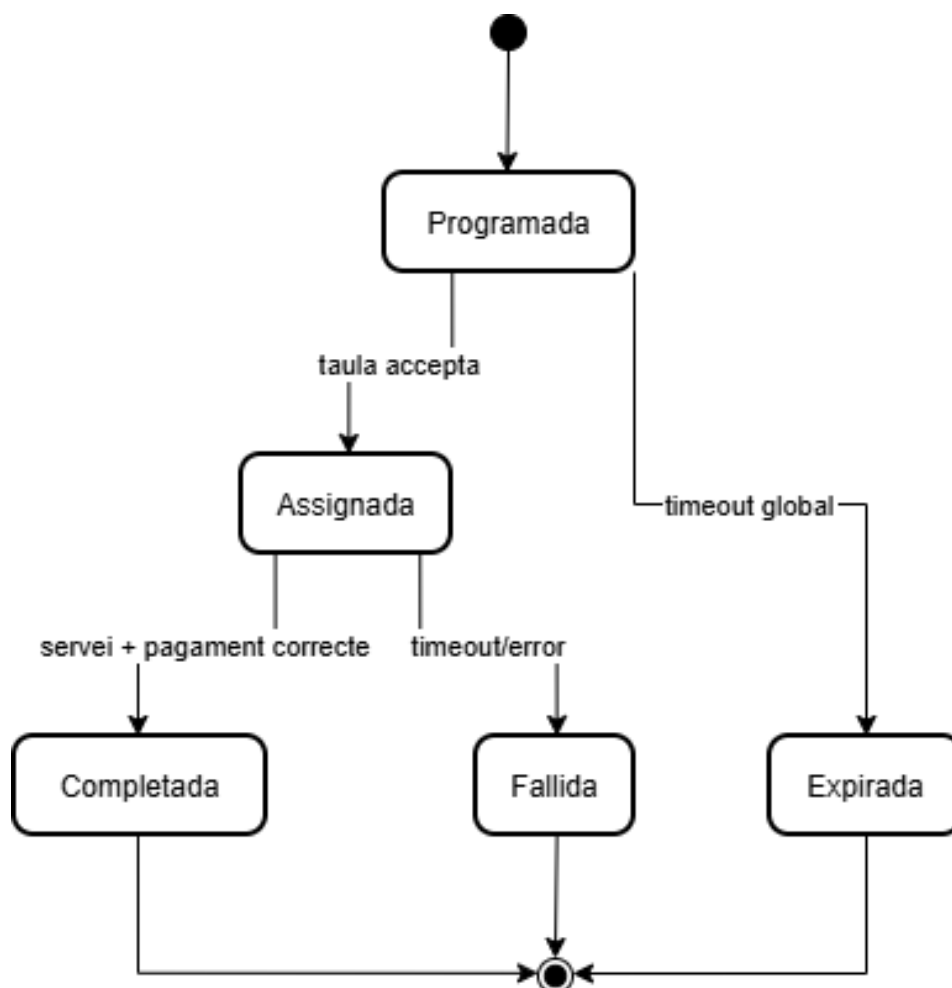


Figura 18. Lifecycle de comanda (alt nivell)

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.9 Component: TPV (pagament) — UI local + validació autoritària

Descripció funcional: El TPV es divideix conceptualment en:

1. **UI local:** recull digits, mostra import i envia sol·licitud de validació.
2. **Domini autoritària:** valida fase, condicions i import; defineix pagament correcte/perfecte i aplica punts.

Requisits que resol: RF-06, RF-07, RQ-01, RNF-02.

Casos d'ús

- UC-TPV-01: Introduir import i confirmar.
- UC-TPV-02: Validar pagament i computar “perfecte”.
- UC-TPV-03: Reintentar en cas d’error fins a timeout.

Models/patrons

Server-authoritative + guard clauses + integració amb scoring.

Tipus de diagrama: Seqüència de pagament (alt nivell)

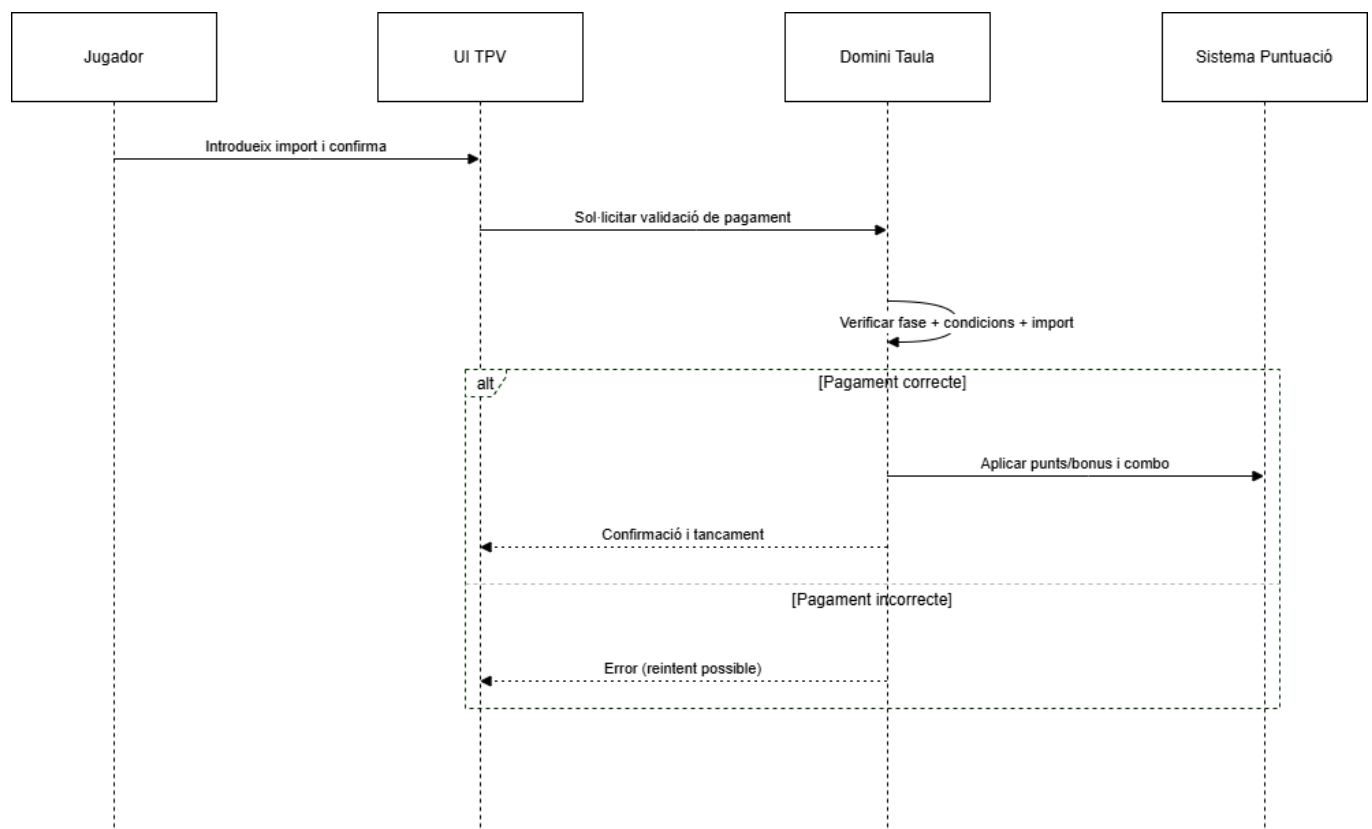


Figura 19. Seqüència de pagament (alt nivell)

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.10 Component: Sistema de puntuació i combo

Descripció funcional: Centralitza score, combo i mètriques finals de rendiment. Les penalitzacions per fallida/timeout reforcen la prevenció de softlocks i el feedback de progressió de la partida.

Requisits que resol: RF-07, RNF-02, RQ-04.

Casos d'ús

- UC-SCORE-01: Afegir punts per entrega/cobrament.
- UC-SCORE-02: Incrementar combo per execució perfecta.
- UC-SCORE-03: Reset de combo i registre de fallides.
- UC-SCORE-04: Generar mètriques (accuracy, durada màx. de combo).

Models/patrons

Server-authoritative i Observer per exposar canvis a la UI.

Tipus de diagrama: Flux de decisió de scoring (resum)

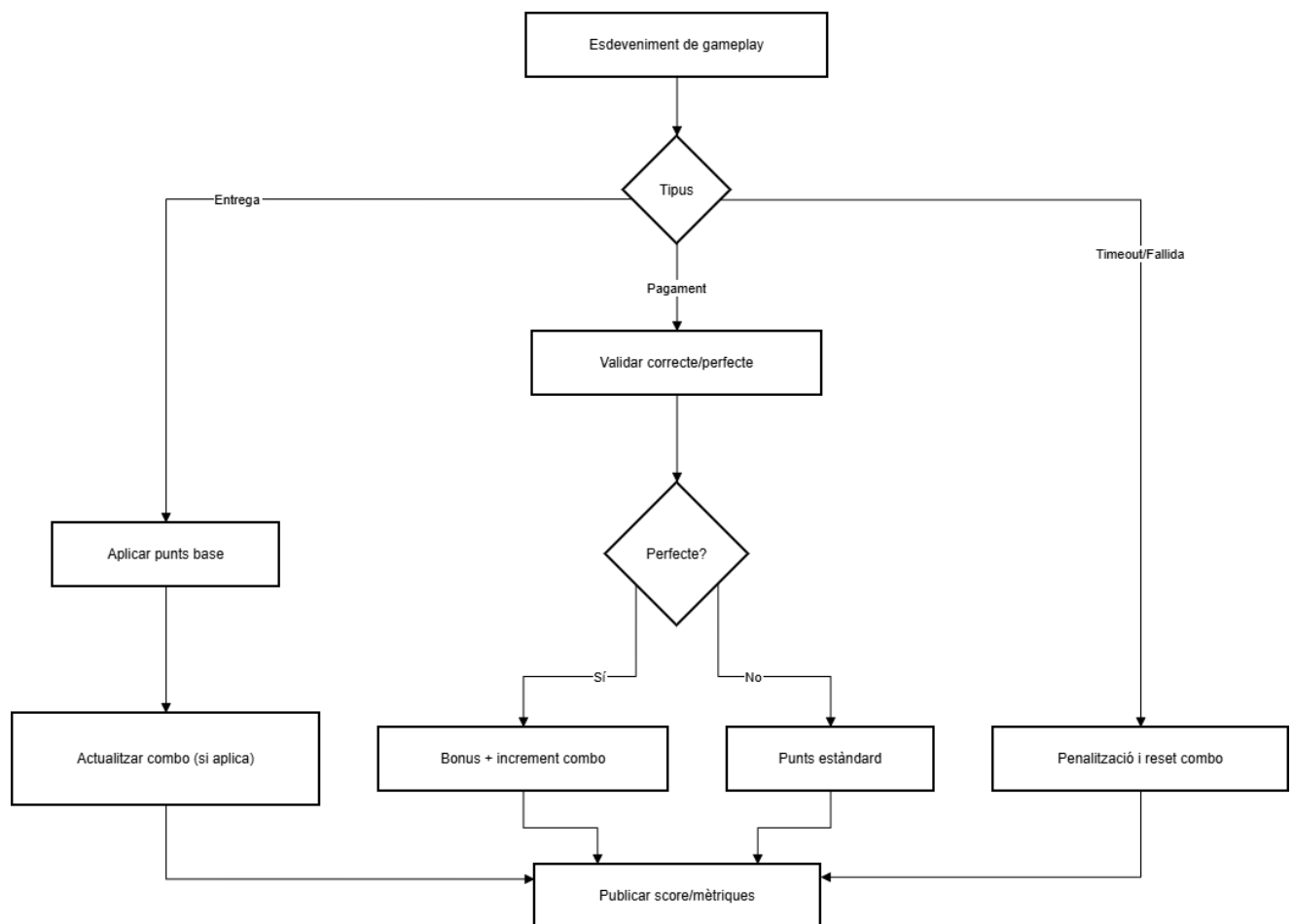


Figura 20. Flux de decisió de scoring (resum)

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.2.11 Component: Capa d'autoritat i sincronització

Descripció funcional: Model conceptual que garanteix que les accions crítiques es processen de forma autoritària i que els participants comparteixen una visió coherent del progrés i estat del joc. L'objectiu és evitar divergències, limitar errors i preservar integritat de dades en sessions online.

Requisits que resol: RNF-02, RF-10, RF-11, RQ-04, RQ-02.

Casos d'ús

- UC-SYNC-01: Validar intents d'interacció en fase correcta.
- UC-SYNC-02: Propagar canvis d'estat i informació necessària per UI.
- UC-SYNC-03: Re-sincronitzar un participant (late join / reconexió) quan correspongui (post-MVP).

Models/patrons

Server-authoritative, separació domini vs representació, events per UI.

Tipus de diagrama: Flux d'autoritat (client-servidor)

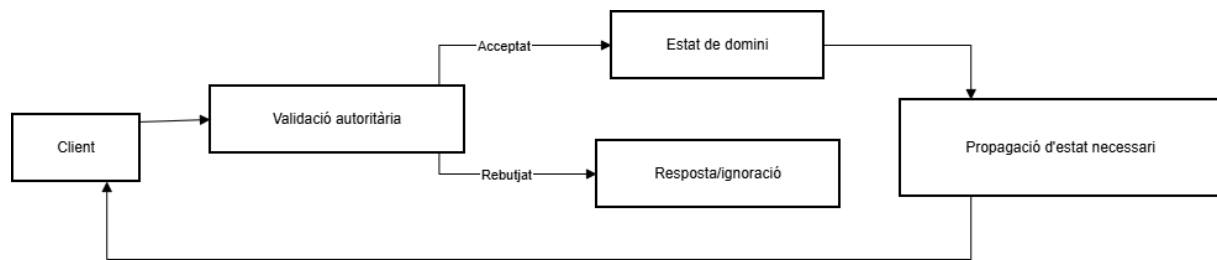


Figura 21. Flux d'autoritat (client-servidor)

Font: Elaboració pròpia mitjançant l'eina Draw.io

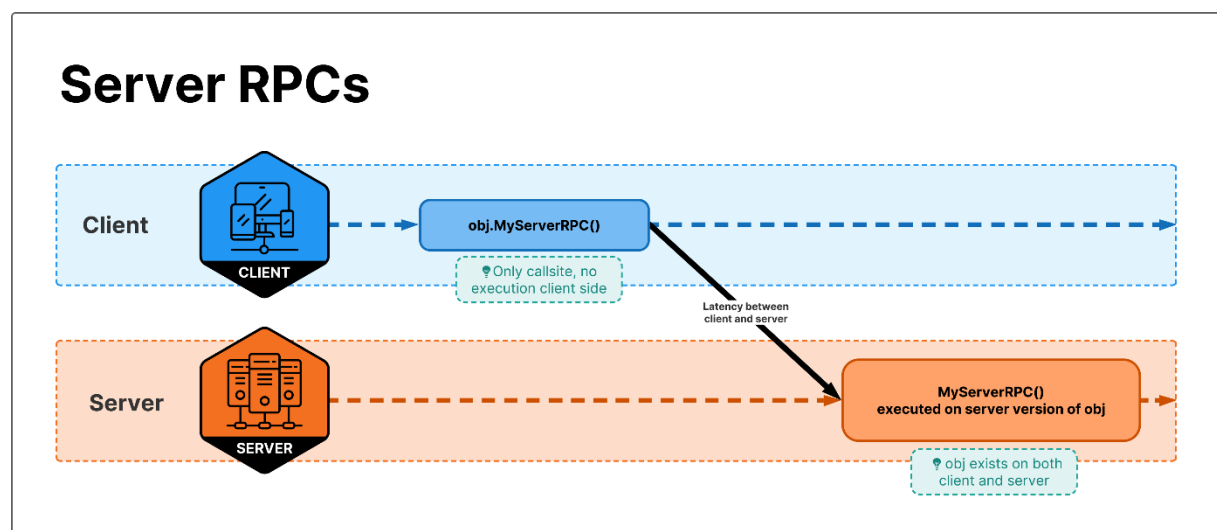


Figura 22. Messaging System [Il·lustració]. Unity Documentation.

Font: Unity Technologies. (2026).

Client RPCs

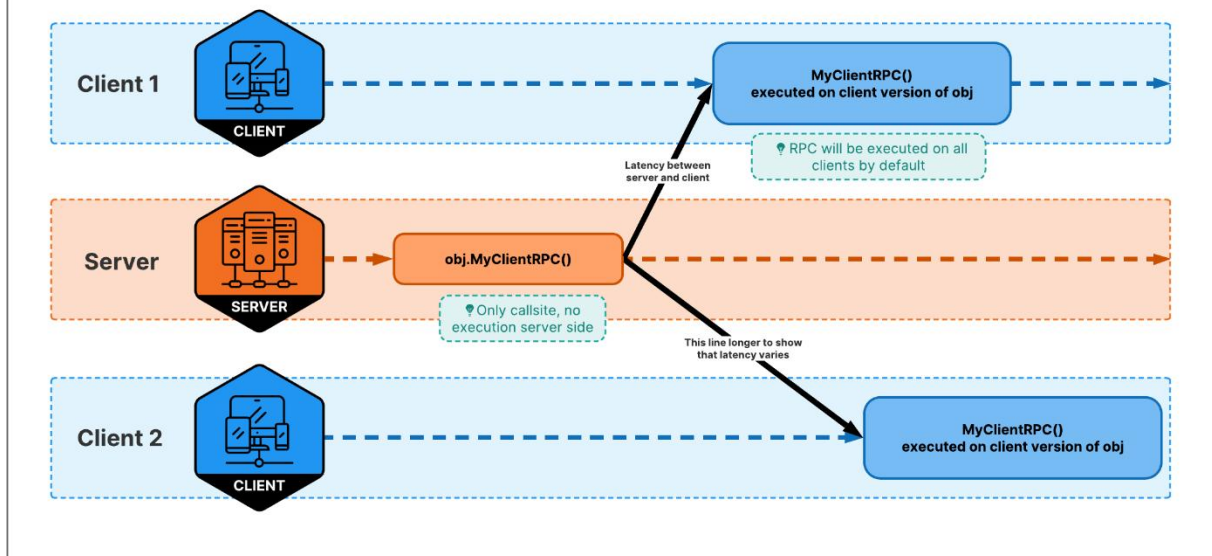


Figura 23 Messaging System [Ilustración]. Unity Documentation.

Font: Unity Technologies. (2026).

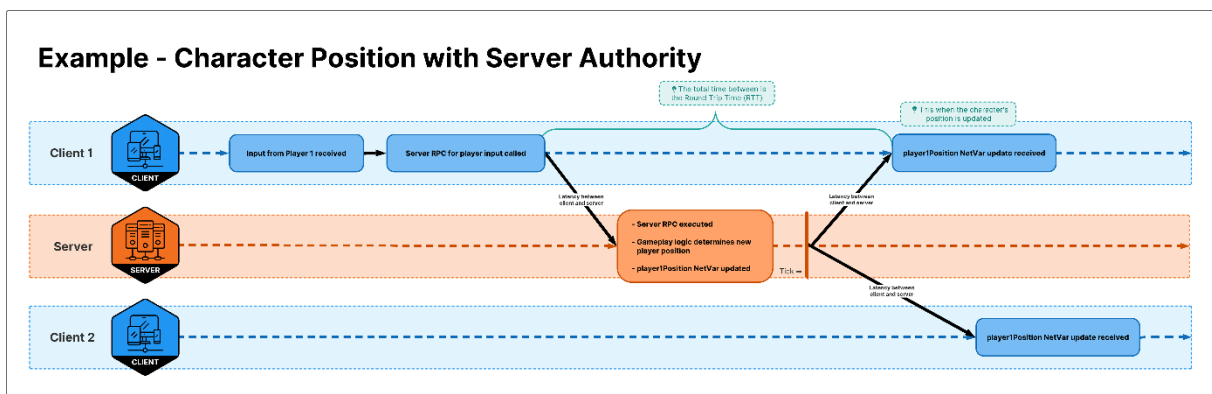


Figura 24 Messaging System [Ilustración]. Unity Documentation.

Font: Unity Technologies. (2026).

3.2.12 Component: Planificador de taules (Scheduler)

Descripció funcional: Activa periòdicament una taula elegible per iniciar el cicle de seating i servei. Evita saturació i distribueix la càrrega de forma gradual, mantenint ritme constant.

Requisits que resol: RF-04, RNF-01, RQ-01.

Casos d'ús

- UC-SCH-01: Filtrar taules elegibles.
- UC-SCH-02: Seleccionar una taula i iniciar cicle.
- UC-SCH-03: Evitar conflictes amb taules ocupades o en cleanup.

Models/patrons

Scheduler + guard clauses.

Tipus de diagrama: Activació temporal de taules

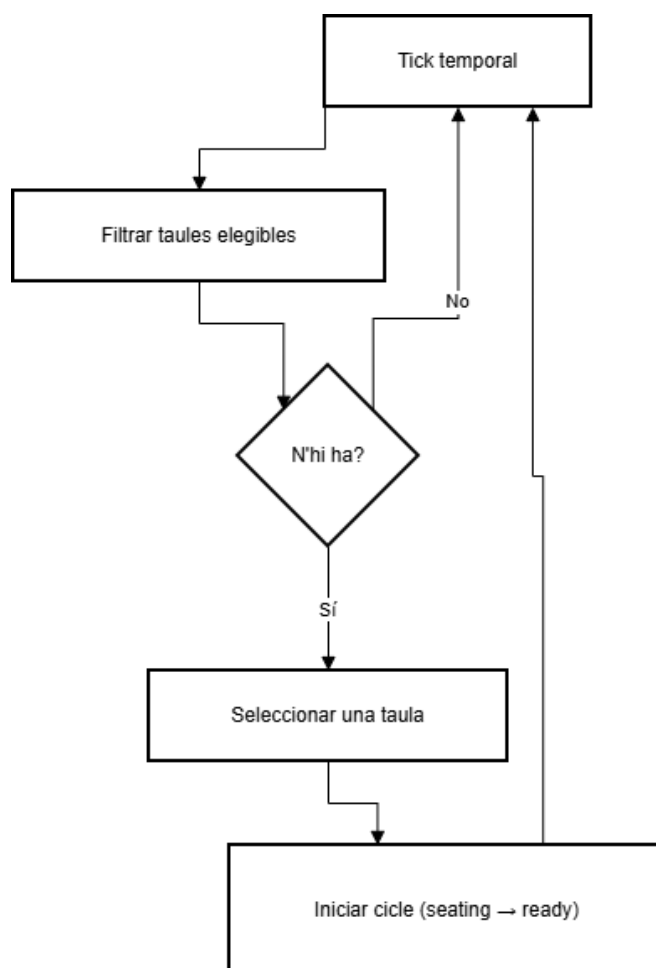


Figura 25. Activació temporal de taules.

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.3 Player - Control de jugador i interacció contextual (WASD / E / Q)

Descripció funcional: Aquest component encapsula la capa de control del personatge i la interacció contextual amb l'entorn. La seva responsabilitat principal és transformar l'entrada del jugador (teclat i ratolí) en accions de gameplay coherents amb l'estat de partida: moviment, orientació, detecció d'objectes interactuables, interacció principal (E), accions mantingudes en estacions de treball i drop d'ítems (Q). El component actua com a pont entre:

1. **Input del jugador,**
2. **domini de gameplay** (taules, estacions, objectes),
3. **capa d'autoritat/sincronització** (validació en servidor en context online o host local).

A nivell de disseny global Hybrid-Core, el component manté el mateix comportament conceptual tant en campanya offline com en modes online, evitant bifurcacions de lògica i reforçant el principi Multiplayer-First.

Requisits que resol

- **RF-01:** moviment del personatge en espai 3D amb control WASD i detecció de col·lisions.
- **RF-02:** interacció amb l'entorn mitjançant tecla E i accions de deixar objectes (Q).
- **RF-03** (parcial): integració amb la safata/inventari físic i transferència d'objectes.
- **ROp-02:** suport a teclat i ratolí amb el Unity New Input System.
- **RNF-02:** comportament compatible amb model host-authoritative i futura escalabilitat cooperativa.
- **RQ-02:** contribució a la integritat d'ítems (interaccions atòmiques i transicions consistents d'objectes).
- **RQ-01** (indirecte): guardes d'estat per evitar interaccions invàlides en fases no permeses.

Casos d'ús (alt nivell)

- **UC-PLAYER-01: Desplaçament i orientació**
El jugador mou el personatge en tercera persona i orienta direcció de marxa.
- **UC-PLAYER-02: Selecció d'objecte interactuable**
El sistema detecta i marca l'element interactuable proper en direcció de focus.
- **UC-PLAYER-03: Interacció principal (E)**
El jugador executa una interacció contextual (taula, estació, spawn, etc.).
- **UC-PLAYER-04: Interacció mantinguda en estacions**
El jugador inicia/atura treball continu segons pressió sostinguda.
- **UC-PLAYER-05: Gestió d'ítems (Q / pick-up / drop)**
Recollida i deixat d'ítems respectant capacitat i regles d'integritat.

- **UC-PLAYER-06: Interacció condicionada per fase**
Algunes accions només són vàlides en fases concretes (p. ex. pagament TPV).

Models i patrons específics del component

- **Input-Action mapping:** separació entre lectura d'input i execució de lògica de joc.
- **Observer/Event-driven:** notificació de canvis de selecció/interacció cap a UI contextual.
- **Guard clauses:** bloqueig d'accions no vàlides segons fase o condicions de portat d'objectes.
- **Server-authoritative interaction path:** les accions crítiques es validen al domini autoritari per mantenir coherència en online.

Relació amb altres components

- Amb **Core de gameplay:** aporta la capa d'entrada i locomoció del jugador.
- Amb **Domini de taules:** activa transicions de fases via interacció contextual.
- Amb **TPV:** habilita cobrament només quan les condicions de fase/ítem són correctes.
- Amb **Integritat d'ítems:** respecta les regles atòmiques de transferència d'objectes.
- Amb **Capa d'autoritat i sincronització:** envia intents d'acció i rep estat coherent de sessió.
- Amb **UI/HUD:** alimenta prompts contextuais i feedback d'interacció.

Diagrama d'activitat (entrada → decisió contextual → acció)

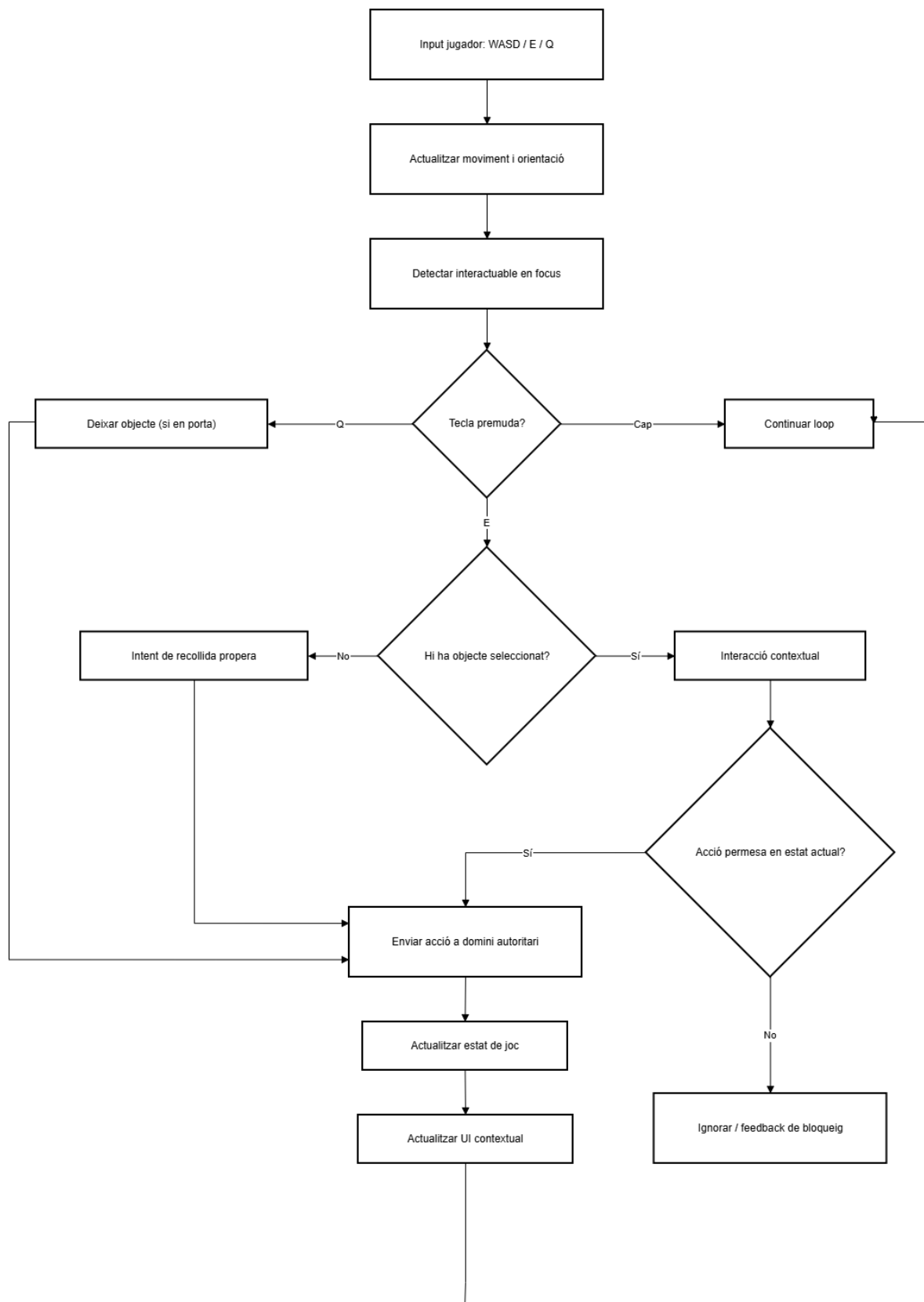


Figura 26. Diagrama d'activitat (entrada → decisió contextual → acció)

Font: Elaboració pròpia mitjançant l'eina Draw.io

Diagrama de seqüència (interacció amb validació autoritària)

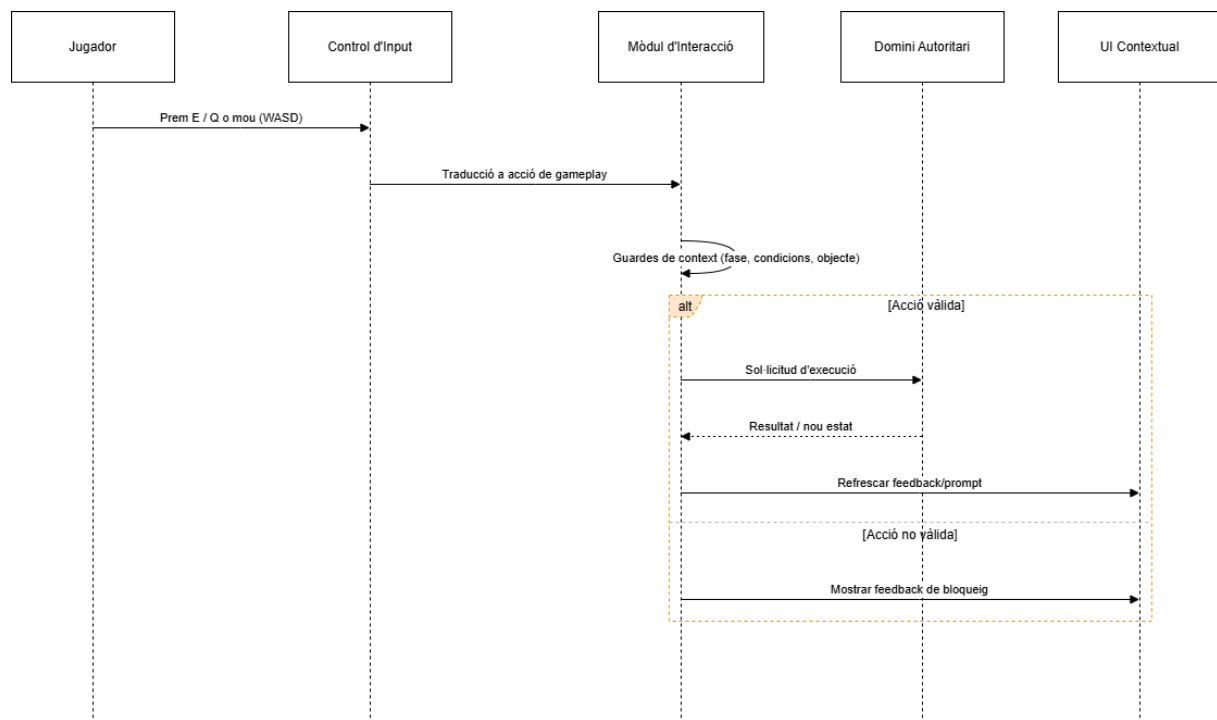


Figura 27. Diagrama de seqüència (interacció amb validació autoritària)

Font: Elaboració pròpia mitjançant l'eina Draw.io

3.3 Rendiment i restriccions operacionals (context de disseny)

El disseny contempla les restriccions:

- Plataforma objectiu Windows x64 (ROp-01)
- Entrada teclat i ratolí (ROp-02)
- Objectiu de rendiment **60 FPS** (RNF-03)

En disseny, això implica prioritzar fluxos simples, modularitat i reducció d'acoblament per facilitar optimització posterior (especialment en escenes/mapes).

Capítol 4: Implementació

4.1 Arquitectura

La implementació final s'ha construït sobre una arquitectura **Hybrid-Core**: un nucli de gameplay únic que funciona tant en execució local (campanya offline) com en execució en xarxa (host-authoritative amb clients). Aquest sistema permet que el joc funcioni sota una única base de codi tant per a mode solitari com multijugador. El mode de joc individual no és una versió simplificada, sinó una execució on la màquina de l'usuari actua simultàniament com a servidor i client. Això garanteix una compatibilitat total de les mecàniques sense necessitat de duplicar codi o realitzar refactoritzacions posteriors.

L'elecció d'aquesta arquitectura per al videojoc respon a la voluntat d'aplicar aquestes **millors pràctiques professionals** (Tot i que la complexitat del TFG ha escalat considerablement). Mitjançant aquest enfocament, s'assegura que qualsevol millora en el "Game Feel" o en la lògica de les estacions de servei s'apliqui de forma simètrica a tots els modes de joc, optimitzant els recursos de desenvolupament i garantint una experiència d'usuari robusta i lliure de desincronitzacions.

Tot i que el plantejament inicial contemplava opcions centrades en Steamworks, la implementació consolidada del MVP final adopta UGS Lobby/Relay com a base operativa principal per reduir fricció d'infraestructura. Això vol dir que la infraestructura multijugador està preparada per funcionar en qualsevol plataforma i no només Steam. Considero que és un bon canvi per temes d'escalabilitat.

Els components tecnològics principals són:

- **Motor i pipeline:** Unity + URP.
- **Networking:** Unity Netcode for GameObjects (NGO) + Unity Transport (UTP).
- **Serveis online:** Unity Gaming Services (Authentication, Lobby, Relay).
- **Persistència local:** PlayerPrefs + fitxers de progrés de campanya/economia.
- **Patrons d'arquitectura:** State, Observer/Event-driven, MVP a la capa UI etc.

(Veure Annex AT-12 per identificar tots els patrons de programació que aplica el projecte)

La implementació actual valida el disseny Hybrid-Core que hem plantejat en capítols anteriors amb execució funcional de: campanya offline, flux online host/client amb lobby i relay i sincronització de partida i resultats.

Diagrama d'arquitectura de components

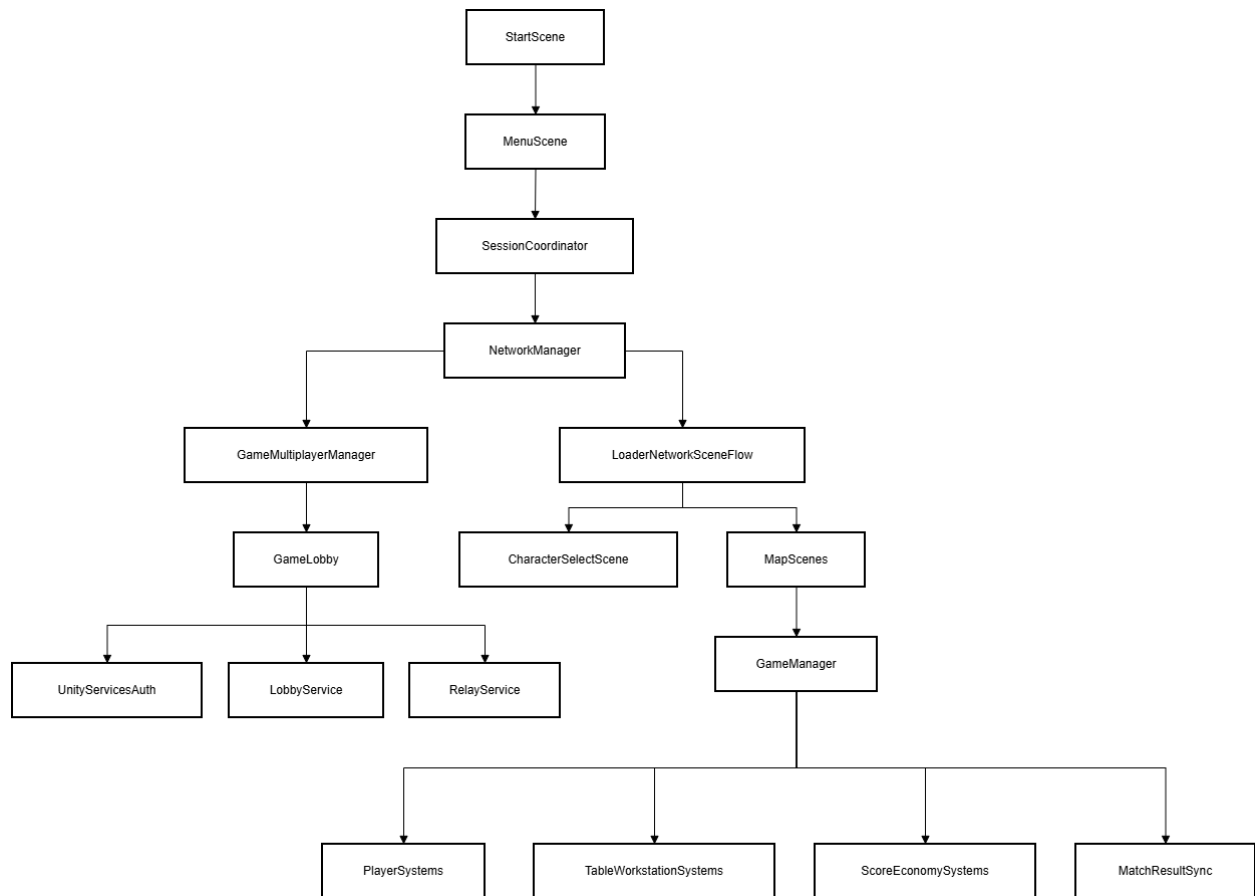


Figura 28. Diagrama d'arquitectura de components

Font: Elaboració pròpia mitjançant l'eina Draw.io

4.1.2 Flux general de funcionament

Flux offline (campanya/tutorial):

1. L'usuari entra al menú i selecciona nivell quan prem un botó (Imatge).
2. SessionCoordinator crea/assegura NetworkManager.
3. El host local s'inicia en local a 127.0.0.1:7777.
4. Loader.LoadNetwork(...) carrega el mapa objectiu.
5. GameManager inicialitza estat, spawns i lògica de partida.

Flux online (multijugador):

1. GameLobby inicialitza serveis online i autenticació anònima.
2. Host crea lobby + assignació Relay; clients fan join per codi/ID o QuickMatch.
3. GameMultiplayerManager arrenca host/client i aplica connection approval.
4. Host carrega CharacterSelectScene; en ready total es carrega el mapa.
5. El servidor governa estat de partida, puntuació i resultats.

Diagrama de flux online de inici de sessió

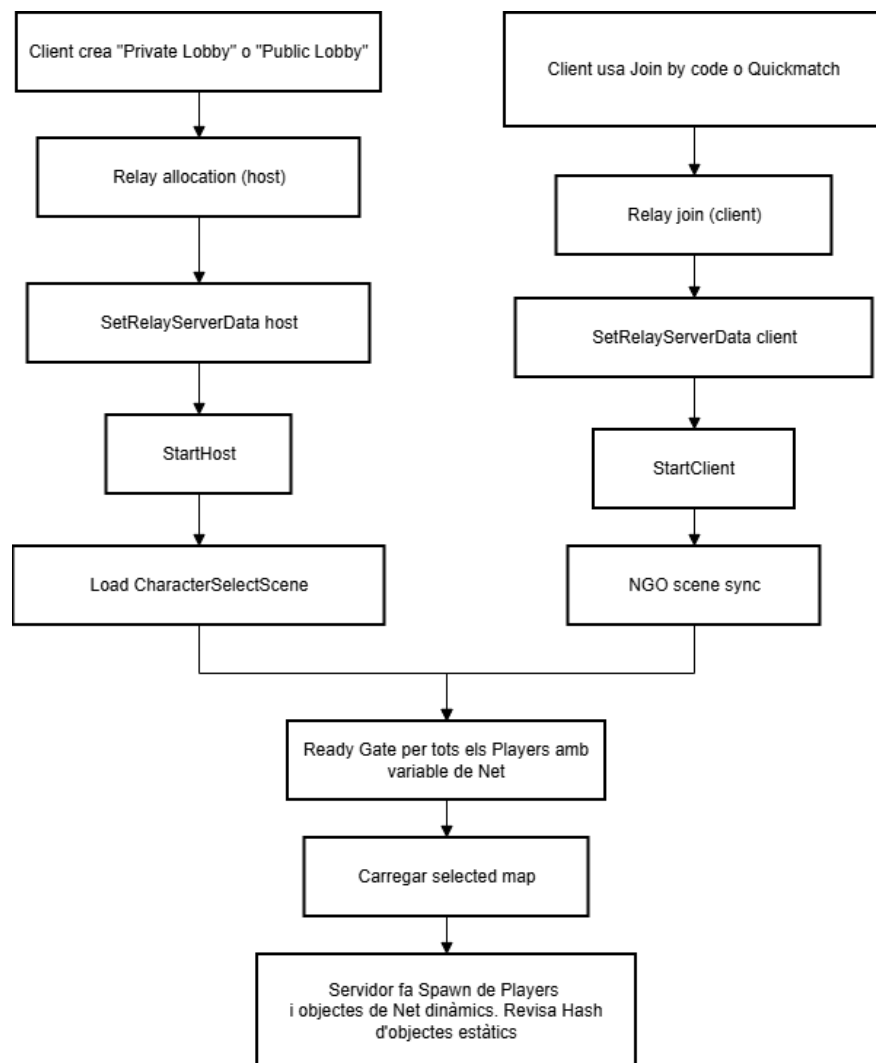


Figura 29. Diagrama de flux online de inici de sessió

Font: Elaboració pròpia mitjançant l'eina Draw.io

Flux de navegació de l'usuari: Start → Menu → Lobby → CharacterSelect → Map → Resultats → Menu.

4.1.3 Decisions tecnològiques clau

NGO + UTP: Es tria per integració nativa amb Unity i menor fricció d'operació respecte alternatives com Mirror/FishNet.

Lobby + Relay: Resol matchmaking i connectivitat sense infraestructura pròpia de backend. Casi automàtic. Podem escollir si volem servers propers.

Host-authoritative: Minimitza inconsistències de joc i simplifica integritat de dades. També per temes de cybersseguretat.

Teardown centralitzat: A `SessionCoordinator.LeaveToMainMenuAsync()` evita softlocks i estat brut entre partides. Fem un reinici net.

4.2 Components

4.2.1 Coordinació de sessió i cicle de vida

El component **SessionCoordinator** és el punt d'entrada de sessió: crea `NetworkManager` si no existeix, inicia host local i executa teardown complet per tornar al menú.

Tecnologies i alternatives

- Implementat amb singleton controlat (`DontDestroyOnLoad`) i flux asíncron.
- **Alternativa:** scene bootstrap amb Service Locator/DI complet. Es va descartar la implementació completa del *Service Locator* per evitar un sobrecost de computació personal; es va prioritzar un lliurament funcional per assegurar que el projecte veiés la llum abans que l'autor deixés de veure-la. Esta mitg implementat al codi del projecte.

Dificultats i solucions

- **Dificultat:** tancar lobby/xarxa/contextos sense Race conditions entre clients.
- **Solució:** Vaig aplicar ordre de shutdown explícit:
`LeaveLobby` → `NetworkManager.Shutdown` → `reset` contexts → `Loader.Load(MenuScene)`.

Codi/config determinant

- `Assets/02_CODE/NETCODE/COORDINATOR/SessionCoordinator.cs`

- **Mètodes clau:**

EnsureNetworkManager(): Verifica la presència del NetworkManager a l'escena i, en cas de no existir, l'instancia des d'un prefab per garantir la disponibilitat del sistema de xarxa. Va ser molt útil al principi quan no estava tot unificat en una mateixa escena.

StartSingleplayerHostLocal(...): Configura el transport a l'adreça local, inicia el motor de xarxa en mode *Host* i executa la càrrega sincronitzada de l'escena de joc objectiu.

LeaveToMainMenuAsync(): Gestiona de forma asíncrona la sortida del lobby, el tancament del servei de xarxa, el buidatge dels contextos de dades i el retorn a l'escena del menú principal.

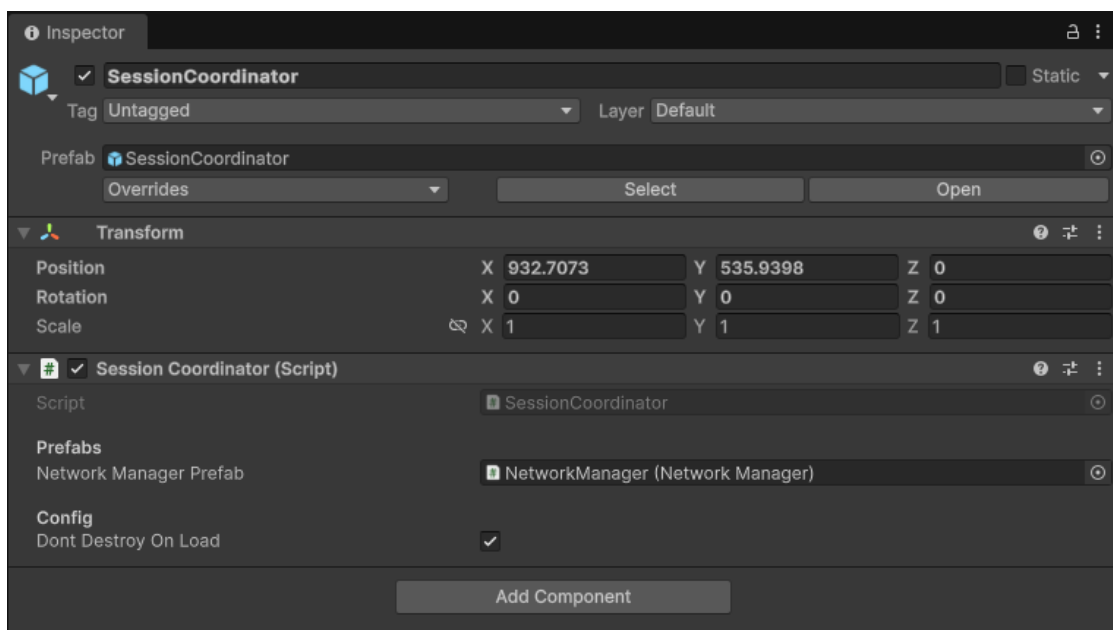


Figura 30. Imatge del component Session Coordinator a Unity GO.

Font: Elaboració pròpia. Entorn de Unity 6

Annex relacionat: Annex AT-01 (Flux de teardown i recuperació de sessió).

4.2.2 Lobby, relay i bootstrap multijugador

GameLobby i GameMultiplayerManager governen el cicle de partida online: creació/join de lobby, assignació Relay, connexió host/client, validació d'entrada i manteniment de llista de jugadors. No tenim “Late joiners” implantat actualment.

Tecnologies i alternatives

- UGS Authentication + Lobby + Relay.
- Alternatives no adoptades en aquesta fase: backend dedicat amb matchmaking propi. Es va avaluar el desenvolupament d'un backend dedicat amb un sistema de matchmaking de propi disseny. No obstant això, aquesta alternativa es va desestimar per tal d'optimitzar el temps de desenvolupament i reduir els costos operatius inicials. La solució integrada d'UGS cobreix íntegrament els requeriments funcionals del Mínim Producte Viable (MVP) amb una API madura i una infraestructura nativa en el motor que minimitza els riscos d'integració.

Dificultats i solucions

- **Dificultat:** concurrència i intents duplicats de QuickMatch.
- **Solució:** guardes de single-flight (_quickMatchInFlight) i neteja best-effort en errors (SafeLeaveQuickMatchLobbyAsync).

Codi/config determinant

- Assets/02_CODE/Lobby/GameLobby.cs
- Assets/02_CODE/NETCODE/GameMultiplayerManager.cs
- Ús de ConnectionApprovalCallback i NetworkConfig.ConnectionData.

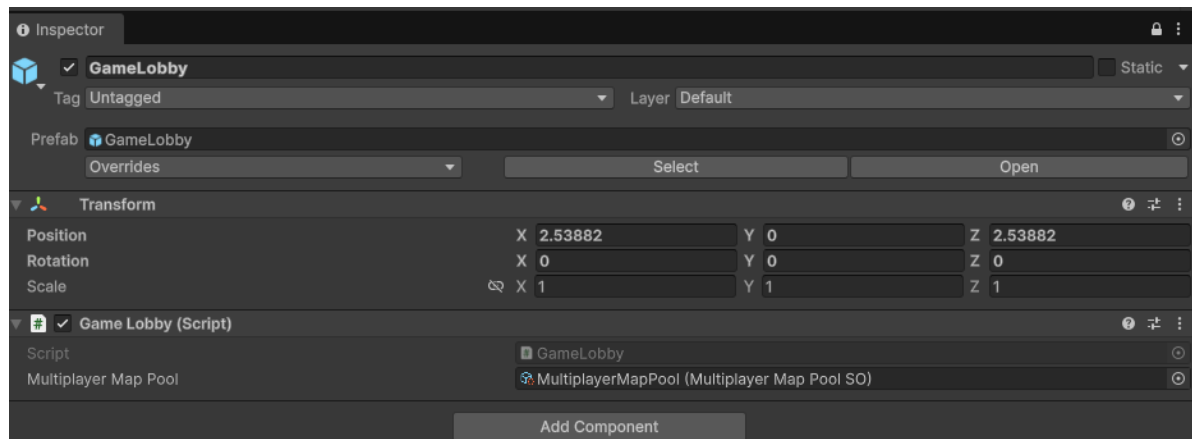


Figura 31. Imatge del component Lobby a Unity GO.

Font: Elaboració pròpia. Entorn de Unity 6

Annex relacionat: Annex AT-02 (Traça completa host/client i relació amb Lobby/Relay)

4.2.3 Gestor de partida i estat de match

GameManager governa la màquina d'estats de partida (MainMenu, Waiting, Countdown, Playing, GameOver) i la sincronització temporal de match.

Tecnologies i alternatives

- NetworkVariable<State> + timers replicats quantitzats.
- Alternativa: RPC intensiu per cada tick. No adoptada per sobrecost de xarxa i major fragilitat.

Dificultats i solucions

- **Dificultat:** Reduir soroll de replicació i mantenir timers consistents i visualment nets. Es a dir evitar la sensació de que els scrollbar s'actualitzen a trosos sense enviars RPC's cada tick.
- **Solució:** Vaig aplicar quantització configurable. (countdownTimerReplicationStep, playingTimerReplicationStep).

Codi/config determinant

- Assets/02_CODE/Managers/GameManager.cs
- **Mètodes:** Update(), SceneManager_OnLoadEventCompleted(...), PublishMatchResultsServer().

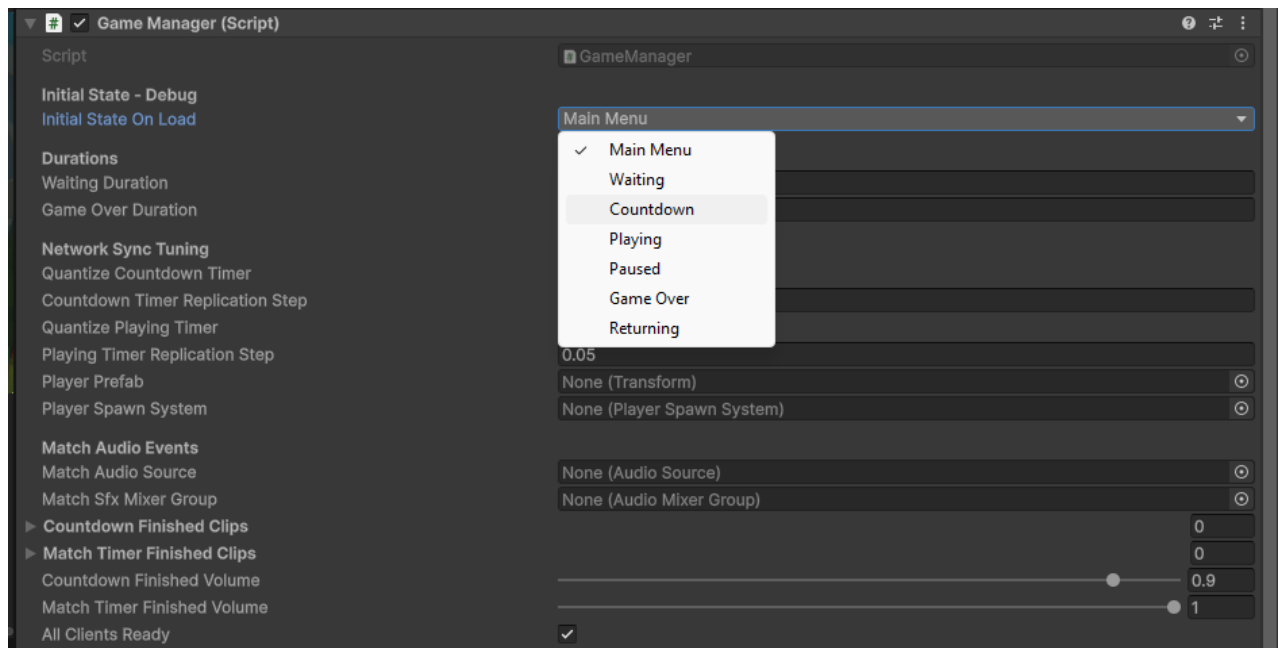


Figura 32. Component de gestor de partida i estat de match a Unity GO.

Font: Elaboració pròpia. Entorn de Unity 6

Annex relacionat: Annex AT-03 (Màquina d'estats i transicions de match).

4.2.4 Personalització i sincronització de jugador

La personalització visual de personatge es desa localment i es replica de forma compacta a xarxa. Un cop només.

Tecnologies i alternatives

- Estructura binària CustomizationData en NetworkVariable.
- Alternativa inicial: JSON llarg dins FixedString4096Bytes. Substituïda per compactació per eficiència i robustesa.

Dificultats i solucions

- **Dificultat:** volum de payload i costos de sincronització. La primera prova penjada al github es veu com triga en carregar molt al principi.
- **Solució:** representació compacta (10 bytes per jugador/sync) i aplicació server-side.

Codi/config determinant

- Assets/02_CODE/Player/Player SOLID/PlayerCharacterCustomized.cs
- **Camp clau:** _netCustomization.

```
private string _PLAYER_PREFS_SAVE = "PlayerCustomizationOptions";

// 10 bytes per player per sync – replaces LEGACY NetworkVariable<FixedString4096Bytes>
private NetworkVariable<CustomizationData> _netCustomization =
    new(default, NetworkVariableReadPermission.Everyone, NetworkVariableWritePermission.Server);

[SerializeField] private BodyPartData[] bodyPartDataArray;

public enum BodyPartType
{
    Hair,
    HairAcc,
    Beard,
    Top,
    Legs,
}
```

Figura 33. Fragment de codi per personalització i sincronització de jugador. Player Prefs.

Font: Visual Studio Code IDE

4.2.5 Core gameplay: interacció, taules, TPV i puntuació

El bucle preparar, servir i cobrar s'articula sobre sistemes de taula/estació, validació server-authoritative de TPV i ScoreManager per resultats de match.

Tecnologies i alternatives

- Patró State a dominis de taula/estació. De fet aquesta solució em va permetre afegir fàcilment el nou estat de taula bruta.
- RPC orientat a accions de jugador + validació central al servidor.
- **Alternativa descartada:** La lògica de puntuació client-side (massa exposada a divergències que em causaven problemes).

Dificultats i solucions

- **Dificultat:** evitar desalineació entre feedback local i autoritat de resultat.
- **Solució:** feedback local immediat + ack servidor per esdeveniments de pagament crítics.

Codi/config determinant

- Assets/02_CODE/Managers/GameManager.cs
- Assets/02_CODE/NETCODE/... (taules/carry/workstations/score sync)

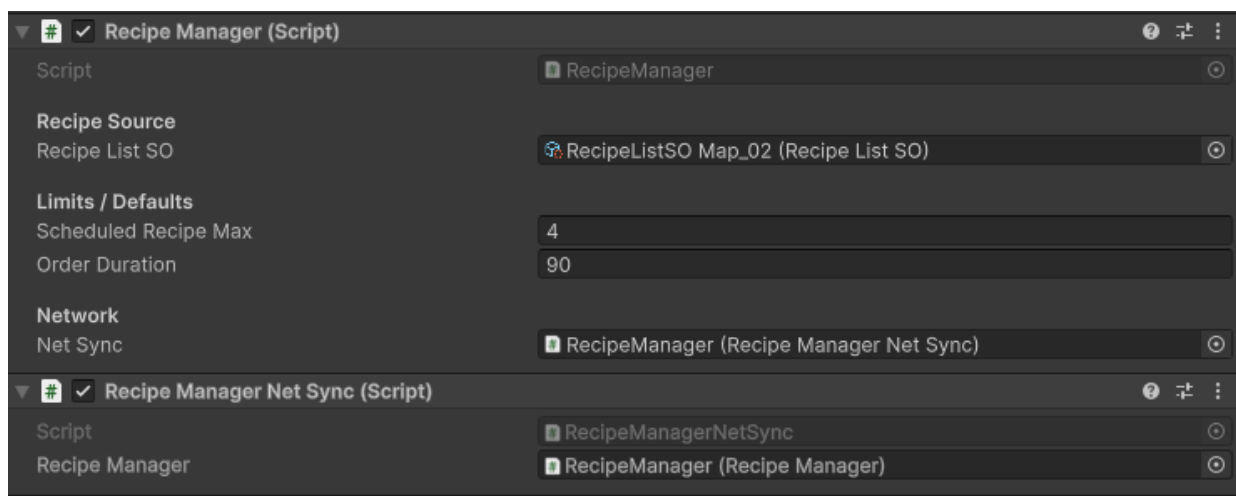


Figura 34. Component Recipe Manager a Unity GO.

Font: Elaboració pròpia. Entorn de Unity 6

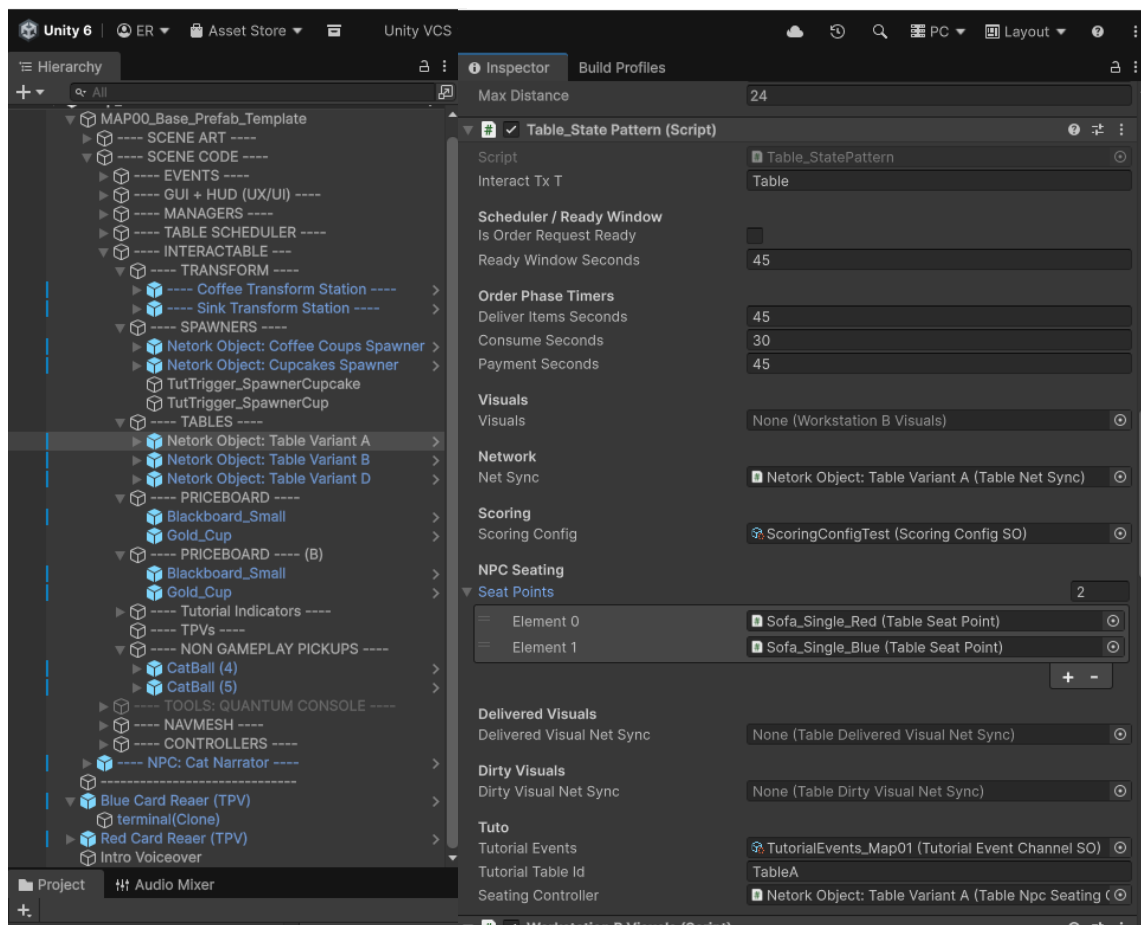


Figura 35. Table State Pattern a l'entorn de Unity 6.

Font: Elaboració pròpia. Entorn de Unity 6

4.2.6 UX, tutorial i controls ampliat

A partir de playtests s'han afegit:

- ✓ indicadors de tutorial,
- ✓ moviment point-and-click,
- ✓ pick up/drop amb mouse,
- ✓ reforç de feedback sonor contextual.

Tecnologies i alternatives

- Integració sobre Input System + presentadors UI existents.
- Alternatives amb remapeig complet de controls no adoptades per impacte en abast. Si premeu Escape a una partida surt una mitja implementació per continuar el remapeig.

Dificultats i solucions

- **Dificultat:** incorporar noves interaccions sense trencar el flux de xarxa.
- **Solució:** mantenir autoritat del servidor en accions de joc i deixar local el feedback no crític com sliders.

Canvis Core: El disseny inicial estava centrat en WASD/E/Q; després de playtests la UX final incorpora **point-and-click** i **interacció amb mouse** com a millora d'accessibilitat i onboarding. Just com em van indicar els tutors els resultats dels playtesters em van fer incorporar aquestes funcionalitats.

Annex relacionat: Annex AT-06 (Resultats de playtesting i impacte en UX).

4.2.7 Persistència, campanya i contextos de runtime

El projecte manté campanya offline real i progressió local, a la vegada que separa contextos de campanya/coop/tutorial per no contaminar sessions.

Tecnologies i alternatives

- Contextos estàtics controlats i netejats en teardown.
- Save local orientat a simplicitat operativa.

Dificultats i solucions

- **Dificultat:** evitar arrastres d'estat entre partides i modes.
- **Solució:** CampaignRuntimeContext.Clear(), CoopCampaignSessionContext.Clear(), TutorialRuntimeContext.Clear() en sortida unificada.

Annex relacionat: Annex AT-04 (Governança d'estat entre modes de joc).

4.3 Desenvolupament i execució

4.3.1 Entorn tecnològic de desenvolupament

- **Sistema operatiu de desenvolupament:** Windows 11.
- **Motor:** Unity (LTS) amb URP.
- **Llenguatge:** C#.
- **IDE/editor:** Visual Studio / Visual Studio Code (segons etapa).
- **Control de versions:** Git + GitHub.
- **Serveis online en desenvolupament:** UGS (Auth, Lobby, Relay).

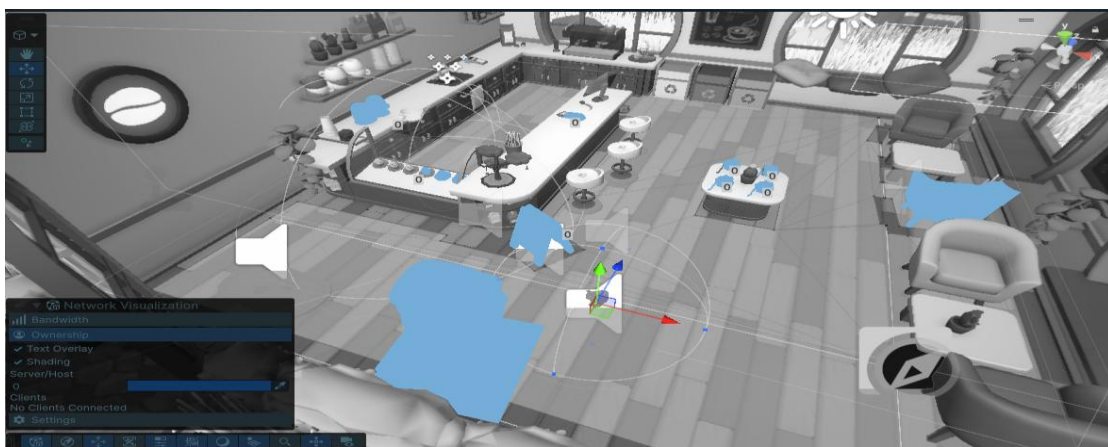


Figura 36. Scene view. Network status Ownership

Font: Elaboració pròpia. Entorn de Unity 6.

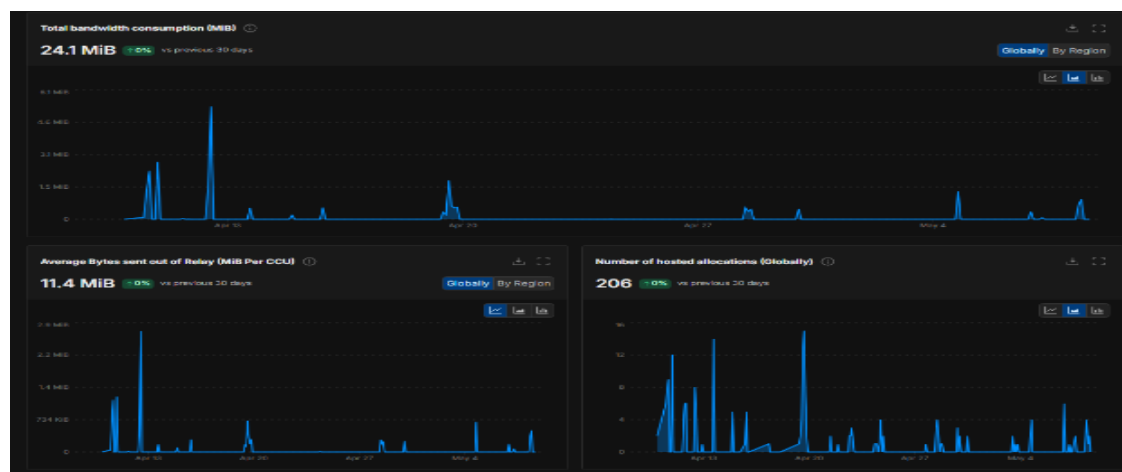


Figura 37. Monitoratge del bandwidth i dades consumides.

Font: Unity Game Services

4.3.2 Entorn necessari d'execució

Execució objectiu

- **Plataforma:** Windows x64.
- **Entrada:** teclat + ratolí.
- **Connexió:** no requerida per campanya; requerida per funcionalitat online.

Requisits mínims

- CPU 4 nuclis moderna.
- 8 GB RAM.
- GPU compatible amb URP.
- 2 GB d'espai lliure.

Requisits recomanats

- CPU 6+ nuclis.
- 16 GB RAM.
- GPU dedicada d'entrada/mitja gamma.



Figura 38. Ingame Skins Tab Scene View

Font: Unity 6 Editor

4.3.3 Posada en marxa (desenvolupament)

1. Clonar repositori.
2. Obrir projecte Unity.
3. Verificar paquets de Netcode, Transport, Authentication, Lobby, Relay.
4. Validar Build Settings (StartScene, MenuScene, CharacterSelectScene, Map_XX).
5. Executar test local offline.
6. Executar test host/client per validació online.

4.3.4 Procés de build i distribució

1. Seleccionar target Windows x64.
2. Crear build incremental (Alpha/Beta/RC segons fita).
3. Validar smoke test post-build.
4. Publicar paquet (itch.io intern/beta).

4.3.5 Annexos de suport (detall tècnic)

- Annex AT-01: Flux de sessió i teardown.
- Annex AT-02: Integració Lobby/Relay i errors habituals.
- Annex AT-03: Estat de partida i replicació.
- Annex AT-04: Persistència i contextos.

Capítol 5: Proves

5.1 Definició

S'ha elaborat un pla de proves mixt complet i coherent amb els requisits tècnics del projecte, indicant les proves realitzades i els requisits que s'estan avaluant en cada cas. Orientat a garantir:

- Correcció funcional del core loop.
- Robustesa de networking host-authoritative.
- Estabilitat entre versions (regressió).
- Usabilitat de controls i onboarding.

Tipologies de prova aplicades:

- **Smoke tests** de build.
- **Black-box functionals** per fluxos de joc.
- **Integració de xarxa** (host/client, lobby, relay, sincronització).
- **Regressió** després de canvis de sistema.
- **Rendiment** (estabilitat FPS i càrrega de sessió).
- **Playtests** per UX i aprenentatge del jugador.

Matriu de traçabilitat prova-requisit

ID Prova	Tipus	Requisits coberts	Condicció inicial
T-01	Smoke	ROp-01, ROp-02	Build neta en Windows
T-02	Black-box	RF-01, RF-02	Sessió iniciada, jugador spawnat
T-03	Black-box	RF-03, RQ-02	Safata buida, objectes interactuables
T-04	Black-box	RF-04, RF-05	Taules/estacions actives
T-05	Black-box	RF-06, RQ-01	TPV obert i comanda pendent
T-06	Black-box	RF-07	Partida en curs (3 min)
T-07	Black-box	RF-08, RF-09	Menú principal + campanya
T-08	Integració xarxa	RF-10, RNF-02	Host i client en xarxa
T-09	Integració xarxa	RF-11, OT-13	Lobby amb mapa seleccionable

T-10	Robustesa	RQ-01, RQ-04	Desconnexió simulada a mig flux
T-11	Regressió	RNF-01, RNF-04	Canvi de codi sobre sistema base
T-12	Rendiment	RNF-03	Sessió completa amb càrrega normal
T-13	Playtest UX	OT-01, OT-15	Jugador nou sense briefing tècnic
T-14	Playtest UX	RF-02, ROp-02	Controls mouse + click-move activats
T-15	Regressió online	RF-06, RF-07, RQ-04	TPV en host/client després de canvis d'àudio

5.2 Avaluació de requisits

En aquest apartat es mostren els resultats per cadascuna de les proves identificades anteriorment (amb captures) tot explicitant si el resultat s'alineja amb l'esperat.

5.2.1 Resultats de proves funcionals i smoke

ID	Resultat esperat	Resultat obtingut	Estat
T-01	Build executable arrenca i arriba a menú	Correcte en proves repetides	Assolit
T-02	Moviment/interacció sense bloquejos	Correcte; sense bloquejos crítics	Assolit
T-03	Integritat d'ítems (sense duplicació)	Correcte en escenaris habituals	Assolit
T-04	Cicle de taula/estació consistent	Correcte amb transicions d'estat esperades	Assolit
T-05	TPV valida import i aplica feedback	Correcte; error/success coherents	Assolit
T-06	Timer, puntuació i final de match correctes	Correcte; retorn controlat al menú	Assolit
T-07	Menú/campanya/guardat operatius	Correcte en campanya offline	Assolit

5.2.2 Resultats de proves de networking i robustesa

ID	Resultat esperat	Resultat obtingut	Estat
T-08	Host/client entren a partida via Lobby+Relay - Host pot expulsar clients.	Correcte amb join code i quick match	Assolit
T-09	Selecció de mapa coherent amb autoritat host	Correcte en flux principal	Assolit amb observacions
T-10	Recuperació neta després de desconnexió/sortida	Retorn a menú i neteja de sessió correctes	Assolit
T-15	TPV online no regressiona amb canvis d'àudio/feedback	Correcte; feedback del pagador sense trencar validació	Assolit

Observació tècnica: Tenim que part de la inicialització de gameplay té validació explícita centrada en Map_01 i Map_02. En el disseny conceptual vull un escalat lineal de mapes; en implementació final alguns punts de cicle de partida mantenen lògica explícita per mapa i requereixen retoc addicional per escalar mapes nous. Es a dir, ara mateix esta una mica hardcoded.

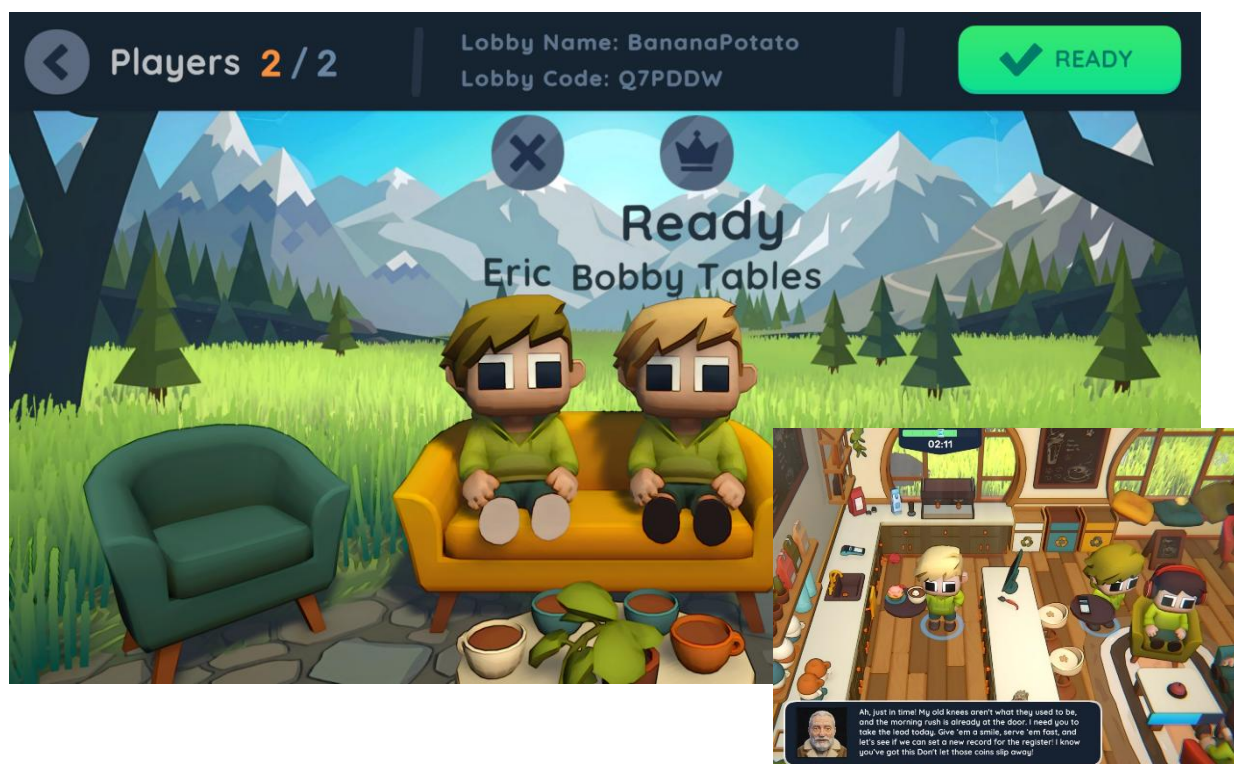


Figura 39 Visualització del videojoc en diferents escenes d'Unity executades en Development Build.

Font: Unity 6 – PC Development Build

5.2.3 Resultats de regressió i rendiment

ID	Criteri	Resultat	Estat
T-11	Sense regressions crítiques en bucle principal	No s'han detectat regressions bloquejants	Assolit
T-12	Estabilitat de rendiment objectiu (RNF-03)	Comportament estable en escenaris habituals de joc	Assolit



Figura 40. Visualització del videojoc. Comandes i fase de pagament.

Font: Unity 6 – Development build

5.2.4 Resultats de la sessió de playtesting I i II (Diada Universitària)

Es van executar sessions de playtesting amb google forms orientades a:

- comprensió del loop sense explicació tècnica extensa,
- facilitat d'interacció amb objectes i TPV,
- percepció de feedback visual/sonor.

Principals canvis induïts per playtesting:

- incorporació d'indicadors de tutorial contextuals,
- incorporació de control point-and-click,
- incorporació de pick up/drop amb mouse.
- Ajustament del tutorial (Temps entre accions - Diada)

Tamany de la mostra del playtest I i II (Diada): 8 persones

ID Obs.	Observació Detectada	Solució Implementada	Impacte / Canvi en el Disseny
UX-01	Confusió en l'ús de tecles [Q] (Drop) vs [E] (Interact) per deixar objectes a les estacions.	Sistema de "Snap" automàtic en estacions de treball i màquina de cafè.	Millora de la fluïdesa i reducció d'errors d'usuari en el loop d'entrega.
UX-02	Els jugadors esperen poder jugar exclusivament amb el ratolí (sense dependre del teclat).	Implementació d'un sistema híbrid (teclat + mouse + click-move) via New Input System.	Reducció de la fricció d'entrada i millora dràstica de l'onboarding.
UX-03	Incomprensió de les mecàniques bàsiques per part dels usuaris que ometen el tutorial.	Incorporació d'una guia visual permanent de controls a la interfície (inferior dreta).	Millora de la comprensió del loop de joc sense requerir explicació tècnica prèvia.
UX-04	Intent d'interacció amb elements de l'escenari que són purament decoratius.	Reducció del "soroll visual" i eliminació d'objectes no interactuables que induïen a error.	Clarificació del feedback visual i millora de la percepció d'objectes actius.

Link: <https://docs.google.com/forms/d/1kRi7YAUGFarfTHt7typJU7j1ztXRHIBzCQfy3HdMpYE/edit>

5.3 Autoavaluació d'objectius

En aquest apartat avaluarem si els resultats obtinguts en les proves permeten validar la solució plantejada i, per tant, resolen els objectius identificats inicialment.

Objectiu	Autoavaluació
OT-01	Assolit (control de jugador + interacció contextual, ampliat amb mouse/click).
OT-02	Assolit (core loop complet i jugable en sessions curtes).
OT-03	Assolit (TPV funcional amb validació i impacte en puntuació).
OT-04	Assolit (gestió de cicle de comanda amb màquines d'estat).
OT-05	Assolit amb ajustos (inventari/safata implementats i refinats en iteració).
OT-06	Assolit (arquitectura modular amb patrons).
OT-07	Assolit (host-authoritative i compatibilitat offline).
OT-08	Assolit (campanya operativa sense dependència de xarxa).
OT-09	Assolit amb observacions (maps implementats; escalat de lògica de match a mapes futurs pendent d'unificació total).
OT-10	Assolit (persistència local de progrés).
OT-11	Assolit (coop amb codis d'unió sobre Relay).
OT-12	Assolit (quick match simplificat funcional).
OT-13	Assolit (selecció de mapa sota autoritat de host/servidor).
OT-14	Assolit parcial alt (millora de lifecycle i teardown; marge de refinament en escenaris límit).
OT-15	Assolit (feedback visual/sonor i iteració per playtests).

5.3.2 Valoració global

Es considera que els resultats de les proves obtinguts validen la solució proposada. La jugabilitat principal és robusta, el mode en línia és funcional i coherent amb l'arquitectura definida, i les iteracions de l'experiència d'usuari (UX) han millorat l'accessibilitat inicial. Finalment, el producte es mostra viable com a base per a una futura evolució comercial.

Capítol 6: Distribució, manteniment i evolució

6.1 Distribució

En aquest apartat definirem com s'empaqueta i es distribueix la solució per a ser implantada en els sistemes on s'ha d'executar i utilitzar. Definir també el procés per a instal·lar o actualitzar la solució.

6.1.1 Empaquetat i canals de distribució

La solució es distribueix com a build Windows x64 empaquetada en carpeta executable:

- Game.exe
- carpetes *_Data
- fitxers de dependències.

Canals de distribució previstos:

- **Canal intern/acadèmic:** paquet zip per tribunal i testers.
- **Canal beta:** itch.io (proves obertes controlades).
- **Canal comercial principal:** Steam (fase de llançament).

Nota: S'ha adoptat una distribució esglaonada (acadèmic → beta → botiga) en comptes d'un salt directe únic a botiga comercial.

6.1.2 Procés d'instal·lació i actualització en fase beta.

Instal·lació

1. Descarregar paquet.
2. Descomprimir en directori local.
3. Executar Game.exe.

Actualització

1. Fer còpia de seguretat de dades locals si cal.
2. Substituir fitxers de build per la nova versió. (Les dades es conserven en APPDATA)
3. Executar i validar versió des del menú principal.

6.2 Manteniment

En aquest apartat definirem quin nivell de suport i manteniment requereix la solució implementada d'acord amb els perfil de la solució desenvolupada.

6.2.1 Model de suport

S'estableixen tres nivells de manteniment:

- **Correctiu:** errors funcionals o regressions.
- **Adaptatiu:** canvis de compatibilitat de plataforma/SDK.
- **Perfectiu:** millores de UX, rendiment i qualitat de contingut.

6.2.2 Perfils i responsabilitats

- **Programació gameplay/network:** incidències de lògica i sincronització.
- **Disseny de joc:** ajust de dificultat, economia i pacing.
- **QA:** regressions, smoke i validació de requisits.
- **Producció/documentació:** changelog, versions i comunicació.

6.2.3 Protocol operatiu de manteniment

1. Recepció d'incidència.
2. Classificació severitat (bloquejant, alta, mitjana, baixa).
3. Reproducció i diagnòstic.
4. Fix + prova de regressió focalitzada.
5. Publicació de patch a la pàgina web i notes de versió.

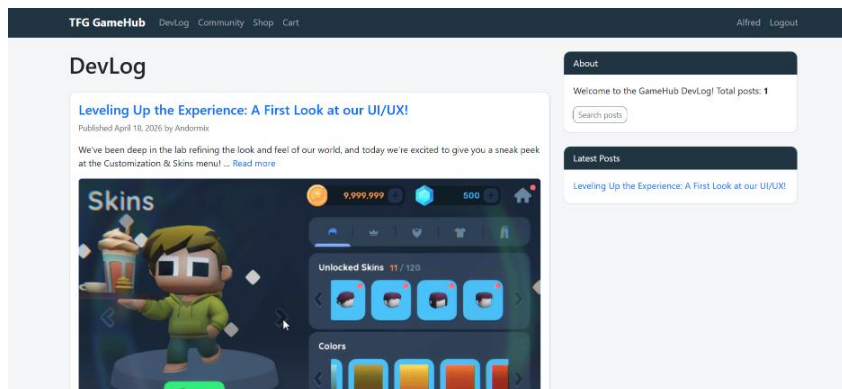


Figura 41. Interfície web del portal comunitari amb la secció de DevLog.

Font: Elaboració pròpia. Django.

6.2 Evolució

Definirem quines són les línies de treball futur per a evolucionar la solució creada, tot establint un calendari de versions amb dates de publicació i les novetats que s'espera incorporar en cada evolució.

6.3.1 Línies de treball futur

- Escalat de mapes i harmonització de fluxos de cicle de partida.
- Enriquiment de contingut (nous objectius, variacions de taules i tipus de clients, nous events, modificadors de partida).
- Optimització continuada de UX i tutorialització.
- Millores de qualitat de vida online (observabilitat i robustesa de sessions).
- Incorporació de la botiga de Skins amb moneda Diamant (Moneda forta).

6.3.2 Calendari de versions previst

Versió	Data prevista	Novetats principals
v0.9.0 (RC TFG)	2026-06-15	Tancament de deute funcional crític + documentació final.
v1.0.0 (Public Beta)	2026-09-20	Distribució externa controlada, recollida de feedback, tuning d'economia.
v1.1.0	2026-11-30	Millores de UX/tutorial i estabilitat networking en escenaris límit. Botiga.
v1.2.0	2027-02-15	Nou paquet de contingut (mapa/situacions) i refinament de progressió.
v1.3.0	2027-05-30	Preparació pre-llançament comercial (qualitat, analytics bàsics, polishing).

Capítol 7: Estratègia de màrqueting i llançament

L'objectiu d'aquest apartat és definir el full de ruta per transformar el projecte acadèmic en un producte comercial viable, centrant-nos en la generació de comunitat i el posicionament a botigues digitals.

7.1 Cronograma de visibilitat

L'estratègia es divideix en tres fases clau basades en l'estat del desenvolupament:

Fase de "Vertical Slice" (Producte presentable):

- Un cop assolit un nivell visual i jugable satisfactori (Alpha o entrega del TFG), es procedirà a la creació de material promocional: *screenshots* d'alta qualitat i un *teaser* de 30 segons.
- **Publicació a xarxes socials:** Obertura de perfils a TikTok i X (Twitter) per mostrar píndoles de *gameplay* (estil *devlog*), enfocades a la temàtica andorrana del joc per captar el nínxol local i els amants dels *party games*.

Presència a Steam (Generació de wishlists):

- **Steam Direct:** Pagament de la quota de 100 € per activar la pàgina de la botiga.
- **L'estratègia del wishlist:** L'objectiu principal és acumular "Llistes de desitjos" abans del llançament fins al límit que farem Release. Steam utilitza aquesta mètrica per decidir quins jocs promociona a la seva portada el dia de la sortida. Publicar la pàgina mesos abans permet que l'algoritme comenci a treballar a favor del projecte.

Demo pública i feedback:

- Llançament d'una versió reduïda (Demo) durant el Steam Next Fest. Aquest esdeveniment és la millor eina gratuïta per a desenvolupadors *indie* per guanyar milers de *wishlists* en pocs dies.

7.2 Canals de difusió

- **Comunitat local:** Contacte amb mitjans i *influencers* d'Andorra. El fet que el joc se situï a Andorra és un diferencial únic que pot generar interès mediàtic regional. En aquest sentit
- **Itch.io:** Utilització d'aquesta plataforma per a les primeres versions de prova (Beta) i recollir *feedback* tècnic de la comunitat de desenvolupadors abans del salt definitiu a Steam.
- **Discord de Andormix Entertainment:** Creació d'un servidor per centralitzar els *beta-testers* i convertir els jugadors més actius en ambaixadors del joc.
- **Creadors de contingut i “streamers”:** Es durà a terme una campanya de relacions públiques dirigida a creadors de contingut de plataformes com Twitch i YouTube.

Màrqueting de proximitat (Nínxol Andorra): Donada la ubicació de l'estudi a Sant Julià de Lòria, es contactarà amb creadors residents al país (com podria ser el cas d'AuronPlay, entre d'altres perfils de referència) per oferir-los còpies gratuïtes del joc. L'objectiu és aprofitar la vinculació geogràfica i la temàtica local del joc per despertar el seu interès.

Distribució de keys: S'utilitzaran plataformes com *Keymailer* o enviaments directes per fer arribar el producte a *streamers* de l'àmbit hispanoparlant que se especialitzin en *party games* o jocs cooperatius, facilitant així la seva viralització orgànica.

7.3 Objectius de Conversió (Ampliació)

El focus no estarà en la publicitat pagada, sinó en el **màrqueting orgànic**:

- **SEO de Steam:** Optimització de les etiquetes (*tags*) com "Multiplayer", "Co-op", "Funny" i "Physics-based" per aparèixer en cerques relacionades.
- **Disseny per a "Moments compartibles" (Viral Loops):** S'ha posat especial èmfasi en què les mecàniques basades en físiques i el caos inherent al sistema cooperatiu generin situacions visuals còmiques i inesperades. Aquests "moments compartibles" són la base per a una estratègia de contingut en format curt (TikTok, Instagram reels i YouTube shorts). L'objectiu és que el propi *gameplay* serveixi de material de màrqueting natural, facilitant la viralització a través de clips de jugadors i *streamers*.
- **Participació en festivals de demos (Steam Next Fest):** El llançament d'una demo pública durant els festivals de Steam serà el principal motor de conversió a *wishlists*. Aquests esdeveniments permeten obtenir una visibilitat massiva i recollir dades de comportament dels jugadors abans del llançament definitiu.
- **El Saló del videojoc (Andorra Telecom):** Com a part estratègica del màrqueting local, el projecte buscarà tenir presència al Saló del Videojoc i la Creativitat

Digital organitzat per Andorra Telecom. Aquesta acció es considera una fita clau per diversos motius:

- **Validació amb usuaris reals (Playtesting):** El Saló és l'entorn ideal per realitzar proves de camp massives. Observar com jugadors de diferents edats interactuen amb la mecànica de la safata o el multijugador permetrà detectar errors de disseny i ajustar la dificultat abans de la publicació a Steam.
- **Generació de "Hype" local:** Establir un estand on els visitants puguin provar *El Pas de la Caixa* servirà per crear una base de seguidors andorrans. Es fomentarà que els jugadors afegixin el joc a la seva Wishlist *in situ* mitjançant codis QR, vinculant l'experiència física amb la conversió digital.
- **Relacions públiques (Networking):** L'esdeveniment és un punt de trobada amb altres professionals del sector i premsa local. La naturalesa del joc (ambientat a Andorra) té un alt potencial per ser notícia en mitjans nacionals, amplificant l'abast del projecte sense cost publicitari.
- **Tornejos i dinàmiques:** Donada la naturalesa cooperativa i competitiva del joc, el Saló permet organitzar minitornejos en directe, generant contingut visual ideal per als "Clips Virals" esmentats anteriorment.

7.4 Mètriques de seguiment i KPI

KPI	Objectiu	Freqüència de revisió	de
Wishlists	Creixement sostingut pre-release	Setmanal	
Conversió demo a wishlist	Millora iterativa de fitxa/trailer	Per esdeveniment	
Retenció de playtest	Detectar fricció d'onboarding	Per sessió	
Engagement social	Validar narrativa de marca	Setmanal	

Capitol 8: Conclusions

In my honest opinion this project achieved its core objective: delivering a technically solid and playable Unity game that combines an offline-first campaign foundation with a real online-ready architecture. The final implementation demonstrates that a host-authoritative model can support both local continuity and multiplayer scalability without splitting the gameplay core into divergent codebases.

From a planning perspective, the initial scope required constant prioritization. The development process benefited from incremental delivery (MVP first, online expansion second), but several iterations forced pragmatic decisions: stabilizing session lifecycle, simplifying operational complexity, and favoring robustness over premature feature breadth. This trade-off proved effective for a degree project where reliability and demonstrable progress are critical.

Methodologically, the project validated the use of agile iteration, frequent regression checks, and repeated black-box testing. The most relevant lesson was that usability findings from playtests had a direct architectural impact. Tutorial indicators, point-and-click movement, and mouse-based pickup/drop were not cosmetic additions; they changed onboarding quality and overall player comprehension, significantly improving first-session success.

Risk management was also a key differentiator. Networking, scene transitions, and authority boundaries were the highest-risk technical zones for me. Consolidating shutdown and session teardown paths reduced instability and prevented softlock-like failure modes. In parallel, keeping critical validation on the server side (payments, match flow, synchronized state) preserved game integrity under multiplayer conditions.

At a personal and professional level, the project exceeded the initial learning expectations in systems architecture, multiplayer integration, and production discipline. The final result is not only an academic deliverable, but also a realistic base for product evolution. The next challenge is no longer proving feasibility, but executing a controlled path toward public release quality through stronger content cadence, telemetry-informed balancing, and sustained community feedback loops.

Capítol 9: Glossari

Aquest glossari recull i defineix els principals termes tècnics utilitzats al llarg de la memòria del projecte.

9.1 Termes generals de desenvolupament

- **MVP (Minimum Viable Product):** producte mínim viable amb el conjunt essencial de funcionalitats per validar la proposta de joc.
- **Core loop:** bucle principal de joc que el jugador repeteix (moure's, interactuar, completar comandes, puntuar).
- **Pipeline:** cadena de processos i eines per produir contingut i generar versions jugables.
- **Backlog:** llista prioritzada de tasques i millores del projecte.
- **Sprint:** iteració temporal curta de desenvolupament amb objectius concrets.
- **Kanban:** metodologia visual de seguiment de tasques per estats (pendent, en curs, completat).
- **Build:** versió compilada/executable del projecte per a proves o distribució.
- **Build candidate:** versió candidata a ser publicada després de validar qualitat mínima.
- **Refactorització:** millora interna del codi sense canviar el comportament funcional extern.
- **Escalabilitat:** capacitat del sistema de créixer en funcionalitats o càrrega sense degradació crítica.
- **Robustesa:** capacitat d'operar correctament davant d'errors o condicions no ideals.

9.2 Motor, render i entrada

- **Unity:** motor de desenvolupament utilitzat per implementar el videojoc.
- **URP (Universal Render Pipeline):** pipeline de renderitzat de Unity orientat a rendiment i consistència visual.
- **Input System:** sistema modern d'entrada de Unity per gestionar teclat, ratolí i altres dispositius.
- **Escena (Scene):** unitat de contingut de Unity que agrupa objectes, UI i lògica d'un estat del joc.
- **GameObject:** entitat base de Unity sobre la qual s'afegeixen components.

- **Prefab:** plantilla reutilitzable d'objectes configurats.
- **ScriptableObject:** asset de dades serialitzables de Unity, útil per separar dades de lògica.
- **DontDestroyOnLoad:** mecanisme de Unity per mantenir objectes vius en canvis d'escena.

9.3 Arquitectura i patrons de codi

- **Arquitectura modular:** divisió del sistema en mòduls independents i cohesionats.
- **Hybrid-Core:** enfocament on el mateix nucli de gameplay serveix tant per offline com per online.
- **Singleton:** patró d'accés global a una única instància d'un component.
- **Observer / Event-driven:** patró basat en esdeveniments on els subscrits reaccionen a canvis.
- **MVP (Model-View-Presenter):** separació entre presentació, vista i lògica d'interacció en UI.
- **Adapter:** patró que adapta una interfície existent a una altra esperada pel domini.
- **State Pattern:** patró que encapsula comportaments segons l'estat actual d'un objecte.
- **FSM (Finite State Machine):** màquina d'estats finits amb transicions controlades.
- **Coordinator / Facade:** component que orquestra fluxos complexos i simplifica punts d'entrada.
- **Guard clause:** comprovació primerenca que talla l'execució davant condicions invàlides.
- **Fail-closed:** principi de seguretat/consistència que denega l'operació si falta context vàlid.

9.4 Xarxa i serveis online

- **UGS (Unity Gaming Services):** conjunt de serveis cloud de Unity (Auth, Lobby, Relay, etc.).
- **Authentication:** servei d'identificació de jugador.
- **Lobby:** servei de creació, descoberta i unió de sales multijugador.
- **Relay:** servei de connectivitat host-client sense infraestructura pròpia de servidors dedicats.
- **QuickMatch:** flux automàtic per trobar i unir-se ràpidament a una sessió compatible.

- **NGO (Netcode for GameObjects):** llibreria de xarxa de Unity per replicació d'estat i RPC.
- **UTP (Unity Transport):** capa de transport de paquets utilitzada per NGO.
- **Host-authoritative:** model on l'autoritat de regles crítiques recau en host/servidor.
- **Client:** instància de jugador que es connecta al host per rebre estat i enviar intents.
- **ServerRPC:** crida remota de client cap a servidor/host per sol·licitar una acció.
- **ClientRPC:** crida remota de servidor/host cap a clients per notificar canvis.
- **NetworkVariable:** dada replicada automàticament per NGO amb permisos de lectura/escriptura.
- **NetworkList:** col·lecció replicada per NGO per sincronitzar llistes entre participants.
- **Connection approval:** validació al servidor d'una nova connexió abans d'acceptar-la.
- **Heartbeat:** senyal periòdic per mantenir viva una sala de Lobby.

9.5 Persistència i dades

- **Persistència local:** emmagatzematge de progrés/configuració al dispositiu del jugador.
- **PlayerPrefs:** sistema de Unity per guardar valors simples de configuració local.
- **JSON:** format de serialització textual per emmagatzemar dades estructurades.
- **Hash (SHA-256):** resum criptogràfic utilitzat per detectar manipulacions de fitxers de dades.
- **Serialització compacta:** representació de dades reduïda per minimitzar cost de xarxa.
- **Context de runtime:** estat temporal de la sessió actiu només durant l'execució.
- **Teardown de sessió:** procés ordenat de tancament i neteja de recursos/estat.

9.6 Gameplay i sistemes del joc

- **Comanda (Order):** petició d'un client/NPC dins del cicle de servei de la partida.
- **TPV:** terminal de punt de venda per validar pagaments dins la lògica de joc.
- **Combo:** multiplicador temporal de puntuació vinculat a accions consecutives correctes.
- **Score / puntuació:** mètrica principal de rendiment del jugador o equip.

- **Workstation (estació):** punt interactiu del mapa amb fases de treball definides.
- **Interactable:** objecte del món que admet accions del jugador.
- **HUD:** elements d'interfície superposats durant la partida (temps, score, feedback).
- **Spawn:** creació d'un objecte o entitat dins l'escena de joc.

9.7 Qualitat, proves i validació

- **Pla de proves:** conjunt estructurat de casos per validar requisits i objectius.
- **Smoke test:** prova ràpida de salut general després d'un canvi o nova build.
- **Black-box test:** prova funcional sense inspeccionar la implementació interna.
- **Regression test:** prova per verificar que canvis recents no trenquen funcionalitats existents.
- **Playtest:** sessió de prova amb jugadors per detectar fricció d'UX i oportunitats de millora.
- **Traçabilitat:** vinculació explícita entre requisits, implementació, proves i resultats.

9.8 Requisits i objectius de la memòria

- **RF (Requisit funcional):** funcionalitat que el sistema ha d'oferir.
- **RNF (Requisit no funcional):** qualitat o restricció de rendiment, mantenibilitat o arquitectura.
- **ROp (Requisit operacional):** condicions d'operació, desplegament i execució.
- **RQ (Requisit de qualitat):** criteris de fiabilitat, consistència i experiència d'ús.
- **OT (Objectiu tècnic):** fita tècnica definida per validar l'assoliment del projecte.

Capítol 10: Bibliografia

10.1 Bibliografia general

- [1] Ghost Town Games. (2018). *Overcooked! 2* [Videojoc]. Team17.
- [2] It's Happening. (2022). *PlateUp!* [Videojoc]. Yogscast Games.
- [3] SMG Studio & DevM Games. (2020). *Moving Out* [Videojoc]. Team17.
- [4] Unity Technologies. (2023). *Universal Render Pipeline (URP) Documentation*. Recuperat de <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest>
- [5] Unity Technologies. (2023). *Netcode for GameObjects (NGO) Documentation*. Recuperat de <https://docs-multiplayer.unity3d.com/netcode/current/about/>
- [6] Unity Technologies. (2024). *Unity Relay: Join codes and allocation*. Recuperat de <https://docs.unity.com/relay/join-codes>
- [7] Unity Technologies. (2024). *Unity Lobby: Create and join lobbies*. Recuperat de <https://docs.unity.com/lobby>
- [8] Unity Technologies. (s.f.). *Boss Room: Small Scale Coop Sample* [Codi font]. GitHub. <https://github.com/Unity-Technologies/com.unity.multiplayer.samples.coop>
- [9] Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning. <http://gameprogrammingpatterns.com/>
- [10] Gamma, E., Helm, R., Johnson, R., i Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [11] Hipple, R. [Unity Technologies]. (2017, 6 d'octubre). Game Architecture with Scriptable Objects [Vídeo]. YouTube. https://www.youtube.com/watch?v=raQ3iHhE_Kk
- [12] Oetunian, D. [Unity]. (2017, 27 de juliol). Multi-scene editing in Unity for FAR: Lone Sails | Unite Europe 2017 [Vídeo]. YouTube. <https://www.youtube.com/watch?v=KRmqy22z0SM>
- [13] GDC. (2017, 31 de març). Low Complexity, High Fidelity: The Rendering of INSIDE [Vídeo]. YouTube. [Low Complexity, High Fidelity: The Rendering of INSIDE](https://www.youtube.com/watch?v=raQ3iHhE_Kk)
- [14] Adobe Systems & Rutzky, N. (2015). *Office Code Pro* [Font software]. SIL Open Font License 1.1. <http://scripts.sil.org/OFL>
- [15] Minty.kit. (2023, setembre 19). *Cat Cafe Pack v2.2 Update* [Paquet d'actius 2D/3D]. itch.io. <https://mintygamekit.itch.io/>
- [16] Minty.kit. (2025, juny 10). *Cozy Character Pack* [Paquet d'actius]. itch.io. <https://mintygamekit.itch.io/>

- [17] Unity Technologies. (s. d.). *Asset Store Terms / EULA* [Termes de llicència]. Unity Asset Store. <https://assetstore.unity.com/>
- [18] Unity Technologies. (s. d.). *Unity Personal* [Llicència de motor]. Unity. <https://unity.com/>
- [19] Blender Foundation. (s. d.). *Blender* [Programari]. <https://www.blender.org/>
- [20] Audacity Team. (s. d.). *Audacity* [Programari]. <https://www.audacityteam.org/>
- [21] Ximian, Inc. et al. (2001-2003). *CSharpCompiler license text* [Llicència de programari]. (Fitxer local de plugin al repositori)

10.2 Bibliografia de videojocs i referents del gènere

- [1] Ghost Town Games. (2018). *Overcooked! 2* [Videojoc]. Team17.
- [2] It's Happening. (2022). *PlateUp!* [Videojoc]. Yogscast Games.
- [3] SMG Studio & DevM Games. (2020). *Moving Out* [Videojoc]. Team17.

10.3 Bibliografia sobre documentació tècnica i eines

- [4] Unity Technologies. (2023). *Universal Render Pipeline (URP) Documentation*. Recuperat de <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest>
- [5] Unity Technologies. (2023). *Netcode for GameObjects (NGO) Documentation*. Recuperat de <https://docs-multiplayer.unity3d.com/netcode/current/about/>
- [6] Unity Technologies. (2024). *Unity Relay: Join codes and allocation*. Recuperat de <https://docs.unity.com/relay/join-codes>
- [7] Unity Technologies. (2024). *Unity Lobby: Create and join lobbies*. Recuperat de <https://docs.unity.com/lobby>
- [8] Unity Technologies. (s.f.). *Boss Room: Small Scale Coop Sample* [Codi font]. GitHub. <https://github.com/Unity-Technologies/com.unity.multiplayer.samples.coop>

10.4 Bibliografia sobre literatura i arquitectura de programari

- [9] Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning. <http://gameprogrammingpatterns.com/>
- [10] Gamma, E., Helm, R., Johnson, R., i Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [11] Hipple, R. [Unity Technologies]. (2017, 6 d'octubre). Game Architecture with Scriptable Objects [Vídeo]. YouTube. https://www.youtube.com/watch?v=raQ3iHhE_Kk

10.4 Altres referències on he après molt sobre el funcionament dels equips i com personalitzar el pipeline de Unity.

[12] Oetunian, D. [Unity]. (2017, 27 de juliol). Multi-scene editing in Unity for FAR: Lone Sails | Unite Europe 2017 [Vídeo]. YouTube.

<https://www.youtube.com/watch?v=KRmqy22z0SM>

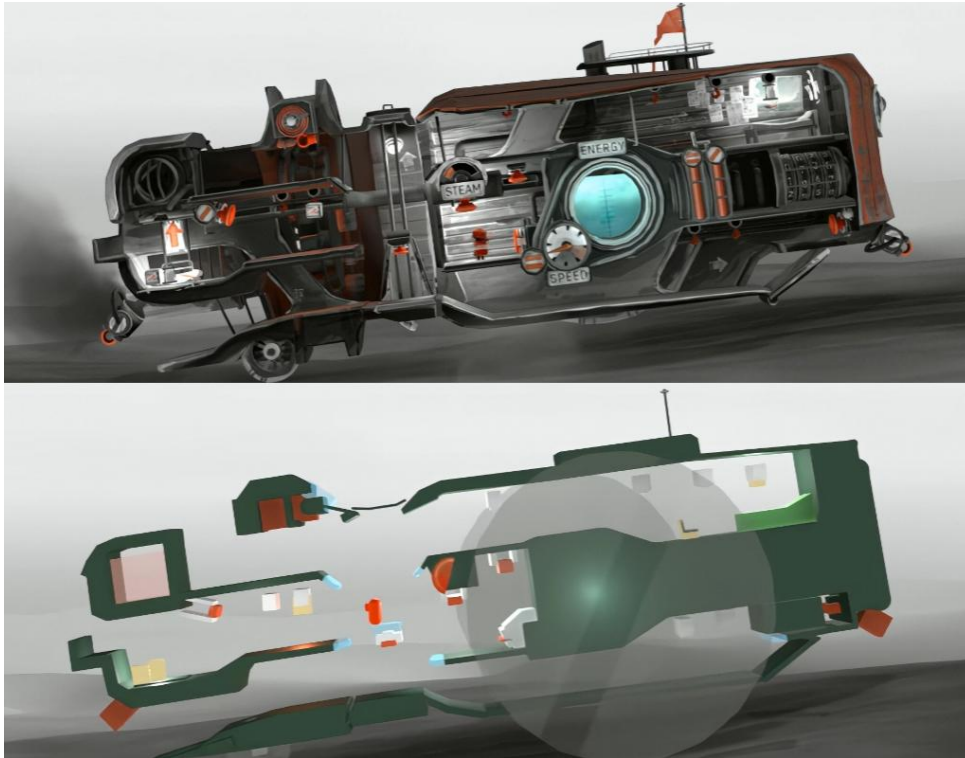


Figura 42. Anàlisi comparatiu d'enginyeria inversa: arquitectura multi-escena original de FAR: Lone Sails (superior) i prototip propi d'imitació estructural desenvolupat a Unity (inferior).

Font: Elaboració pròpia a partir d'Oetunian (2017)



Figura 43. Integració simultània de shaders d'herba dinàmica, sistema de pluja i efecte de postprocessat.

Font: Elaboració pròpia. Entorn de Unity 6.

[13] GDC. (2017, 31 de març). Low Complexity, High Fidelity: The Rendering of INSIDE [Vídeo]. YouTube. [Low Complexity, High Fidelity: The Rendering of INSIDE](#)



Figura 44. Implementació d'efectes atmosfèrics i corba de renderitzat per a la simulació de boira a INSIDE.

Font: Playdead, extret de GDC (2017).



Figura 45. Anàlisi de referents visuals: integració d'efectes atmosfèrics i composició d'il·luminació a INSIDE.

Font: Elaboració pròpia a partir de GDC (2017).

Capítol 11: Llicències del projecte

Aquest document recull les llicències dels recursos utilitzats al projecte amb criteri de traçabilitat acadèmica. S'ha aplicat un doble criteri:

- **Evidència tècnica del repositori** (fitxers de llicència, rutes de paquets, metadades).
- **Declaració d'ús del desenvolupador**: els actius de pagament utilitzats provenen de Unity Asset Store i han estat adquirits; els actius externs gratuïts provinents d'itch.io utilitzats al projecte són sota CC0.

11.1 Inventari de recursos de pagament (Unity Asset Store)

Tots els actius de pagament utilitzats en producció provenen de la Unity Asset Store i disposen de compra vàlida.

1. **Low Poly Animated Cats** (polyperfect)

- Evidència de ruta: Assets/04_ASSEET STORE/polyperfect/Low Poly Animated Cats/...

2. **GUI Pro - Simple Casual** (Layer Lab)

- Evidència de ruta: Assets/04_ASSEET STORE/Layer Lab/GUI Pro-SimpleCasual/...

3. **OilPaint** (FronkonGames)

- Evidència de ruta: Assets/04_ASSEET STORE/FronkonGames/Artistic/OilPaint/...

4. **GameInputControllerIconsFree** (Paquet gratuït - Regit per termes Asset Store)

- Evidència de ruta: Assets/04_ASSEET STORE/GameInputControllerIconsFree/...

Exemple d'evidència de compra:

My Assets

Sort by Purchased Date View Results 25

1 - 25 of 41 items

Release Notes Manage my labels

Search for my assets

	JUSTCREATE Low Poly Food Lite 1.7 MB Purchase date: May 4, 2026 Organization: tbcipher(Personal)	Last updated: Jun 22, 2023 • Version: 1.0 First release Add label Hide asset	Open in Unity
	URSAANIMATIONS Animated Goat and Shee... 1.9 MB Purchase date: Apr 30, 2026 Organization: tbcipher(Personal)	Last updated: Jun 4, 2024 • Version: 1.0.1 First release Add label Hide asset	Open in Unity
	POLYKEBAP Stylized Coin Pack – 15 C... 804.5 KB Purchase date: Apr 30, 2026 Organization: tbcipher(Personal)	Last updated: Nov 26, 2025 • Version: 1.0 First release Add label Hide asset	Open in Unity
	AMANZ Game Input Controller Ic... 1.4 MB Purchase date: Apr 17, 2026 Organization: tbcipher(Personal)	Last updated: Feb 17, 2025 • Version: 1.1 First release more Add label Hide asset	Open in Unity
	POLYPERFECT Low Poly Animated Cats 60.0 MB Purchase date: Apr 17, 2026 Organization: tbcipher(Personal)	Last updated: Feb 2, 2026 • Version: 1.0 1.0 First release more Add label Hide asset	Open in Unity

Status

☐ Unlabeled

☐ Hidden Assets

☐ Deprecated

Labels(0)

Get started by selecting [Manage my labels](#) or [Add label](#) to start organizing your assets.

Categories

☐ 2D (6)

☐ 3D (16)

☐ Essentials (2)

☐ Templates (3)

☐ Tools (10)

☐ VFX (5)

Publishers

Figura 46. Inventari de llicències i recursos adquirits a la Unity Asset Store per a la integració d'assets al projecte.

Font: Elaboració pròpia.

11.2 Declaració de compliment

Per als paquets de pagament, el projecte declara:

- A.** Adquisició legal sota compte de desenvolupador,
- B.** Ús dins del límit d'ús permès per l'EULA de Unity Asset Store,
- C.** No redistribució dels fitxers font dels paquets com a producte independent.

11.3 Paquets CC0 utilitzats al projecte

1. Cat Cafe Pack v2.2 Update (Minty.kit)

- Fitxer de llicència local: Assets/01_ART/Prefabs/Minty/Free Personal and Commercial License.txt
- Text explícit: License: (Creative Commons Zero, CC0).

2. Cozy Character Pack (Minty.kit)

- Fitxer de llicència local: Assets/01_ART/Prefabs/Minty2/Free Personal and Commercial License.txt
- Text explícit: License: (Creative Commons Zero, CC0).

Condicions de compliment aplicades

- A. Ús permès en projectes personals, educatius i comercials,
- B. Atribució no obligatòria (CC0), però recomanable a crèdits com a bona pràctica,
- C. Manteniment de registre intern de procedència de fitxers.

Evidència de compra:

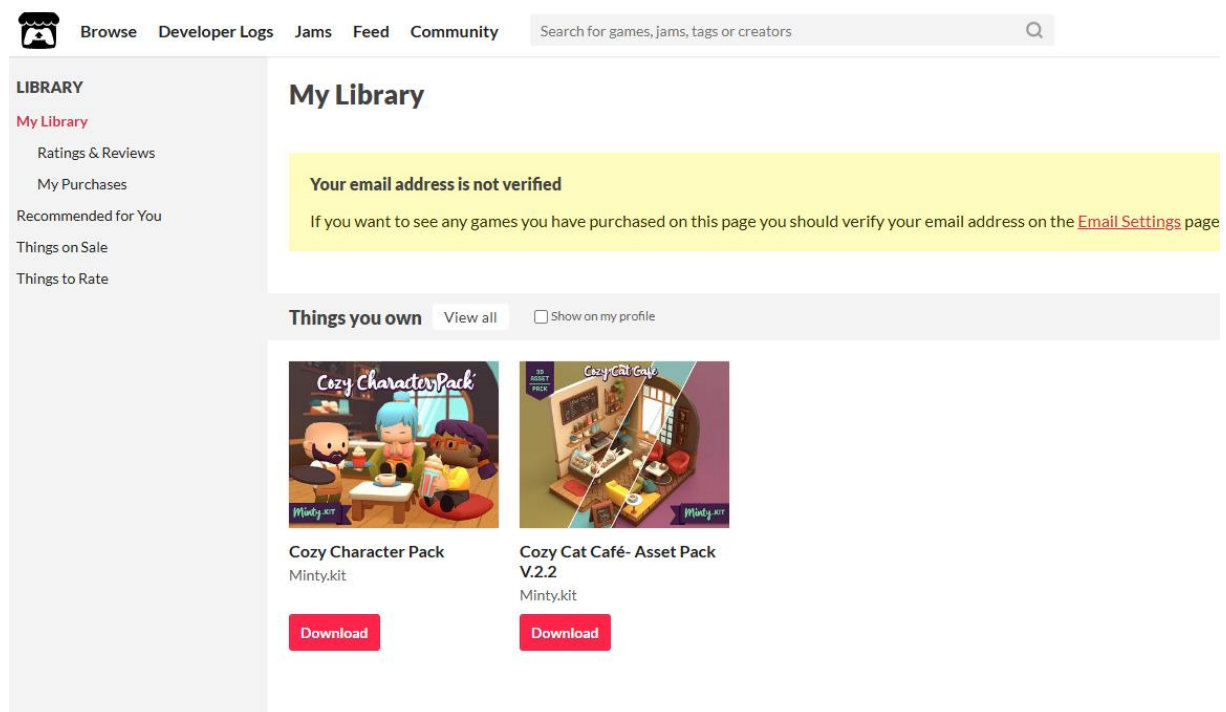


Figura 47. Inventari de llicències i recursos adquirits a Itchio per a la integració d'assets al projecte.

Font: Elaboració pròpia.

11.3 Programari, eines i llicències de desenvolupament

Motor i plataforma

- **Unity Personal:** ús sota termes oficials de Unity per desenvolupament indie/educatiu segons llindars vigents del pla.

Eines auxiliars

- **Blender:** llicència GPL (programari lliure).
- **Audacity:** llicència GPL (programari lliure).
- **Git/GitHub:** ús de servei i repositori sota termes de plataforma.

11.4 Política de propietat intel·lectual del projecte

El codi font original del projecte i la composició final del videojoc es consideren autoria del desenvolupador sense perjudici dels drets dels tercers sobre els assets i plugins incorporats sota les seves llicències respectives.

12 Annexos

Annex AT-01: Flux de sessió i teardown

Aquest annex descriu amb detall el cicle de vida de sessió del projecte, des de l'entrada al menú fins al retorn segur, incloent:

- Inici offline (host local),
- Inici online (host/client sobre Lobby+Relay),
- Canvis d'escena sincronitzats,
- Procediment de tancament (*teardown*) i neteja de context.

L'objectiu és garantir traçabilitat tècnica i justificar per què el sistema evita estats inconsistents entre partides.

Components principals implicats

- Assets/02_CODE/NETCODE/COORDINATOR/SessionCoordinator.cs
- Assets/02_CODE/Lobby/GameLobby.cs
- Assets/02_CODE/NETCODE/GameMultiplayerManager.cs
- Assets/02_CODE/Managers/LoaderManager.cs
- Assets/02_CODE/Managers/GameManager.cs
- Assets/02_CODE/NETCODE/NetTickBudget/NetTickBudget.cs
- Assets/02_CODE/Campaign/CampaignRuntimeContext.cs
- Assets/02_CODE/Campaign/COOP/CoopCampaignSessionContext.cs
- Assets/02_CODE/TUTORIAL + HINTS/TUTO V2/TutorialRuntimeContext.cs

Arquitectura funcional del flux de sessió

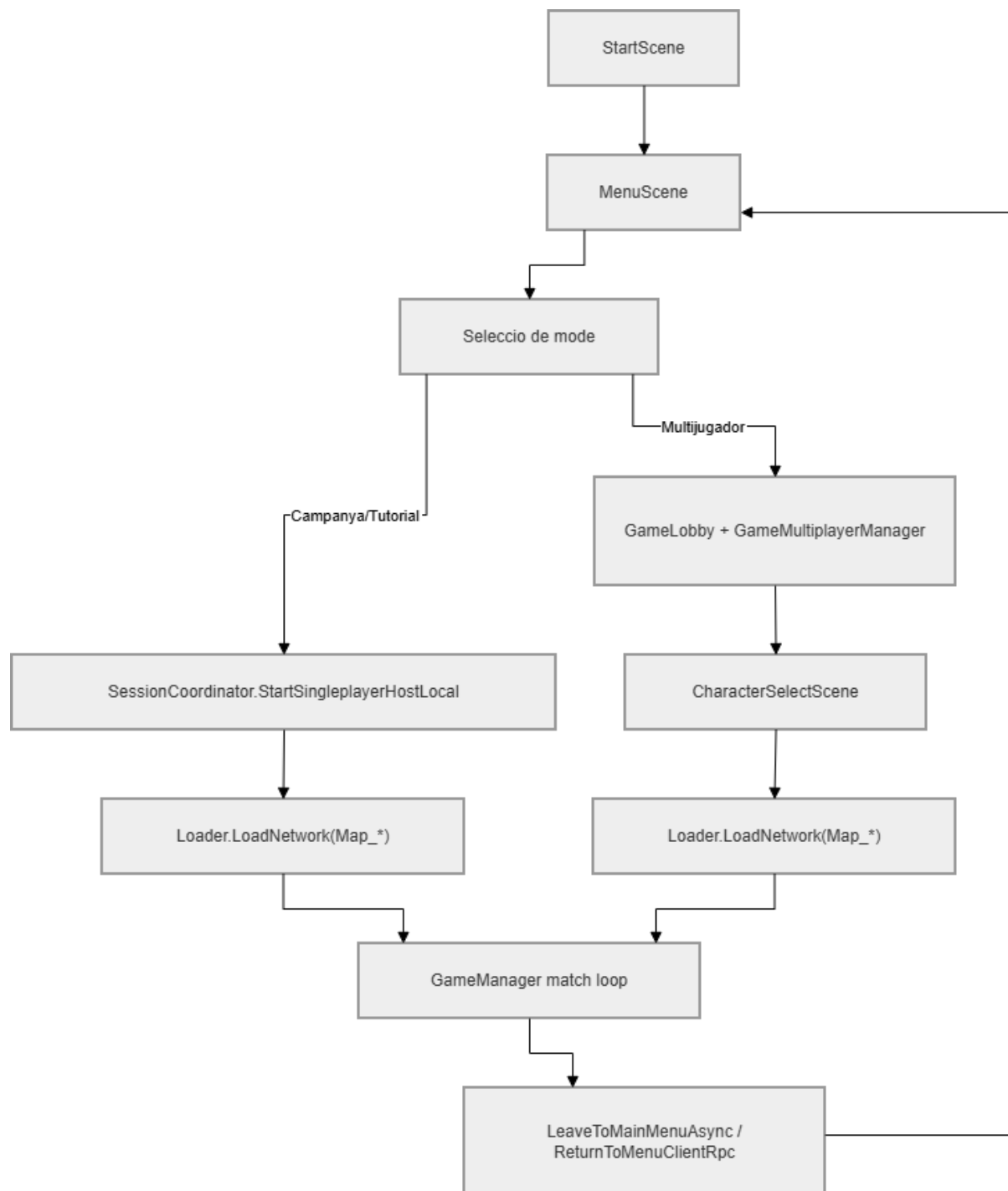


Figura 48. Arquitectura funcional del flux de sessió

Font: Elaboració pròpia mitjançant l'eina Draw.io

Flux de sessió offline (campanya / tutorial)

1. L'usuari selecciona nivell des del menú.
2. `SessionCoordinator.EnsureNetworkManager()` crea o reutilitza `NetworkManager`.
3. Es configura `UnityTransport` en local (`127.0.0.1:7777`).
4. `NetworkManager.StartHost()` inicia sessió local autoritària.
5. `Loader.LoadNetwork(targetScene)` carrega directament l'escena de joc.

El flux offline reutilitza la mateixa infraestructura de networking i es minimitza bifurcació de codi entre campanya i online. Es compleix requisit d'execució offline real quan no calen serveis externs.

Flux de sessió online (host/client)

Host:

1. `GameLobby.CreateLobbyInternal(...)` inicialitza serveis online.
2. Es crea lobby i s'assigna relay allocation.
3. Es publica join code i metadades de mode/mapa.
4. Es configura `SetRelayServerData(...)` al transport.
5. `GameMultiplayerManager.StartHost()` arrenca host.
6. Es carrega `CharacterSelectScene` i posteriorment mapa.

Client:

1. Join per codi, id o quickmatch.
2. Resolució de relay join allocation.
3. Configuració de transport client amb relay server data.
4. `GameMultiplayerManager.StartClient()`.
5. Sincronització d'escena via NGO des del host.

Captura de MenuScene en creació de lobby per part del host.

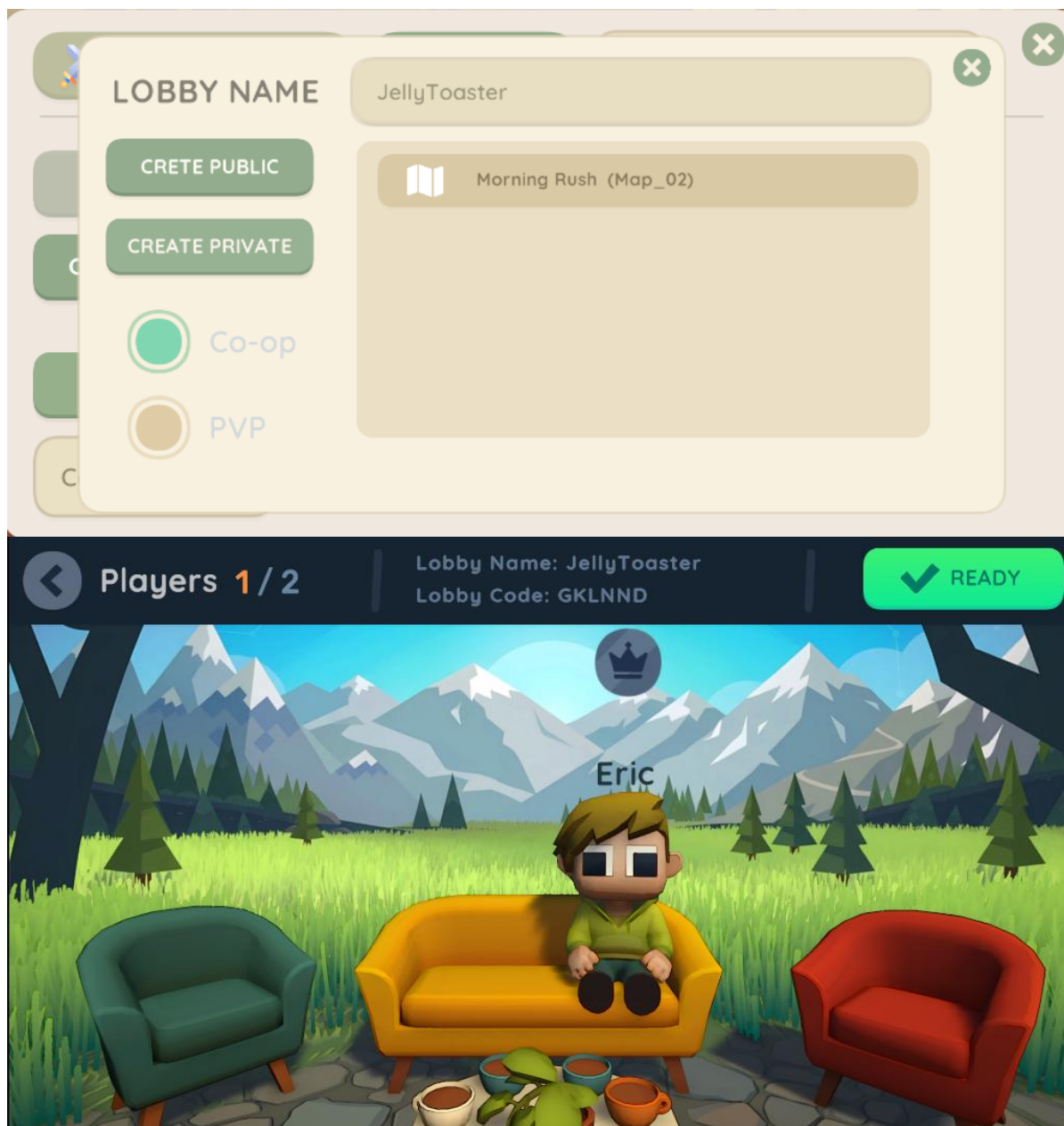


Figura 49. Interfície de joc durant el flux de creació d'un lobby.

Font: Unity 6 – Development build

Captura de client entrant a CharacterSelectScene.

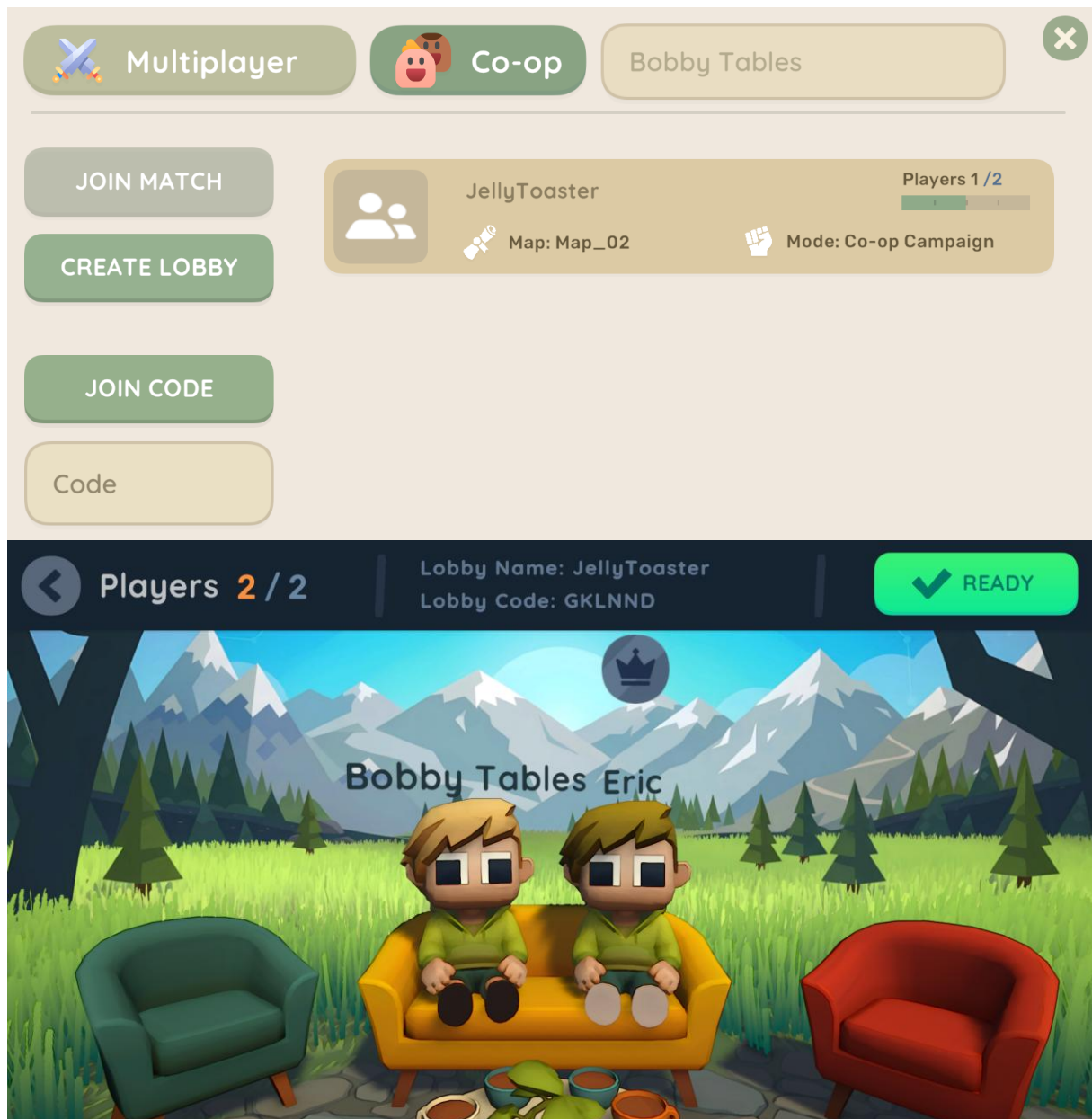


Figura 50. Interfície de joc durant el flux d'unió al lobby.

Font: Unity 6 – Development build

Captura de client i host en partida.



Figura 51 Visualització de sessió en temps real: perspectiva del client (esquerra) i de l'amfitrió o host (dreta).

Font: Elaboració pròpia.

Teardown unificat (sortida segura)

El teardown s'implementa de forma centralitzada a SessionCoordinator. LeaveToMainMenuAsync() per eliminar divergències de neteja entre fluxos.

Passos exactes de neteja:

1. Guarda de concurrència `_leaving` per evitar doble sortida simultània.
2. Marcat de sessió en tancament a `GameMultiplayerManager.MarkSessionShuttingDown()`.
3. Intent de `GameLobby.LeaveLobby()` amb tractament de fallides no crítiques.
4. `NetworkManager.Shutdown()` si hi ha escolta activa.
5. Doble `Task.Yield()` per estabilitzar transició de shutdown.
6. `NetTickBudget.ResetAll()`.

7. Neteja de contextos:

- CampaignRuntimeContext.Clear()
- CoopCampaignSessionContext.Clear()
- TutorialRuntimeContext.Clear()

8. GameManager.ResetSessionState().

9. Retorn de navegació amb Loader.Load(MenuScene).

Gestió d'errors i condicions límit

Errors habituals en transició

- NetworkManager no inicialitzat.
- Excepció en LeaveLobby per carrera de desconnexió.
- Doble invocació de sortida (botó + callback de xarxa).
- Persistència d'estat estàtic entre sessions.

Mitigacions implementades

- Guardes de nul·litat i estat d'escolta.
- Try/catch no crític en sortida de lobby.
- Flag_leaving per serialitzar teardown.
- Clear explícit de contextos i buffers de tick budget.

Robustesa i recuperació (timeouts, fail-closed, teardown unificat)

Per reduir el risc de softlocks i garantir sortida neta, el sistema incorpora:

- Timeouts explícits per fases crítiques.
- Guardes de concurrència i recuperació segura.
- Un procediment únic de sortida a menú (teardown), aplicable a offline i online.

Requisits: RQ-01, RQ-04, ROp-03.

Tipus de diagrama: Flux de recuperació d'errors

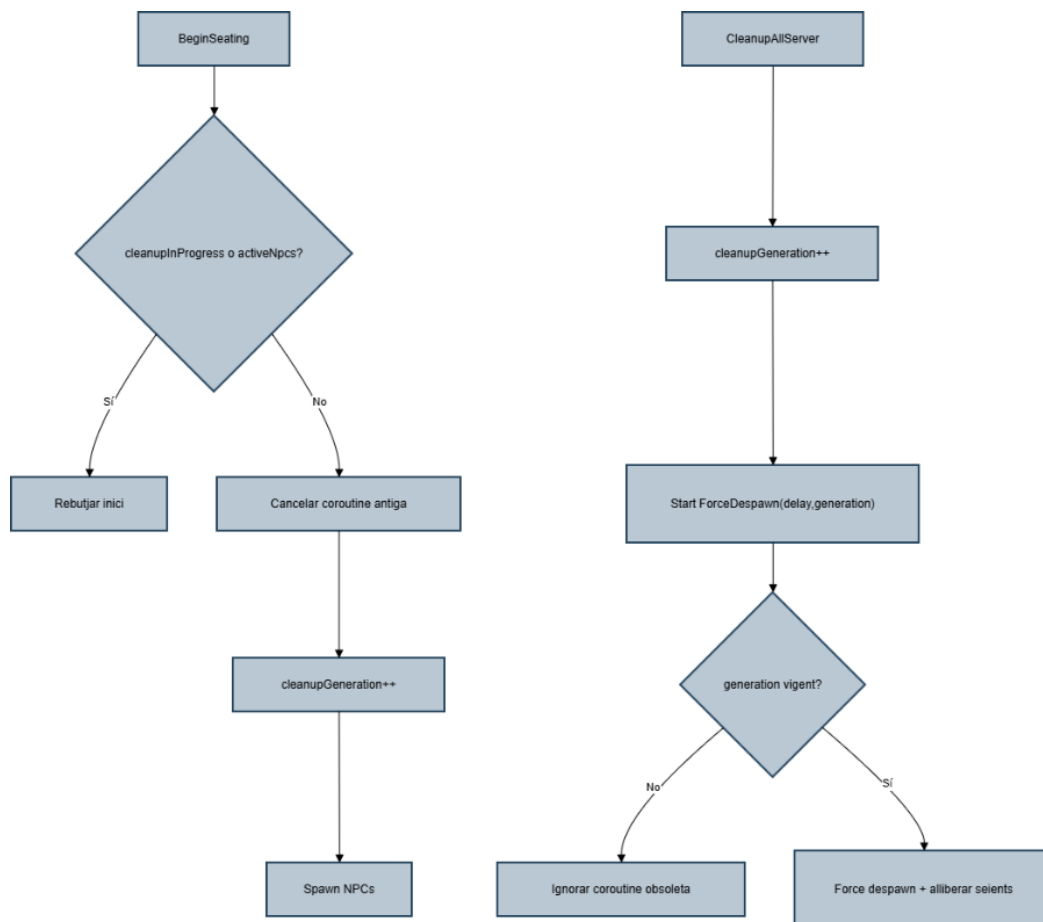


Figura 52. Diagrama de flux del protocol de recuperació d'errors i neteja asíncrona

Font: Elaboració pròpia mitjançant l'eina Draw.io.

Conclusió de l'annex

Finalment dir que disseny de sessió i teardown centralitzat és una de les decisions arquitectòniques més rellevants del projecte perquè redueix la complexitat operativa de modes, evita acumulació d'estat inconsistent, facilita manteniment i diagnòstic de fallides de transició i consolida l'estratègia Hybrid-Core sobre una base robusta. Inicialment va ser plantejat per solucionar totes les inconsistències de sortida entre els tres modes de joc, sortides desde menús auxiliars i/o desconexions.

Annex AT-02: Integració Lobby/Relay

En aquest annex documentarem en profunditat la integració de serveis online de Unity (Authentication, Lobby i Relay)

Fitxers clau

- Assets/02_CODE/Lobby/GameLobby.cs
- Assets/02_CODE/NETCODE/GameMultiplayerManager.cs
- Assets/02_CODE/NETCODE/COORDINATOR/SessionCoordinator.cs
- Assets/02_CODE/GUI + HUD/Lobby/LobbyUI.cs (interfície)
- Assets/02_CODE/GUI + HUD/Lobby/LobbyResponseMessageUI.cs (feedback d'errors)

Pila de serveis i responsabilitats

- **UGS Authentication:** identitat de jugador.
- **UGS Lobby:** creació, consulta, unió i heartbeat de sala.
- **UGS Relay:** connectivitat host-client amb join code.
- **NGO + UTP:** sessió de transport i sincronització de partida.

Seqüència

1. EnsureOnlineServicesReady() comprova serveis i autenticació.
2. CreateLobbyInternal(...) crea lobby amb metadades (gameMode, selectedMap).
3. AllocateRelay() crea allocation de relay.
4. GetRelayJoinCode() obté codi d'unió.
5. UpdateLobbyAsync() publica join code per membres.
6. UnityTransport.SetRelayServerData(...).
7. GameMultiplayerManager.StartHost().
8. Loader.LoadNetwork(CharacterSelectScene).

Flux detallat de creació de lobby (host)

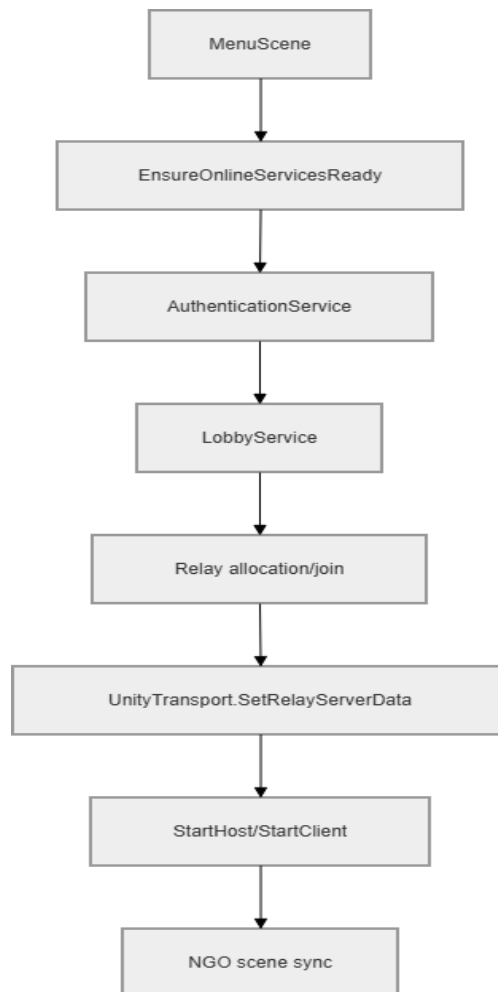


Figura 53. Flux detallat de creació de lobby (host)

Font: Elaboració pròpia mitjançant l'eina Draw.io.

Dades de lobby rellevants

- RelayJoinCode
- SelectedMap
- GameMode (multiplayer o coop_campaign)

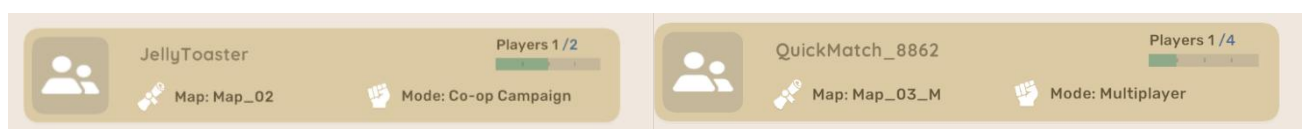


Figura 54. Dades de lobby rellevants – Join modes.

Font: Elaboració pròpia - Scene view Unity 6

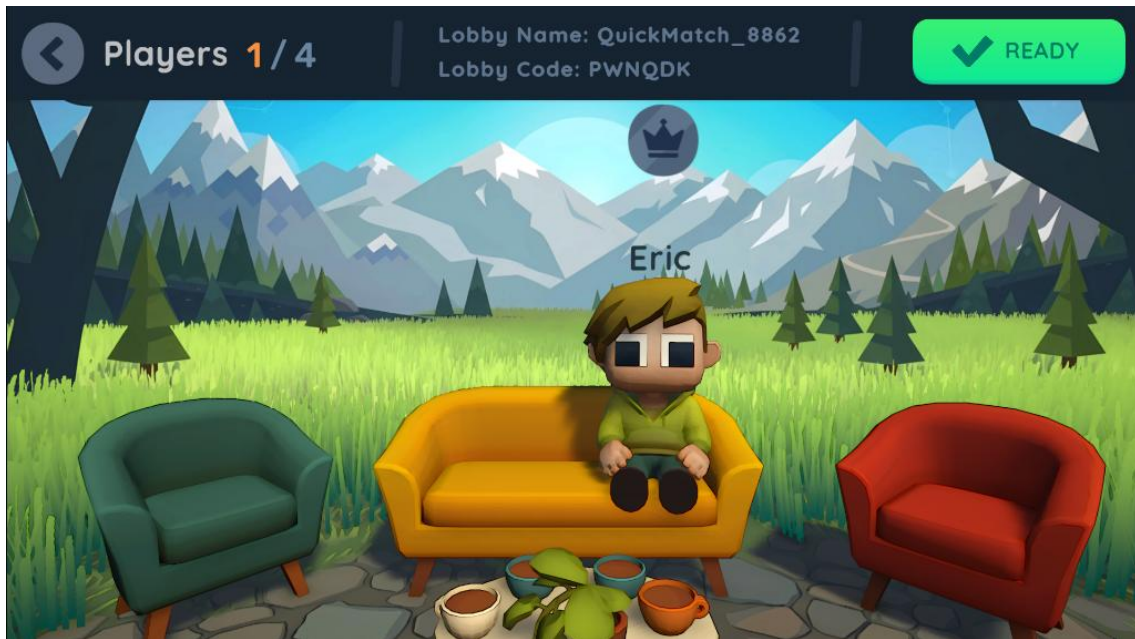


Figura 55. Interfície de joc durant el flux d'unió al lobby.

Font: Elaboració pròpia - Scene view Unity 6

Flux detallat de unió (client)

Join per codi/ID

1. Sol·licitud de unió a lobby.
2. Lectura de relayJoinCode del lobby.
3. JoinRelay(relayJoinCode).
4. Configuració transport client.
5. StartClient().
6. Sincronització d'escena des de host.

QuickMatch

- S'aplica guarda de concurrència (_quickMatchInFlight).
- Es prioritza unió a candidatures de lobby disponibles.
- Si no n'hi ha, es crea lobby nou automàticament.
- Fallides intermèdies fan SafeLeaveQuickMatchLobbyAsync (best effort).

Connection approval i seguretat funcional

GameMultiplayerManager.NetworkManager_ConnectionApprovalCallback(...) aplica la restricció de unió només a CharacterSelectScene, ara mateix tenim hardcoded el límit de jugadors (MAX_PLAYER_AMOUNT). Fa recollida de customization payload. He desactivat CreatePlayerObject automàtic. Aquest model redueix unió tardana en fase de joc i evita inconsistències de spawn.

Conclusió de l'annex

La integració Lobby/Relay està dissenyada amb un enfocament resilient: inicialització lazy per respectar offline, guardes de concurrència en quickmatch, maneig d'errors orientat a degradació controlada i una separació clara entre responsabilitats de serveis i arrencada de sessió de joc.

Annex AT-03: Estat de partida i replicació

Aquest annex descriu el model de replicació de l'estat de partida, incloent:

- A. Màquina d'estats global de match,
- B. Temporitzadors replicats,
- C. Sincronització de taules i fases,
- D. Puntuació i resultats per client/equip,
- E. Impacte de l'autoritat de servidor en consistència.

Fitxers de referència

- Assets/02_CODE/Managers/GameManager.cs
- Assets/02_CODE/NETCODE/Table/TableNetSync.cs
- Assets/02_CODE/SCORE/ScoreManager.cs
- Assets/02_CODE/NETCODE/GameMultiplayerManager.cs

Model d'autoritat

El sistema opera sota esquema host-authoritative: el servidor decideix estat canònic de match, els clients envien intents d'acció, els canvis crítics es validen i es repliquen des del servidor.

Màquina d'estats de partida

A GameManager.State es defineixen:

- MainMenu
- Waiting
- Countdown
- GameOver
- Returning

Diagrama d'estats de partida

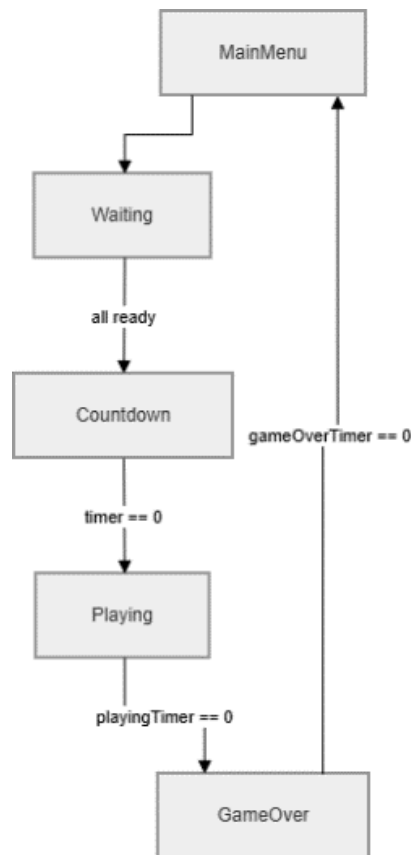


Figura 56. Diagrama d'estats de partida.

Font: Elaboració pròpia mitjançant l'eina Draw.io.

Temporitzadors i estratègia de replicació.

GameManager utilitza `NetworkVariable` per: estat global (`NetworkVariable<State>`), countdown i temps de partida. A més, inclou quantització configurable de temps (`countdownTimerReplicationStep`, `playingTimerReplicationStep`) per reduir soroll de xarxa. Els beneficis són que tenim menys *churn* de replicació i menys càrrega en actualitzacions de valor, afavorint una percepció més estable en els clients. Això comporta assumir una lleugera pèrdua de precisió absoluta en el sincronisme de la variable d'alta freqüència, un *trade-off* acceptable atès que no penalitza l'experiència jugable del bucle principal. Es nota sobretot als clients i és millorable.

Replicació d'estat de taules.

TableNetSync replica: fase de taula (phaseId), inici i durada de fase (phaseStartServerTime, phaseDuration), elements requerits restants (NetworkList<int>), esdeveniments de feedback de pagament dirigits.

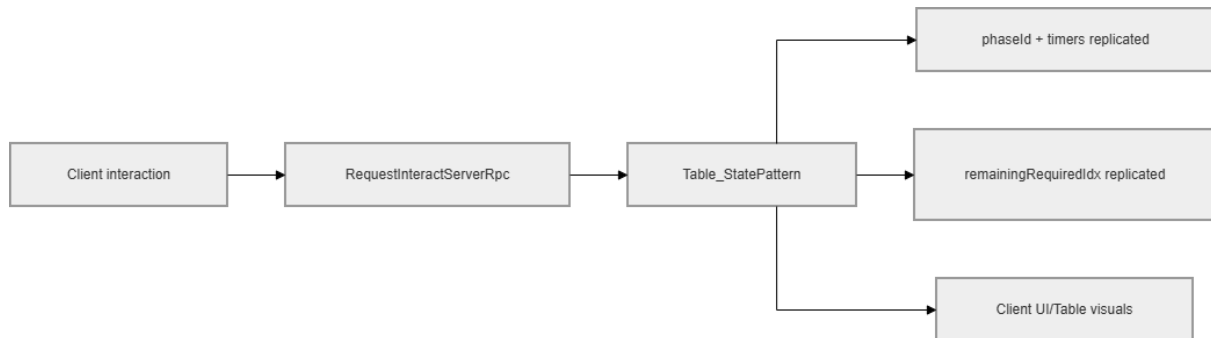


Figura 57. Diagrama de replicació d'estat de taules.

Font: Elaboració pròpia mitjançant l'eina Draw.io.



Figura 58. Interfície de joc durant replicació d'estat de taules.

Font: Elaboració pròpia - Scene view Unity 6

Replicació de puntuació i combos.

ScoreManager manté:

NetworkVariable<int> score global,

- NetworkVariable<float> comboMultiplier legacy global,
- NetworkVariable<int> clientsLost,
- NetworkList<PlayerScoreEntry> per jugador (clientId, score, combo).

Això permet: score per equip i per jugador, càlcul de resultats i top player i persistència de coherència entre host i clients.

Publicació de resultats de match.

Al final de partida:

1. `GameManager.PublishMatchResultsServer()` calcula ranking i puntuacions d'equip.
2. `MatchResultNetSync.ServerPublishResults(...)` publica resum compartit.
3. `ApplyLocalSummaryClientRpc(...)` envia resum individualitzat per client.

Aquest patró evita exposar càlcul final a clients i garanteix resultat consistent.



Figura 59. Interfície de joc durant la publicació de resultats de match

Font: Elaboració pròpia - Scene view Unity 6

Spawn i sincronització de jugadors.

Durant `SceneManager.OnLoadEventCompleted(...)` el servidor verifica clients connectats, instancia `playerPrefab`, executa `SpawnAsPlayerObject(clientId, true)` i aplica customization i assegura fila de score. Aquesta etapa és crítica per evitar clients sense representació de jugador (Player NPC).

Conclusió de l'annex.

La replicació de partida està estructurada per capes:

- Estat global (`GameManager`),
- Estat de domini d'interacció (`TableNetSync`),
- Estat econòmic/score (`ScoreManager` + `MatchResult`).

Aquest model dona una base sòlida per consistència multijugador i facilita l'auditoria d'errors de sincronització.

Integritat d'ítems (invariant atòmic)

Regla central: cada ítem només pot existir en un únic “contenedor” lògic a la vegada (món, jugador/safata o taula). Això evita duplicacions i inconsistències, especialment rellevant quan hi ha sincronització online.

Requisits: RQ-02, RF-03, RF-02.

Tipus de diagrama: Estat/invariant d'ítems

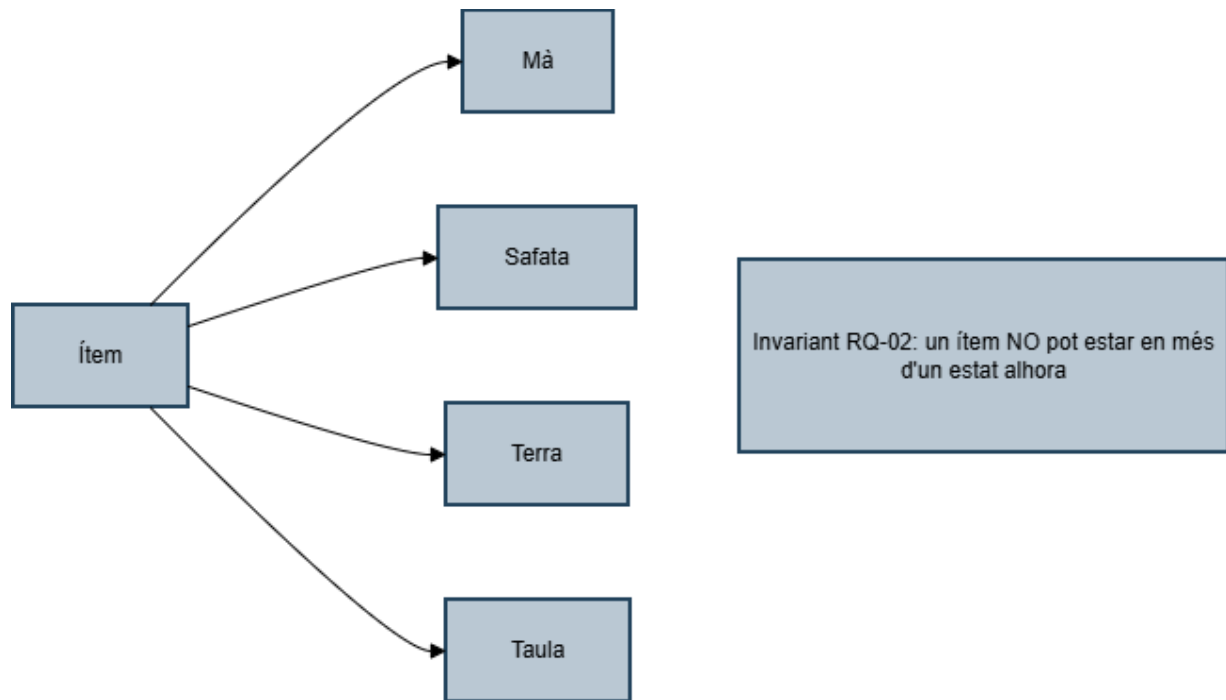


Figura 60. Diagrama d'integritat d'ítems (invariant atòmic)

Font: Elaboració pròpia mitjançant l'eina Draw.io.

Annex AT-04: Persistència i contextos

En aquest Annex descriurem el sistema de persistència local i la governança dels contextos de runtime que coordinen mode campanya, coop i tutorial.

Fitxers de referència

- Assets/02_CODE/Campaign/CampaignSaveSystem.cs
- Assets/02_CODE/Campaign/CampaignManager.cs
- Assets/02_CODE/Economy/EconomySaveSystem.cs
- Assets/02_CODE/Economy/EconomyManager.cs
- Assets/02_CODE/Campaign/CampaignRuntimeContext.cs
- Assets/02_CODE/Campaign/COOP/CoopCampaignSessionContext.cs
- Assets/02_CODE/TUTORIAL + HINTS/TUTO V2/TutorialRuntimeContext.cs
- Assets/02_CODE/NETCODE/COORDINATOR/SessionCoordinator.cs

Arquitectura de dades persistents

El sistema separa dues persistències principals:

- **campanya/progressió,**
- **economia/recompenses.**

Persistència de campanya

Fitxer i schema

CampaignSaveSystem:

- carrega fitxer si existeix,
- crea nou save si cal,
- garanteix schema via EnsureSchema(...),
- desbloqueja primer nivell per defecte.

Noms de fitxer:

- editor: campaign_save_EDITOR.json
- build: campaign_save_BUILD.json

Integració amb gestió de progrés

CampaignManager manté save en memòria, resol definicions de nivell, registra resultats (RecordLevelResult) i aplica desbloqueig lineal per nextLevelId.



Figura 61 Interfície de joc – Campaign menu.

Font: Elaboració pròpia - Scene view Unity 6

Persistència d'economia

EconomySaveSystem: desa en JSON, manté hash separat (.hash) amb SHA256 + secret local i valida integritat en càrrega.

Si hi ha mismatch:

- warning de manipulació,
- regeneració de save net.

Noms de fitxer:

- editor: economy_save_EDITOR.json + .hash
- build: economy_save_BUILD.json + .hash
-

Integració de recompenses: EconomyManager.GrantMatchRewards(...) calcula monedes per estrelles i mode, aplica diamants per fites (star vault, campanya, win diari), desa i notifica canvis de wallet.

CampaignRuntimeContext: Camps de IsCampaignRun + SelectedLevelId. Indiquen que la sessió actual prové de campanya.

CoopCampaignSessionContext: Camps IsCoopCampaignRun, SelectedLevelId, LocalWasLevelUnlockedAtMatchStart. Marquen execució coop de campanya i estat d'entrada.

Neteja de contextos en teardown

SessionCoordinator.LeaveToMainMenuAsync() executa neteja explícita:

- A. CampaignRuntimeContext.Clear()
- B. CoopCampaignSessionContext.Clear()
- C. TutorialRuntimeContext.Clear()

Aquesta neteja evita arrastre de mode entre partides, selecció de nivell residual i comportament tutorial no intencionat en sessions noves.

Conclusió de l'annex

Considero que la separació entre persistència (campanya/economia) i contextos de runtime és essencial per a l'estabilitat del projecte ja que les dades permanents es mantenen consistents i recuperables, l'estat temporal de sessió es neteja de forma determinista i el sistema evita transicions amb comportaments residuals entre modes.

Annex AT-05: Proposta d'Arquitectura Basada en Esdeveniments

L'objectiu d'aquest disseny és establir una base sòlida per al desenvolupament del videojoc en Unity (URP), assegurant que el projecte sigui:

- **Modular:** Sistemes independents que es poden modificar sense afectar la resta.
- **Reutilitzable:** Facilitar l'ús de components en diferents escenes o minijocs.
- **Mantenible:** Codi net que permet la depuració ràpida.
- **Scalable (Netcode Ready):** Preparar l'arquitectura per a una futura transició completa a multijugador, evitant dependències globals que causarien conflictes en entorns servidor-client.
- **Lleuger:** S'eviten frameworks d'injecció de dependències (DI) externs per no afegir complexitat innecessària (*overkill*) al projecte.

L'Estratègia de desacoblament

L'acoblament defineix el grau de dependència entre dos scripts. En aquest projecte s'ha optat per un acoblament baix per evitar el "codi espagueti", tot i que s'ha gestionat amb cura per no perdre la traçabilitat del flux lògic.

A. L'ús de Singletons (Accés Global)

- **Aplicació:** Reservat exclusivament per a serveis transversals i únics (ex. AudioManager o NetworkManager).
- **Risc detectat:** S'ha evitat l'ús de Player.Instance, ja que en un entorn multijugador hi ha múltiples instàncies de jugador i un Singleton bloquejaria l'escalabilitat.

B. Referències Serialitzades (Cablejat per Inspector)

- **Aplicació:** Ús de [SerializeField] per connectar components directament des de l'editor d'Unity.
- **Avantatge:** Proporciona una visualització clara de les dependències ("qui parla amb qui") directament des de la jerarquia, facilitant la composició d'objectes sense necessitat de codi de cerca tipus GameObject.Find.

El Patró Observer: Comunicació mitjançant Esdeveniments

S'ha triat el patró **Observer** per a notificacions d'estat (ex: OnHealthChanged, OnOrderSpawned).

- **Problema de traçabilitat:** Els esdeveniments de C# estàndard ocorren en temps d'execució i no són visibles a l'Inspector d'Unity, cosa que dificulta saber quants scripts estan "escoltant".
- **Solució adoptada:**
 - **Cicls de vida estrictes:** Totes les subscripcions es realitzen a OnEnable i es cancel·len a OnDisable per evitar fuites de memòria (*memory leaks*).
 - **Canals d'Esdeveniments (ScriptableObjects):** Ús d'actius que actuen com a "emissores" visibles des de l'editor, permetent veure les interaccions entre sistemes sense obrir el codi.

Arquitectura MVP (Model-View-Presenter)

Per solucionar la "invisibilitat" dels esdeveniments, especialment en la interfície d'usuari (UI), s'aplica el patró **MVP**:

1. **Model (Dades):** El script que conté la lògica (ex. Timer.cs). No sap que existeix una UI. Només dispara un esdeveniment quan la dada canvia.
2. **View (UI):** Scripts purament visuals (ex. TimerUI.cs). Només tenen mètodes públics com UpdateBar(float value).
3. **Presenter:** L'únic script que coneix ambdós costats(ex. TimerPresenter.cs). Es subscriu a l'esdeveniment del Model i crida el mètode de la View.

Nota de disseny: Centralitzar les subscripcions al Presenter fa que, davant un error, només haguem de revisar un únic lloc per entendre per què la UI no s'actualitza, resolent així el problema de depuració dels esdeveniments. Tamé em permet tindre la camera neta a l'hora d'editar el canvas i previsualitzar (Editor 2D view + Cam).

Annex AT-06: Desenvolupament realitzat i evidències de progrés (Primera fita)

En aquest apartat es documenta l'estat d'execució del projecte a data actual 16/03/2026, detallant les fites assolides, les decisions tècniques que han modelat l'arquitectura i les proves que validen la robustesa del sistema.

A continuació teniu l'enllaç al meu github amb totes les proves validades amb GIFs : https://github.com/Andormix/TFG_UDA_Modular_3D_Party_Game_MVP_in_Unity_URP

Arquitectura de Programari: Desenvolupament Unificat (Hybrid-Core)

Una de les decisions més crítiques del projecte ha estat el disseny d'una arquitectura de xarxa nativa des del primer dia. Aquesta estratègia, denominada Hybrid-Core basada en el model de Servidor Integrat (Integrated Server), permet que el joc funcioni sota una única base de codi tant per a mode solitari com multijugador.

- **Singleplayer com a Host Local:** El mode de joc individual no és una versió simplificada, sinó una execució on la màquina de l'usuari actua simultàniament com a servidor i client. Això garanteix una compatibilitat total de les mecàniques sense necessitat de duplicar codi o realitzar refactoritzacions posteriors.
- **Pla de Contingència i Viabilitat:** Aquesta estructura actua com una assegurança de viabilitat per al TFG. Si el calendari de desenvolupament es veïés compromès, el projecte permet una transició immediata cap a un enfocament *Singleplayer* pur, simplement ometent la integració dels serveis de *Relay*, però mantenint el 100% de la lògica de joc ja desenvolupada.

L'adopció d'una arquitectura de Servidor Integrat (Integrated Server), denominada en aquest projecte com a Hybrid-Core, no és una decisió aïllada, sinó que s'alinea amb els estàndards de qualitat de les grans empreses del sector dels videojocs. Aquesta metodologia és la pedra angular en títols de reconegut èxit on la paritat entre l'experiència en solitari i la multijugador és crítica:

- **Valve (Motor Source):** Títols com *Counter-Strike* o *Half-Life 2* executen un servidor local en segon pla durant les partides *singleplayer*. Això garanteix que les físiques i la lògica de joc es comportin exactament igual que en una partida competitiva online.
- **Mojang (Minecraft):** Des de les versions inicials, el joc va unificar el seu codi de manera que el mode d'un sol jugador és, tècnicament, un servidor privat per a un sol usuari. Això va permetre una escalabilitat sense precedents en el desenvolupament de *mods* i funcionalitats cooperatives.

- **DICE (Frostbite):** L'ús de sistemes autoritatius per a la simulació de vehicles i destrucció d'entorns permet mantenir la coherència visual entre jugadors.

L'elecció d'aquesta arquitectura per a "Provisional" respon a la voluntat d'aplicar aquestes millors pràctiques professionals (Tot i que la complexitat del TFG ha escalat considerablement). Mitjançant aquest enfocament, s'assegura que qualsevol millora en el "Game Feel" o en la lògica de les estacions de servei s'apliqui de forma simètrica a tots els modes de joc, optimitzant els recursos de desenvolupament i garantint una experiència d'usuari robusta i lliure de desincronitzacions.

Full de Ruta Tècnic i Estat de les EPICS

El desenvolupament s'ha estructurat en vuit grans blocs funcionals (EPICs). A continuació es resumeix l'estat d'implementació:

- **EPIC A, B i C (Finalitzades):** S'ha consolidat la base tècnica (moviment WASD, càmera (als gifs utilitzem una altra camera), sistema d'interacció contextual), el bucle principal de comandes (timers, validació d'entregues) i les estacions de transformació d'ítems.
- **EPIC D i E (Finalitzades/En curs):** S'ha implementat el sistema de safata amb gestió d'inventari (slots LIFO) i el sistema diferencial de pagament mitjançant TPV, incloent-hi la generació de preus dinàmics per *run*.
- **EPIC F i G (En desenvolupament):** El sistema de puntuació i economia ja és funcional al costat del servidor, i actualment es treballa en el disseny dels mapes demo (tutorial de Sant Julià i nivell d'Andorra la Vella).

ID	Tasca / Requisit	Prioritat	Estat
EPIC A — Base tècnica i fundació		P0	100%
A1	Configuració projecte Unity + build pipeline	P0	☑ Finalitzat
A2	Player Lateral view perspective (WASD) + col·lisions	P0	☑ Finalitzat
A3	Càmera semi-fixa (seguiment estable)	P0	☑ Finalitzat
A4	Sistema d'interacció contextual (E) + prompt	P0	☑ Finalitzat
A5	Sistema d'ítems (X / X') + pickup/drop	P0	☑ Finalitzat

EPIC B — Comandes i taules		P0	100%
B1	Taula amb comanda + UI sobre taula	P0	☒ Finalitzat
B2	Timer EnEspera (Servir) + fail per temps	P0	☒ Finalitzat
B3	Servir: comprovació d'entrega (X')	P0	☒ Finalitzat
B4	Transició a estat “Requereix Pagament”	P0	☒ Finalitzat
B5	Timer Cobrar + estat Completada/Fallida	P0	☒ Finalitzat
EPIC C — Estacions de transformació		P0	100%
C1	Estació 1 (press/hold) transforma X → X'	P0	☒ Finalitzat
C2	Estació 2 (press/hold) transforma X → X'	P0	☒ Finalitzat
C3	Feedback visual/sonor (SFX/VFX/emissive)	P1	☒ Finalitzat
EPIC D — Safata & Físiques		P0	83%
D0	PickUp and Drop System	P0	☒ Finalitzat
D1	Equipar safata + inventari 3 slots	P0	☒ Finalitzat
D2	Afegir/treure ítems de la safata	P0	☒ Finalitzat
D3	Servir des de la safata (entrega parcial)	P0	☒ Finalitzat
D4	Caiguda d'ítems per xoc (físiques)	P0	☐ Pendent
D5	Ítems a terra recollibles + límit FIFO	P0	☒ Finalitzat
EPIC E — TPV i economia (Diferencial)		P0	100%
E1	Generador de preus dinàmic (seed)	P0	☒ Finalitzat
E2	Panell de preus al mapa	P0	☒ Finalitzat

E3	Memòria curta UI (recordatori temporal)	P0	☑ Finalitzat
E4	Equipar TPV + restriccions d'ús	P0	☑ Finalitzat
E5	UI TPV (increment/decrement/confirmar)	P0	☑ Finalitzat
E6	Lògica d'error TPV + penalització suau	P0	☑ Finalitzat
EPIC F — Score / Propina / Estrelles		P0	50%
F1	Càlcul de Score base + pes de la propina	P0	☑ Finalitzat
F2	Bonus “a la primera”	P0	☑ Finalitzat
F3	Partida 3 min + sistema d'estrelles + retry	P0	☐ Pendent
F4	Ratxa de cobraments perfectes	P1	☐ Pendent
EPIC G — Mapes Demo		P0	0%
H1	Sant Julià (Tutorial: layout + spawns)	P0	☐ Pendent
H2	Andorra la Vella (Avançat: layout + tuning)	P0	☐ Pendent

Sprints setmanals i tauler Kanban (A data del 16/03/2026)

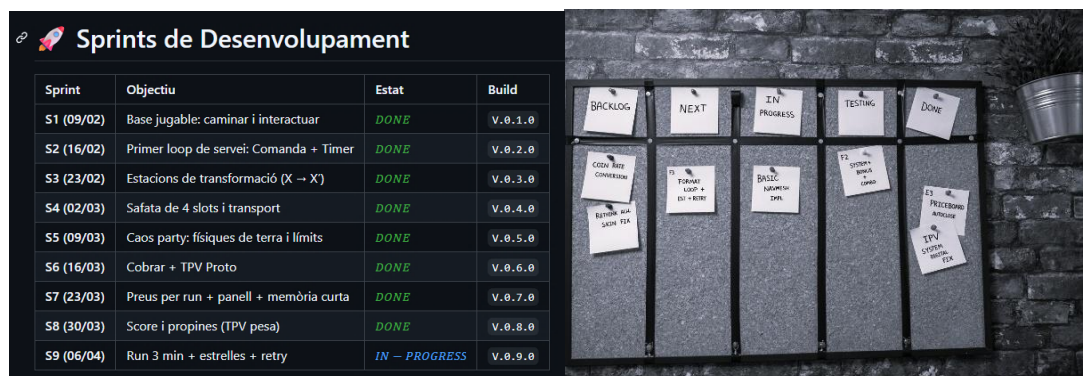


Figura 62. Historial de planificació de sprints de desenvolupament

Font: Elaboració pròpia.

Desenvolupament del Sistema Multijugador (Networking)

La implementació de xarxa utilitza **Unity Netcode for GameObjects (NGO)** sota un model *Host-Authoritative*. Les evidències de progrés en aquest àmbit es divideixen en:

1. **Sincronització d'Estat:** S'ha passat d'una sincronització basada en *JSON Strings* (4096 bytes) a una basada en Structs binaris compactes (10 bytes). Aquesta optimització ha permès una millora del 99.7% en l'eficiència de la xarxa.
2. **Serveis de Lobby i Relay:** S'ha integrat el servei de *Lobby* de Unity amb un sistema de *Heartbeat* per mantenir sessions actives i el servei de *Relay* per permetre connexions online superant barreres de *firewalls* i NAT.
3. **Gestió de Payload:** El sistema ara gestiona la sincronització de customitzacions (skins) mitjançant el *Connection Payload*, assegurant que cada client visualitzi correctament els altres jugadors des del moment del *spawn*.

Evidències de Proves (Testing)

S'han realitzat protocols de prova de connectivitat *Cross-Device* entre equips de sobretaula i portàtils:

- **Test 01:** Validació de la creació de lobby privat, generació de *Join Code* i sincronització de moviments.

Resultat: Èxit en la connectivitat. Com a correcció derivada, es va implementar la refactorització del format de dades de JSON a binari per optimitzar la càrrega de la CPU i l'ample de banda.

Incidències i Decisions Tècniques Preses

Durant el desenvolupament han sorgit diversos reptes tècnics que han requerit ajustos:

- **Problemes de Col·lisió:** Es va detectar que alguns ítems queien a través del terra en ser deixats anar. S'ha corregit ajustant la detecció de col·lisions a *Continuous* en el *Rigidbody* dels objectes de xarxa.
- **Race Conditions:** En la càrrega del lobby es van produir conflictes de sincronització de customitzacions. La solució va ser moure la lògica del *Awake()* al mètode d'aprovació de la connexió del servidor.
- **Optimització de Latència:** Per al *Host*, s'ha garantit latència zero en els *inputs* processant-los en el mateix frame (*Host-Authoritative*), mentre que per als clients s'ha implementat predicció bàsica per ocultar el *ping*.

Observacions del desenvolupador:

Tot i que el calendari inicial preveia una implementació de xarxa en fases més tardanes, s'ha decidit prioritzar la integració total del sistema multijugador online durant les darreres setmanes. Aquesta decisió estratègica respon a la voluntat d'assegurar la

viabilitat tècnica de l'arquitectura *Hybrid-Core* des d'un inici, permetent validar la sincronització de físiques i la gestió de dades binàries en entorns reals de xarxa (*Unity Relay*). Aquesta anticipació en la resolució dels reptes de xarxa garanteix que la recta final del projecte es pugui dedicar exclusivament al disseny i l'acabat estètic, minimitzant el risc de conflictes tècnics estructurals en l'entrega final.

Amb el nucli de xarxa ja estable i operatiu, el projecte entra ara en la seva fase final de producció, la qual se centrarà en:

1. **Confecció de nivells (Level Design):** Implementació dels mapes definitius de Sant Julià de Lòria (Tutorial curt) i Andorra la Vella, aplicant el *layout* i el *tuning* de dificultat específics.
2. **Experiència d'Usuari (HUD i Menús):** Refinament de la interfície gràfica per assolir un acabat visual professional i intuïtiu.
3. **Poliment (Polishing) i Feedback:** Ajust de les sensacions de joc (*game feel*), integració d'efectes visuals (VFX) i sonors (SFX) per elevar la qualitat del producte final.
4. **Optimització de xarxa i model de negoci:** Si tenim temps es treballarà en la reducció del trànsit de dades per minimitzar els costos operatius de *Unity Relay* i es confeccionarà un model d'economia transparent. Aquest sistema complirà amb la normativa de consum actual, mostrant en tot moment la paritat de valor real (Euros a Diamants i Monedes) per garantir la claredat en les transaccions.
5. **Estratègia de llançament i màrqueting:** Es continuarà desenvolupant el pla de comunicació detallat a l'annex final del projecte. L'enfocament se centrarà en l'optimització de la pàgina de Steam (SEO), la creació de contingut orgànic basat en *hooks* de jugabilitat i la gestió de comunitats, amb l'objectiu de maximitzar la visibilitat del títol de cara al seu llançament.

Annex AT-10: Diagrama de GANTT

S'adjunta el diagrama de Gantt generat amb l'eina "Gantt Project", on es visualitzen de forma gràfica les interdependències i el camí crític del projecte.

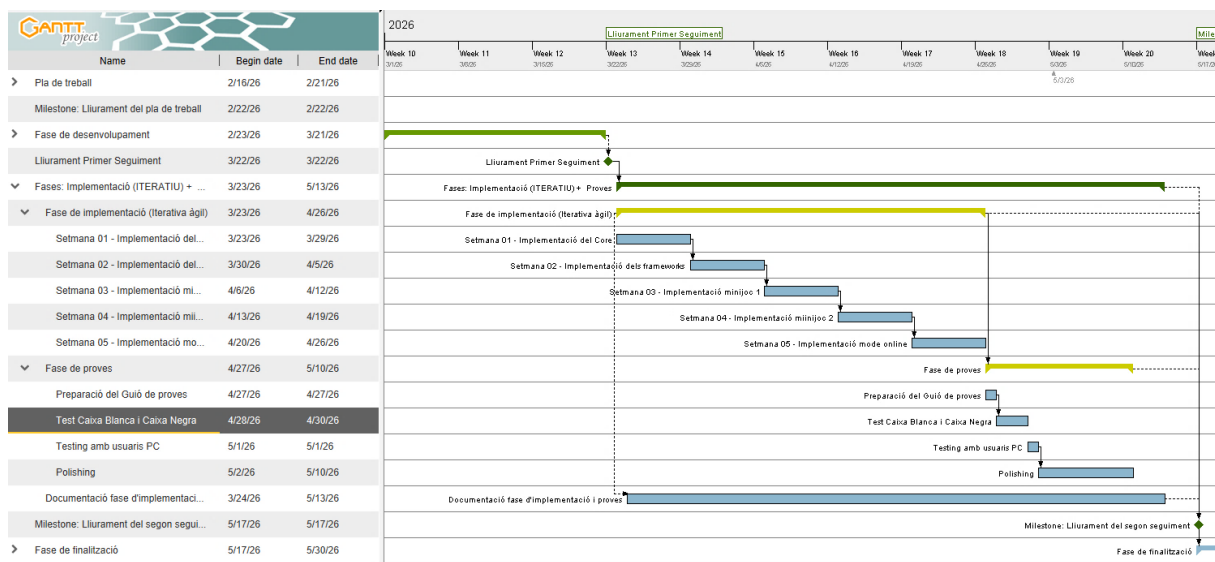


Figura 63. Cronograma i planificació temporal del projecte mitjançant un diagrama de Gantt.

Font: Elaboració pròpia mitjançant l'eina GanttProject.

Taula de tasques i fites

ID	Fase / Sprint	Tasca	Subtasques principals	Durada	Dependència
0	Fase 0	Estudi TFGs anteriors	Revisió de memòries UdA i UOC Open Access.	16/02	-
1	Fase 0	Estudi d'eines	Investigació de motors (Godot, UE5, Unity). Elecció d'URP.	17/02	0
2	Fase 0	Anàlisi d'abast	Definició del MVP (Single Player + 2 minijocs) i limitacions.	18/02	1
3	Fase 0	Reunió docents	Validació de la idea, originalitat i prioritat del Core.	19/02	2

4	Fase 0	Pla de treball	Redacció del pla de la memòria i test Gantt Project.	20/02 - 21/02	3
5	Fita	Lliurament Pla de Treball	Confecció final del document i el Gantt Timeline.	22/02	4
6	Fase 1	Anàlisi de requisits	Recollida de pre-requisits, Core Loop i criteris d'èxit.	23/02 - 01/03	5
7	Fase 1	Disseny	Arquitectura modular, diagrames de flux, UI i estats.	02/03 - 15/03	6
8	Fase 1	Documentació A&D	Confecció contínua de l'esborrany de la memòria (Punts 2 i 3).	23/02 - 21/03	6, 7
9	Fita	Lliurament 1r Seguiment	Empaquetar i lliurar els entregables de la primera fita.	22/03	8
10	Fase 2 (S1)	Implementació Core	Setup GitHub, GameInput, Managers Core i UI base.	23/03 - 29/03	9
11	Fase 2 (S2)	Frameworks	Plantilles UI, transicions, Timer i sistema de Scoring.	30/03 - 05/04	10
12	Fase 2 (S3)	Minijoc 1	Mecàniques, regles, adaptació UI i feedback.	06/04 - 12/04	11
13	Fase 2 (S4)	Minijoc 2	Desenvolupament del segon mapa/minijoc del MVP.	13/04 - 19/04	12
14	Fase 2 (S5)	Mode Online	Investigació i implementació de xarxa (Steamworks).	20/04 - 26/04	9, 10, 11, 12
15	Fase 3	Guia de proves	Definició de casos d'ús i criteris d'acceptació.	27/04	13
16	Fase 3	Testeig Tècnic	Execució de proves de caixa blanca i caixa negra.	28/04 - 30/04	15

17	Fase 3	Testing Usuaris	Playtesting amb usuaris reals i recollida de feedback.	01/05	16
18	Fase 3	<i>Polishing</i>	Correccions i ajustaments finals basats en el feedback.	02/05 - 10/05	17
19	Fase 3	Documentació 2 i 3	Confecció dels apartats 4 i 5 de la memòria.	24/03 - 13/05	10
20	Fita	Lliurament 2n Seguiment	Lliurament dels entregables de la segona fita.	17/05	18, 19
21	Fase 4	Memòria Final	Distribució, manteniment, conclusions i valoració crítica.	17/05 - 04/06	20
22	Fita	Prelliurament Memòria	Enviament de l'estat actual de la memòria.	31/05	-
23	Fita	Lliurament Final	Memòria definitiva + Build final del videojoc (RC).	09/06	21
24	Fase 5	Presentació	Creació del PPT i pràctica de la demo.	09/06 - 14/06	23
25	Fase 5	Soft Skills	Pràctica de comunicació verbal i no verbal.	10/06 - 14/06	21
27	Fita	Lliurament Presentació	Lliurament oficial del document PPT per a la defensa.	14/06	23

Annex AT-11: Sistema de Puntuació

Aquesta secció documenta els **sistemes de puntuació** del nostre videojoc en la seva fase Pre-Alpha.

Dynamic prices per run

- Cada SceneObjectSO té un preu base + rang aleatori.
- Un servidor-autoritatiu SceneObjectPriceManager genera els preus en temps d'execució.
- Els preus són consistents per a tots els clients i visibles a través del Price Board.

TPV payment validation

- El pagament només es pot completar mitjançant la UI del TPV (Enter).
- El servidor valida:
 - Que el jugador tingui el TPV
 - Que l'import coincideixi amb el total de la comanda
 - Que la fase sigui la correcta (AwaitingPayment)
- Es fa un seguiment dels intents de pagament (primer intent vs reintent).

Global team score (Server-Authoritative)

- La puntuació es calcula només al servidor (ScoreManager).
- Tots els clients reben els mateixos valors de puntuació i combo.
- Evita desincronitzacions i trapes (anti-cheating).

Combo Flow System

Escala de combo: $x1 \rightarrow x1.2 \rightarrow x1.5 \rightarrow x2.0 \rightarrow x3.0$

El combo es reinicia quan:

- El temporitzador d'entrega expira
- El temporitzador de pagament expira
- Es perd un client

Scoring Rules (Per Table Cycle)

State 1 — ReadyToRequest (Fast Service) - Si s'agafa la comanda en el primer 20%:

- +50 pts
- (opcional) +15% de temps d'entrega

State 2 — Deliver items (U-Curve)

- Perfecte (0–30%): +100 pts × combo + combo++
- Estàndard (30–80%): +100 pts × combo
- Clutch (80–100%): +200 pts

State 3 — Consuming

- Bonus “Ready to Pay” si el TPV està a prop de la taula:
- +25 pts (només una vegada)

State 4 — Awaiting payment

- Pagament perfecte (primer intent + abans del 50% del temps):
- ×2 punts de pagament

Nota: Aquesta és l'oportunitat perfecta per posar el achievements de STEAM

Annex AT-12 Patrons implementats al projecte

Patrons confirmats (forts)

- **Singleton:** Molts gestors globals son Singleton per conveniència, perquè permet un accés global a un estat compartit. GameManager, SessionCoordinator, GameLobby, GameMultiplayerManager, ScoreManager, CampaignManager, EconomyManager, UserManager, etc.). També he de dir que em sembla que acobla massa llavors crec que és millor no abusar.

- ✓ Assets/02_CODE/Managers
- ✓ Assets/02_CODE/NETCODE/COORDINATOR/Session Coordinator.cs,
- ✓ Assets/02_CODE/Lobby/GameLobby.cs.

- **State Pattern (GoF):** S'ha implementat en el sistema de taules mitjançant les classes Table_StatePattern, IWorkstationState i els seus estats concrets. Aquest patró permet alterar el comportament de la taula de manera dinàmica, com si es tractés d'una classe nova, segons l'estat en què es trobi. Així mateix, facilita la modificació de les lògiques d'interacció i l'escalabilitat del sistema; la incorporació de l'estat «Taula bruta» de forma ràpida i eficient va constatar l'efectivitat d'aquesta arquitectura.

- ✓ Assets/02_CODE/Objects Scripts/Tables/State Pattern.

- **FSM (màquina d'estats):** GameManager amb enum State + NetworkVariable<State> i transicions del cicle de partida.

- ✓ Assets/02_CODE/Managers/GameManager.cs.

- **Observer / Event-Driven:** Ús extens d'event Action, EventHandler, callbacks de NetworkVariable / NetworkList. Útil per desacoblar.

- ✓ Assets/02_CODE/SCORE/ScoreManager.cs
- ✓ Assets/02_CODE/NETCODE/RecipeUI/RecipeUiBus.cs
- ✓ Assets/02_CODE/TUTORIAL + HINTS/TutorialEventChannelSO.cs.

- **MVP-like / Presenter pattern (UI):** Realment tenim molts presenter separant orquestració de UI i lògica de fluxc com PauseUiPresenter MainMenuPresenter, TeamScoreHudPresenter etc. Facilita el testeig entre altres. Alguns presenters calculen dades que poden calcular ells, llavors és una versió que considero “millorada” ja que no tenen que anar al model per exemple per actualitzar el slider si ja tenen la data inicial del temps.

✓ Assets/02_CODE/GUI + HUD.

- **Adapter:** Confirmat amb RecipeManagerOrderServiceAdapter cap a IOrderService. També existeix un altre per un tema de networking per poder comunicar dos interfícies incompatibles.

✓ Assets/02_CODE/Objects Scripts/Tables/State Pattern/Adapter.

- **Coordinator / Facade (de sessió):** Per exemple SessionCoordinator centralitza boot/teardown, EnsureNetworkManager, retorn a menú. Simplifica el codi client.

✓ Path: Assets/02_CODE/NETCODE/COORDINATOR/SessionCoordinator.cs.

- **Data-driven amb ScriptableObject:** Campanya, tutorial, receptes, map pool, configs diversos etc. Molt útils.

✓ Assets/02_CODE/General SO Code

✓ Assets/02_CODE/Campaign/CAMPAIGN MODELS,

✓ Assets/02_CODE/TUTORIAL + HINTS.

- **Host-authoritative networking (patró arquitectònic de xarxa):** Servidor/host com a font d'autoritat; validacions server-side, ServerRpc, NetworkVariable server-write.

✓ Assets/02_CODE/NETCODE

✓ Assets/02_CODE/SCORE/ScoreManager.cs

✓ Assets/02_CODE/NETCODE/Table/TableNetSync.cs.

- **Defensive programming (guard clauses + fail-closed):** Quan vaig fer el hardening. Molt if (...) return; amb validació de context i denegació per defecte en fluxos crítics.
 - ✓ Paths: GameLobby.cs, GameMultiplayerManager.cs, SessionCoordinator.cs, taules/net sync. Etc.

Patrons parcialment presents (mitjans)

Factory-like (lleuger): Hi ha creació encapsulada (SceneObjectSpawn), però no una jerarquia Factory clàssica completa tal com indica els apunts de patrons de Pau Lanao.

- ✓ Assets/02_CODE/NETCODE/SceneObjects/SceneObjectSpawn.cs.

Pooling (local, no framework central): Tinc reutilització/pools en alguns subsistemes visuals/audio, però no un ObjectPool global unificat per tot el joc.

